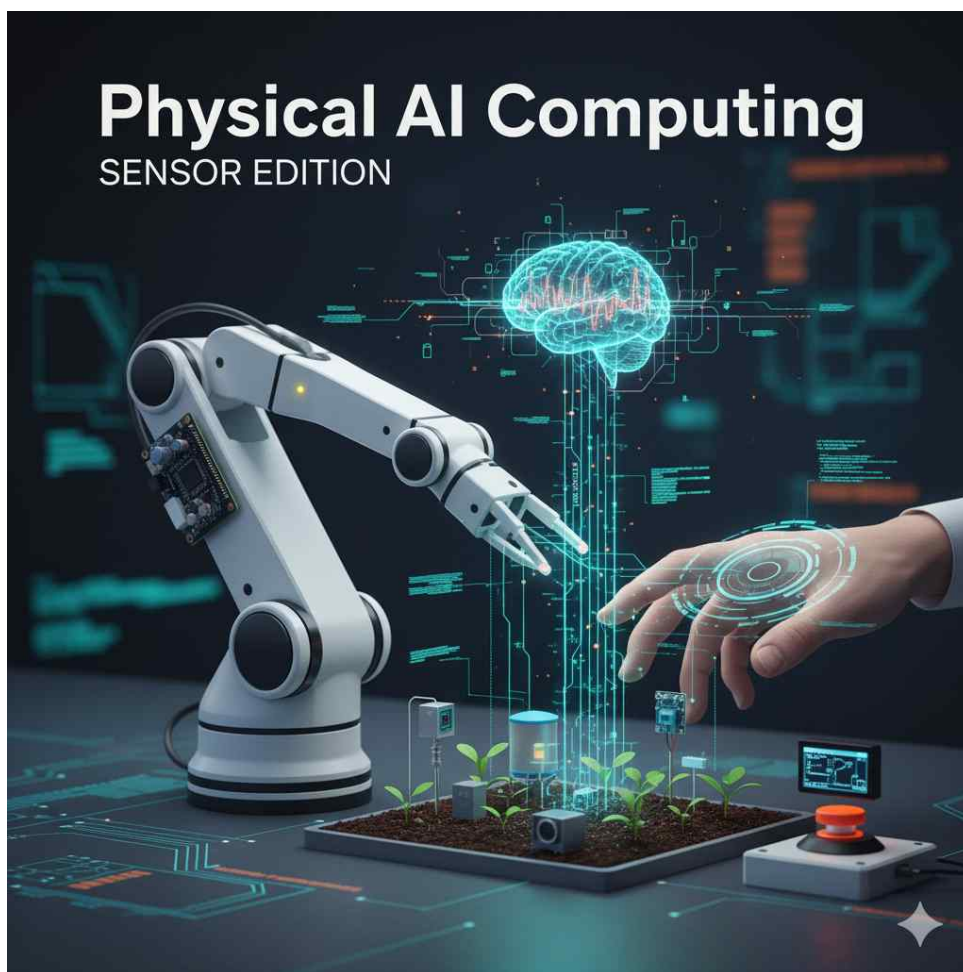


2025년 10월 1일 버전업 된
라즈베리파이OS 최신 버전 13(Trixie) 반영

피지컬AI 컴퓨팅

스마트팩토리편

라즈베리파이와 파이썬으로 만드는 나만의 스마트 팩토리 실습



김 석 진

<제 목 차 례>

Chapter 1. 스마트 팩토리 IT 기초 및 환경 구축	7
1. 스마트 팩토리 및 엣지 컴퓨팅의 이해	7
1) 디지털 두뇌와 물리적 신체의 만남	7
2) 공장의 골든타임을 지키는 기술: 엣지 컴퓨팅(Edge Computing)	8
3) 미래 산업 현장의 피지컬 AI 활용 사례	8
2. 환경 설정 : 라즈베리파이 OS 설치 및 부팅	9
1) 라즈베리파이 OS 설치 방법	9
2) HDMI 모니터, USB 키보드, USB 마우스 있을 때	10
3. 공장 설비 제어의 첫걸음 : GPIO 입출력 제어 원리	14
1) GPIO 핀의 이해	14
2) GPIOZero 라이브러리	17
3) 코드 구성 요소	19
4. 파이썬 프로그래밍 환경	22
1) 로컬 환경 소스 준비	22
2) 개발환경 - 라즈베리파이(Thonny)	23
3) 브레드보드(빵판)	24
4) 저항 기초	24
5) 실습 키트 전체 구성품 한눈에 보기	26
6) 라즈베리파이와 브레드보드(830) 기초 연결	28
Chapter 2. 스마트 팩토리 환경 모니터링	29
1. 센서 기초 : LED와 부저, 버튼	29
1) LED(Light Emitted Diode)	29
2) 버튼 , Tactile Switch	29
3) 부저	30
4) 회로도 및 연결정보	30
5) 기본 실습	32
6) 버튼을 통한 LED 및 부저 제어 실습	34
2. 아날로그 데이터 읽기 : ADC와 아날로그 센서	35
1) ADC(Analog Digital Converter)	35
2) MCP (Multi-Chip Package)	35

3) 스마트 팩토리 적용 실습	37
3. 초음파 센서를 활용한 거리 측정 및 경고	41
1) 개요	41
2) 클래스 : DistanceSensor()	41
3) 연결정보 및 회로도	42
4) 스마트 팩토리 적용 실습	43
4. 인체 감지 및 보안 : PIR 모션 센서를 활용한 작업자 인식	44
1) 개요	44
2) 클래스 : MotionSensor()	44
3) 연결정보 및 회로도	44
4) 스마트 팩토리 적용 실습 - 위험 구역 경고 시스템	46
5. 작업장 환경 측정 : 온습도 센서(DHT11)를 활용한 온도 측정	47
1) 개요	47
2) 라이브러리 : adafruit-circuitpython-dht	47
3) 연결정보 및 회로도	48
4) 스마트 팩토리 적용 실습	50
6. 수동 불량품 카운터 (버튼 + FND)	52
1) 개요	52
2) 클래스 : LEDCharDisplay	52
3) 연결정보 및 회로도	53
4) 스마트 팩토리 적용 실습	55
7. 온습도(DHT11) 센서를 활용한 온습도 디스플레이	56
1) 개요	56
2) 라이브러리 : RPLCD	57
3) 연결정보 및 회로도	59
4) 실습해보기	59
5) 응용해보기	60
6) 스마트 팩토리 적용 실습	61
Chapter 3. 원격 접속을 통한 피지컬AI 기초	62
1. 원격 접속 환경 구축 (라즈베리파이 엣지 설정)	62
1) 원격 제어용 소프트웨어 준비 및 핫스팟 설정	62
2) [실습] 모니터 없이 설치하기 : 헤드리스(Headless) 원격 접속 비법	64

3) VNC 원격 데스크톱 연결과 OS 화면구성 기초	71
4) 원격 작업 소스 준비	75
2. 본격적인 원격 개발의 시작 : VS Code기반 개발 환경 구축	76
1) VS Code 특징 및 설치	76
2) 접속 환경 설정 및 SSH 연결	77
3) 원격 환경에 실행 버튼 달기	78
4) 꼬임 방지를 위한 독립 구역 : 가상 환경(venv) 구축 및 패키지 설치	80
Chapter 4. 물류 자동화(AGV) 기초 주행	83
1. AGV 뼈대 및 모터 조립	83
1) AGV 뼈대 조립을 위한 준비물	83
2) 한 장으로 끝내는 회로도 : AGV 전체 하드웨어 회로도	84
3) 두뇌와 신경망 연결 : 핀맵(Pin Map)과 GPIO 할당표	84
4) AGV 뼈대, 프레임 제작하기	86
5) 모터 드라이버와 모터 연결정보	89
6) [실습] AI-Rover의 상태를 알리는 빛 : LED 연결	92
7) [실습] AGV의 목소리 : 부저(Buzzer) 연결	92
8) [테스트] 감각이 살아있나요? : 부품별 정상 작동 상태 점검	92
2. 충돌. 안돼요~ : 전후방 초음파 센서를 활용한 작업자 감지 및 긴급 제동 시스템 (AEB) 실전 코딩	94
1) AEB (Autonomous Emergency Braking, 긴급 제동 시스템)	94
2) 하드웨어 핀맵(Pin Map) 복습	94
3) 스마트 팩토리 적용 시나리오	94
4) source code	94
3. 덜컹거림 없는 주행의 비밀 : 노이즈를 극복하기 위한 센서 데이터 필터링 기초 ...	97
1) 센서 노이즈(Noise)란 무엇인가요?	97
2) 이동 평균 필터(Moving Average Filter)의 원리	97
3) Source Code (필터가 적용된 초음파 센서)	97
Chapter 5. Vision AI 기반 품질 검수(머신 비전)	99
1. 설비의 시신경 깨우기 : Picamera2 라이브러리를 활용한 실시간 영상 스트리밍 및 기초 설정	99
1) 스마트 팩토리 와 머신 비전(Machine Vision)	99
2) 하드웨어 연결 및 환경 설정	99

3) 라이브러리 : Picamera2	100
4) 색깔이 뒤집혔어요! : OpenCV만의 독특한 색 표현 방식(BGR)	101
5) Source Code	101
2. [실습] 인공지능의 진짜 맛을 보다 : 사전 학습(Pre-trained) YOLO 모델로 작업자 인식하기	103
1) 개요 : 준비된 인공지능, 사전 학습(Pre-trained) 모델	103
2) YOLO	103
3) 모델 준비 : 초경량 Nano 모델 선택	103
4) Source Code : AGV 인공지능 시력 테스트	103
3. 인공지능의 눈과 로봇의 몸 연결하기 : 탐지된 객체 추출 및 판별 로직	105
1) 화면에 그리는 것을 넘어, 데이터로 읽어오기	105
2) [실습] 스마트폰을 불량품으로 인식하는 비전 검수기 만들기	105
3) 사전 학습(COCO) 클래스 활용	106
Chapter 6. 공장 중앙 센터 웹 모니터링	107
1. 스마트 팩토리와 웹 서버의 만남 : Flask 기초	107
1) 왜 Flask(플라스크)인가?	107
2) 환경 준비 : Flask 설치	107
3) [실습] 세상에서 가장 작은 관제 서버 열기 (Hello, Factory!)	107
4) 접속 및 확인 : 내 스마트폰으로 서버 들어가기	108
2. 실시간 데이터 스트리밍 : 온습도 대시보드	109
1) 화면이 깜빡이지 않는 마법 : 비동기 통신(AJAX/Fetch)의 이해	109
2) [실습] 스마트폰에 현장 온습도 대시보드 띄우기	109
3) 실행 및 결과 확인	110
3. 스마트폰이 운전대가 되다 : AGV 원격 제어	111
1) 웹 버튼으로 모터를 돌리는 원리	111
2) [실습] 무선 조이스틱 웹 서버 구축하기	111
3) 실행 및 무선 주행 테스트	112
4. 1인칭 주행 영상(FPV) 스트리밍 : 이동형 CCTV	113
1) 영상 스트리밍의 원리 : 초고속 플립북(MJPEG)	113
2) [실습] 스마트폰에 실시간 카메라 영상 띄우기	113
3) 실행 및 결과 확인	115
Chapter 7. 통합 스마트 팩토리 시스템 구현	116

1. 융합의 시작 : 통합 시스템 아키텍처 설계	116
1) 왜 역할을 나누는가? : 분산 제어(Distributed Edge Computing)의 이해	116
2) 2인 1조 역할과 책임 정의 : IT 파트 vs OT 파트	116
3) 엣지와 엣지가 대화하는 법 : 동일 네트워크와 HTTP 통신	116
2. [조원 B] 현장(OT) 파트 : AI 비전 검수 및 공장 환경 모니터링	117
1) 엣지 노드(Edge Node)의 임무	117
2) 통신 준비 : 파트너(조원 A)의 IP 주소 확인하기	117
3) 고정형 검수 스테이션 구축 및 센서(온습도 DHT11, I2C LCD) 통합	117
4) 컨베이어 벨트 위의 눈 : YOLO(또는 색상 추적)를 활용한 실시간 불량품 판별	118
5) Source Code : 현장 엣지 통합 코드	119
3. [조원 A] 관제(IT) 파트 : 통합 대시보드 및 AGV 원격 제어	122
1) 수동 조작과 자동 안전의 결합 : 원격 제어(Teleoperation)와 긴급 제동(AEB)	122
2) 대통합의 장 : Flask 기반 마스터 대시보드 구축	122
3) [실습] 조원 A 마스터 코드 실행 : it_control_A.py	122
4. [통합 실습] 마이크로 스마트 팩토리, 시스템 가동!	126
1) 접속 준비 : 두 대의 라즈베리파이 네트워크 연동 및 IP 주소 교환	126
2) 시스템 런칭 : 스마트폰 브라우저로 접속하는 우리 조만의 공장 관제 화면	126
3) 융합 시나리오 구동 : AGV 물류 이송과 비전 품질 검수의 완벽한 앙상블	126
4) 현장 트러블슈팅(Troubleshooting) : 데이터 통신 지연 및 스트리밍 끊김 해결법	127
5. 에필로그 : 피지컬 시가 바꿀 우리의 산업 현장	128
1) 장난감을 넘어 현장으로 : 스마트 팩토리 실무 적용 아이디어 발굴	128

Chapter 1. 스마트 팩토리 IT 기초 및 환경 구축

1. 스마트 팩토리 와 엣지 컴퓨팅의 이해



1) 디지털 두뇌와 물리적 신체의 만남

우리는 지금까지 챗GPT나 유튜브 추천 알고리즘처럼 컴퓨터 화면 속의 인공지능(AI)에 익숙합니다. 하지만 이제 다음 단계는 이 똑똑한 인공지능이 모니터 밖으로 나와 물리적 현실 세상(Physical Real World)과 직접 상호작용하는 것입니다. 스마트 팩토리를 움직이는 핵심 개념은 다음 4가지의 발전 과정으로 이해할 수 있습니다.

구분	핵심 개념	비유	데이터 처리	일상생활 예시	산업 · 로봇틱스
01 IoT 사물인터넷	모든 사물이 인터넷에 연결	신경망 단순 전달	센서 데이터를 수집해 서버로 전송만 한다(단순 전달)	스마트 온도계 측정 온도만 앱에 그대로 표시	무선 센서 노드 공장의 온도 습도 대시보드 표시
02 AIoT 지능형 사물인터넷	연결된 사물이 스스로 생각	두뇌 + 신경망 판단력 추가	수집 데이터를 AI가 분석해 상황을 판단(엣지 컴퓨팅)	지능형 홈 카메라 사람·사물 구분 후 알림	시각 품질 검사 시스템 불량품을 AI가 자동 판독
03 피지컬 컴퓨팅 PHYSICAL COMPUTING	디지털 코드로 현실 세계를 제어	신체 + 반사신경 감각과 행동	센서 입력값에 따라 코딩된 규칙대로 즉각 물리 제어(조건문 기반)	자동문 · 스마트 가로등 센서 감지 → 모터·빛 작동	기초 단계 pi-Rover · 1세대 AGV 코딩된 규칙대로만 주행
04 · GOAL GOAL 피지컬 AI PHYSICAL AI	신체를 가진 AI가 스스로 행동	완전한 생명체 지능 + 행동	복잡한 현실을 인지하고 스스로 판단·적응하여 행동	자율주행 자동차 최신형 AI 비전 청소기	자율 탐사 로봇 AI-Rover 지능형 협동 로봇(Cobot)

- ☐ IoT (사물인터넷) : 모든 사물이 네트워크로 인터넷에 연결되어 센서를 통해 데이터를 수집하는 단계입니다. 수집(Collect), 연결(Connect), 제어(Control)의 3대 요소를 바탕으로 단순한 데이터 통로(신경망) 역할을 합니다.
- ☐ AIoT (지능형 사물인터넷) : 연결만 되어 있던 사물(IoT)에 인공지능 두뇌를 달아준 형태입니다. 수집된 데이터를 AI가 학습하고 분석하여 사물이 스스로 상황을 똑똑하게 판단(두뇌+신경망)합니다.
- ☐ 피지컬 컴퓨팅 (Physical Computing) : 디지털 코드로 현실 세계를 제어하는 기술입니다. 마이크로컨트롤러(라즈베리파이 등)가 센서(입력) 값에 따라 정해진 프로그래밍 논리대로 모터나 LED 같은 액추에이터(출력)를 물리적으로 제어합니다.

- **피지컬 AI (Physical AI)** : 정해진 규칙대로만 움직이던 기계(피지컬 컴퓨팅)에 자율성과 학습 능력을 부여한 형태입니다. 신체를 가진 AI가 복잡한 현실 환경을 인식하고 스스로 적응하여 목표를 달성하는 완전한 지능형 생명체와 같습니다.

2) 공장의 골든타임을 지키는 기술: 엣지 컴퓨팅(Edge Computing)

스마트 팩토리와 온디바이스 AI(On-Device AI) 시스템을 구축할 때 가장 중요한 핵심은 '어디서 생각하느냐'입니다.

- 클라우드 AI의 한계 : 데이터를 중앙 데이터센터로 보내서 처리하는 방식은 성능은 좋지만, 인터넷이 끊기거나 데이터 왕복에 따른 지연(Latency)이 발생할 수 있습니다.
- 엣지 AI의 필요성 : 시속 100km로 달리는 자율주행차나, 프레스 기계가 내려오는 공장 라인에서는 서버의 응답을 기다릴 시간이 없습니다. 0.01초 만에 현장에서 즉시 판단하고 제어해야 사고를 막을 수 있습니다.
- 이처럼 현장에 있는 기기(Edge)가 근접한 곳에서 자체적으로 데이터를 분석하고 즉시 판단(이상 징후 탐지 등)을 수행하는 것이 바로 엣지 컴퓨팅이며, 우리가 공장 자동화에 소형 컴퓨터인 라즈베리파이(엣지 디바이스)를 사용하는 가장 큰 이유입니다.

3) 미래 산업 현장의 피지컬 AI 활용 사례

우리가 이번 과정에서 배우게 될 센서 데이터 수집과 모터 제어 기술은 다음과 같은 미래 산업의 핵심 기술로 직결됩니다.

- **스마트 팩토리 (예지 보전)**: 기계 모터의 미세한 진동 소리나 센서 데이터를 AI가 분석하여, 기계가 고장 나기 전에 미리 "베어링 교체가 필요하다"라고 경고를 보내는 시스템입니다.
- **스마트 물류 (로지스틱스)**: 거대한 물류센터에서 작업자나 다른 장비와 부딪히지 않고 자재를 나르는 키바(Kiva) 로봇이나 무인 이송 로봇(AGV)에 적용됩니다.
- **자율주행 및 작업장 안전**: 공장 내에서 도로 위 장애물이나 작업자를 인식하여 인지-판단-제어를 수행하며 스스로 멈추고 달리는 거대한 피지컬 AI 로봇을 구현합니다.



그림 4 미래산업현장 피지컬A 활용 사례

2. 환경 설정 : 라즈베리파이 OS 설치 및 부팅

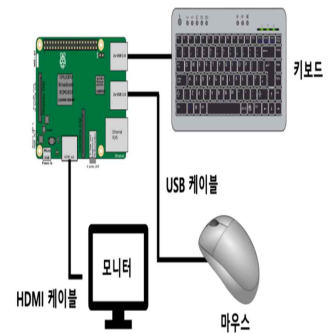
현장 설비에 부착되는 제어기에는 모니터나 입력 장치를 상시 연결해 두기 어렵습니다. 따라서 HDMI 모니터, USB 키보드, USB 마우스 없이 노트북의 모바일 핫스팟을 활용하여 원격으로 연결하는 헤드리스(Headless) 방식을 사용합니다

1) 라즈베리파이 OS 설치 방법

① HDMI 모니터가 있을 경우 (전통적 방식)

HDMI 모니터 및 연결선(Micro HDMI 젠더), USB 마우스, USB 키보드가 모두 준비된 경우.

일반 컴퓨터	라즈베리파이
① imager 다운로드 및 설치 ② micro SD 카드 준비 ③ SD Card에 라즈베리파이 OS 설치	④ SD Card 장착 ⑤ 부팅(전원케이블 연결) ⑥ 와이파이 연결(로컬 네트워크) ⑦ 라즈베리파이 환경설정



② HTMI 모니터가 없을 경우 (헤드리스(Headless)¹⁾방식)

○ 주로 노트북을 활용하는 경우 아래와 같이 진행

일반 컴퓨터	라즈베리파이
① 모바일 핫스팟 설정 ② imager 다운로드 및 설치 ③ micro SD 카드 준비 ④ SD Card에 라즈베리파이 OS 설치 ⑦ 모바일 핫스팟 활성화 ⑨ putty 다운로드 및 설치, 실행 ⑩ 라즈베리파이 vnc 활성화(putty 창에서 작업) ⑫ vnc viewer 설치, 라즈베리파이 접속, 확인	 ⑤ SD Card 장착 ⑥ 부팅(전원케이블 연결) ⑧ 와이파이 연결(모바일 핫스팟) ⑪ 재부팅 ⑬ 라즈베리파이 환경설정

1) Headless : HDMI 모니터, USB 키보드, USB 마우스 없이 연결하는 것을 말함.

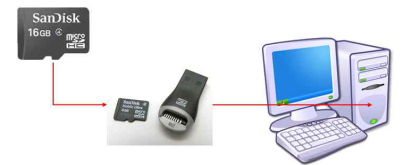
2) HDMI 모니터, USB 키보드, USB 마우스 있을 때

노트북이 없고 일반 데스크탑 컴퓨터가 있는 환경에서 작업할 경우입니다. 작업 과정은 모니터가 없는 경우에 비해서는 단순하지만 네트워크 설정을 위해서는 무선 LAN 동글이가 필요합니다.

일반 컴퓨터	→	라즈베리파이
① SD 카드 준비 ② 설치용 이미지 프로그램(imager) 다운로드 및 설치 ③ SD Card에 라즈베리파이 OS 설치		④ SD Card 장착 ⑤ 준비(키보드, 마우스, 모니터 연결) ⑥ 부팅(전원케이블 연결) ⑦ 와이파이 연결

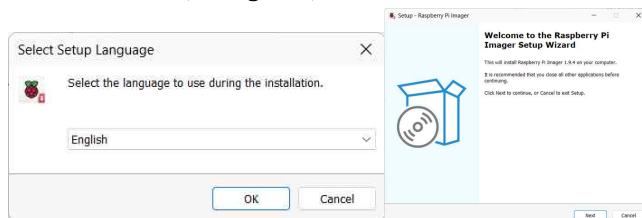
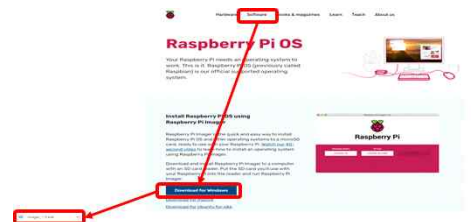
① micro SD card 및 SD card reader 준비

- ☐ micro SD card : 최소 8GB, 추천 16GB 이상
- ☐ SD 메모리 카드 연결
 - SD카드를 USB 젠더에 삽입
 - USB 젠더를 컴퓨터 USB 포트에 삽입
 - SD Card를 직접 삽입할 수 있는 노트북도 있습니다.



② OS 설치용 이미지 프로그램 다운로드 및 설치

- ☐ 라즈베리파이 OS 설치용 imager 다운로드
 - 홈페이지 : <https://www.raspberrypi.org>
 - software 를 클릭
 - Download for windows선택
- ☐ imager 설치
 - 언어 선택, English, Next 클릭



③ Micro SD card에 OS 설치 (Imager 굽기)

□ 1. 윈도우 - [시작] - Raspberry Pi - Raspberry Pi Imager 실행 혹은 바탕화면 Imager 실행

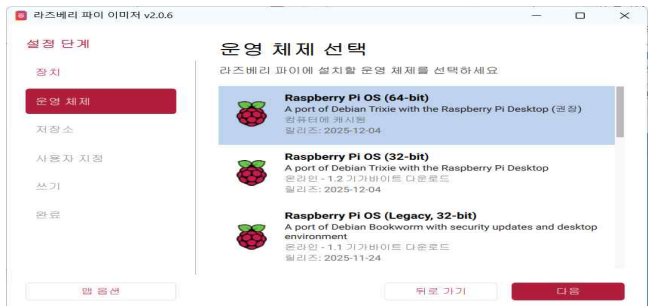
□ 2. 장치 선택

○ 사용 라즈베리파이 모델 선택



□ 3. 운영체제 선택

; 64bit 선택(매우 중요) PyTorch, YOLO 사용시 32비트는 에러날 수 있음.



□ 4. 저장장치 선택



□ 5. OS 커스터마이제이션

○ 호스트 이름 입력

- 예 zeroToAI, 공백없이 영어로 다른 이름 가능. 보통 본인의 이니셜, raspberry, pipi 등등



□ 6. OS 커스터마이제이션

○ 지역 선택

- Seoul(South Korea)
- Ais/Seoul



□ 7. OS 커스터마이제이션

○ 사용자 이름 선택

- 너무 길거나 어려운 비밀번호는 자제
- 예) 아이디 : pi, 비밀번호 : 12345678



□ 8. OS 커스터마이제이션

- 사용자 지정 : Wi-Fi 선택
- SSID : 위에서 지정한 모바일 핫스팟 ID, 비밀번호
(예 : hanuri / 12345678)

□ 9. OS 커스터마이제이션

- SSH 사용여부 - 활성화(★ 필수)
- 외부 원격접속 허용
- tennet 접속(putty 접속시 필수)

□ 10. OS 커스터마이제이션

- 라즈베리파이 커넥트

- 원격 접속 통계, 라즈베리파이 OS 관련 통계 수집 등 기록 전송 여부, **비활성**

□ 11. 이미지 기록 내용 요약 - 내용 확인

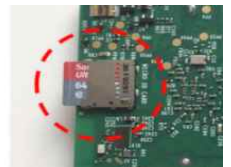
□ 12. 내용물 삭제 경고

□ 13. 완료

□ 윈도우에서 디스크 삽입 창이 뜨면 취소..

④ Micro SD card를 라즈베리파이에 장착

- SD카드를 라즈베리파이 뒷면의 소켓에 삽입
- 그림처럼 sd card의 로고가 보이도록 삽입

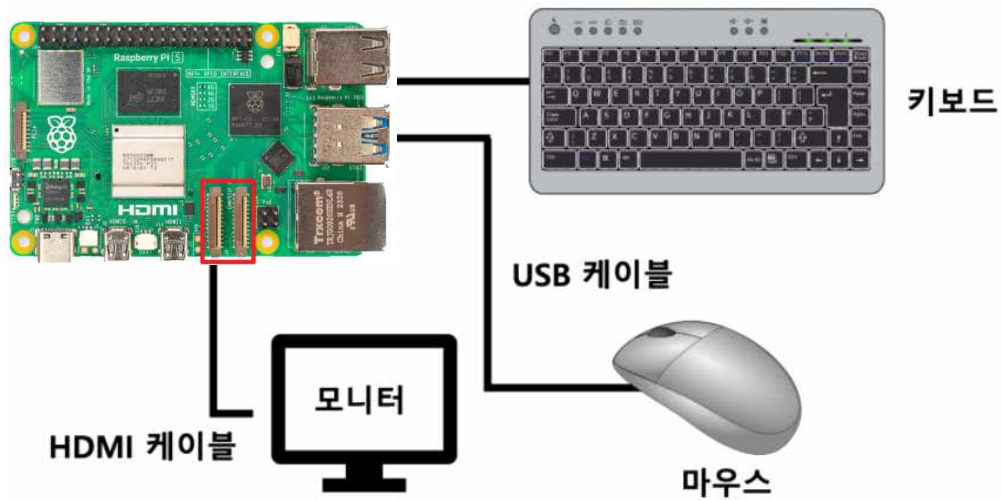


⑤ 준비 (키보드, 마우스, 모니터 연결)

; 부팅 후 와이파이 등의 초기 설정을 위해 모니터, 키보드, 마우스가 필요함.

○ HDMI 모니터 연결

○ USB 키보드, USB 마우스 연결



⑥ 부팅 (전원 케이블 연결)

☐ 부팅이 시작됩니다. 시간이 걸릴 수 있습니다.

☐ 부팅 후 첫 시작화면이 보이면 부팅 성공입니다.

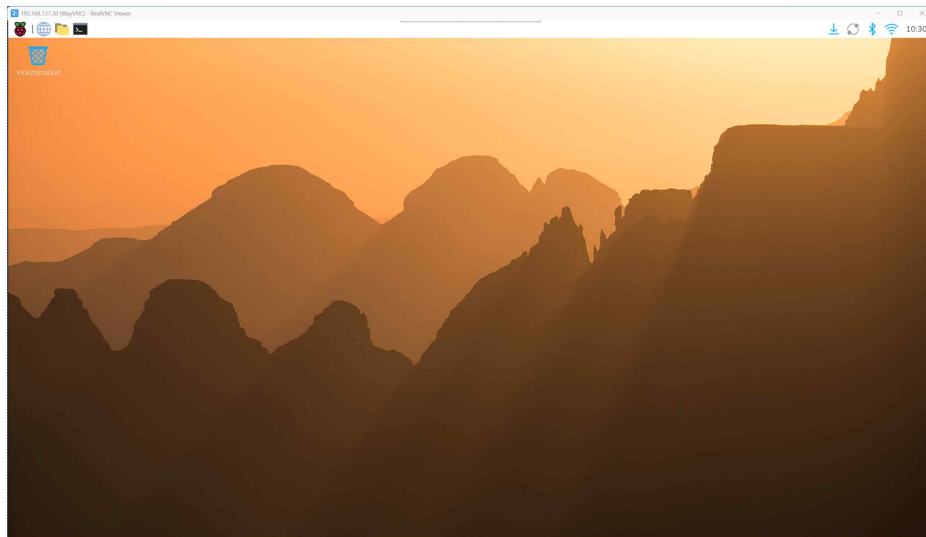


그림 24 라즈베리파이os 초기화면

3. 공장 설비 제어의 첫걸음 : GPIO 입출력 제어 원리

스마트 팩토리의 라즈베리파이는 센서로 현장을 읽어 들이고, 모터나 경고 LED를 작동시켜 물리적인 조치를 취합니다. 이러한 소프트웨어와 하드웨어의 소통은 라즈베리파이 보드에 솟아있는 GPIO 핀을 통해 이루어집니다.

1) GPIO 핀의 이해

① 다양한 언어로 제어

다양한 프로그래밍 언어 및 도구를 사용하여 라즈베리파이의 GPIO 핀을 제어 할 수 있다.

프로그래밍 언어	스크래치	C언어	파이썬
사용 라이브러리	Pi GPIO	wiringPi pigpio	RPi.GPIO gpiozero

본 교재에서는 파이썬 언어와 gpiozero 라이브러리를 주로 사용합니다.

② GPIO(General Purpose Input/Output : 다목적(범용) 입출력)

라즈베리파이에는 총 40개의 물리적 핀이 존재하며, 그중 26개가 사용자가 직접 프로그래밍하여 제어할 수 있는 GPIO(General Purpose Input/Output) 핀입니다.

공장의 장치나 장비들을 안정적으로 구동하기 위해서는 5V 및 3.3V의 전원 공급 핀과, 회로의 기준점이 되는 여러 개의 GND(공통 접지) 핀을 올바르게 이해하고 결선해야 합니다.

코드 상에서 출력 핀으로 지정된 GPIO는 3.3V(높음) 또는 0V(낮음)의 전압을 내보내어 장치를 켜거나 끄고, 입력 핀으로 지정된 GPIO는 외부 센서에서 들어오는 전압 상태를 읽어 들입니다.

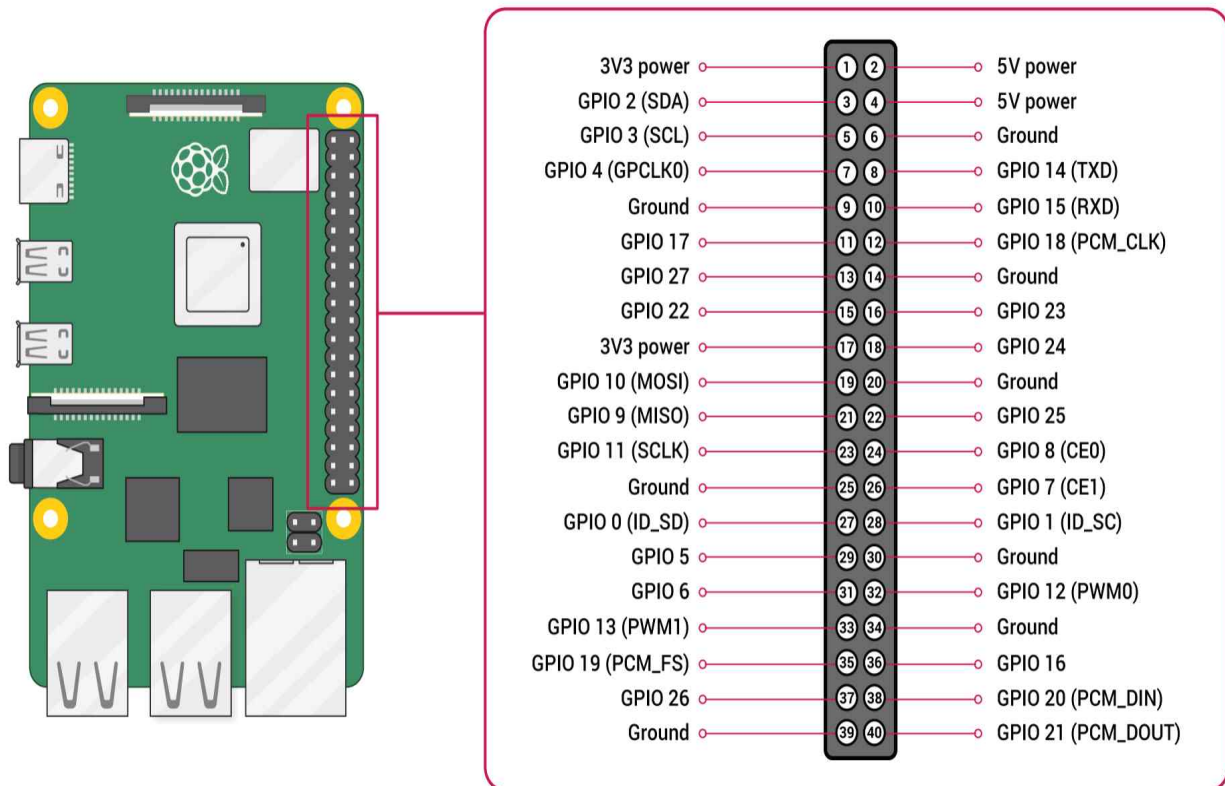
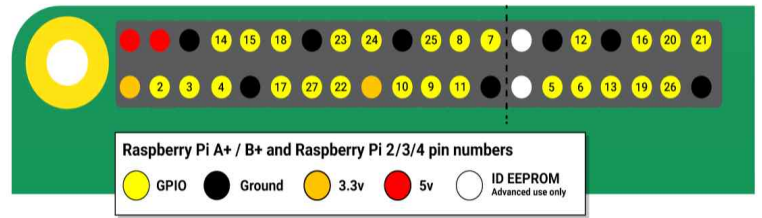


그림 25 라즈베리파이 GPIO핀 번호

GPIO 핀 번호체계는 순차적이지 않다. 위 그림의 가운데 1 ~ 40번은 물리적 핀번호로 순차적으로 번갈아 배열되어 있고 각각의 물리적 핀번호는 BCM(Broadcom chip-specific pin numbers) 핀번호로 불리운다. 즉 물리적 핀번호 11번은 GPIO 17, 물리적 핀번호 13번은 GPIO 27이다.

③ 라즈베리파이 GPIO 확인 : pinout



④ 전압

2개의 5V 핀과 2개의 3V3 핀과 다수의 접지(Ground) 핀 (0V)이고 나머지 핀은 모두 범용 3V3 핀으로, 출력은 3V3으로 설정되고 입력은 3V3 허용한다.

⑤ 출력

출력 핀으로 지정된 GPIO 핀은 높음(3V3) 또는 낮음(0V)으로 설정한다.

⑥ 입력

입력 핀으로 지정된 GPIO 핀은 높음(3V3, 5V) 또는 낮음(0V)으로 읽을 수 있다. 내부 풀업 또는 풀다운 저항을 사용하면 보다 쉽게 사용할 수 있는데, 핀 GPIO 2 및 GPIO 3에는 고정 풀업 저항이 있지만 다른 핀의 경우 소프트웨어상에서 구성한다.

□ 핀 테이블

○ GPIO 핀 0과 1(물리적 핀 27과 28)은 보드에 존재하

지만 고급 사용을 위한 핀이라 일반적으로는 사용하지 않는다.

기능	명칭	핀 갯수			물리적 핀번호	표시
전압	5V	2			2, 4	빨강
	3V3	2			1, 17	주황
Ground	GND	6			6, 9, 14, 20, 25, 30, 34, 39	검정
GPIO	GPIO 핀번호	26	일반	13	4, 5, 6, 12, 13, 17, 18, 22, 23, 24, 25, 26, 27	노랑
			추가 특수기능	13	2, 3, 9, 7, 8, 16, 10, 11, 14, 15, 19, 20, 21	

⑦ 기타

GPIO 핀은 간단한 입력 및 출력 장치뿐만 아니라 주변 장치와도 다양한 입출력을 지원한다.

☐ PWM (펄스 폭 변조)

- 모든 핀에서 소프트웨어 PWM 사용 가능.
- GPIO 12, GPIO 13, GPIO 18, GPIO 19에서 하드웨어 PWM 사용 가능.

□ 시리얼 통신

- TX(Transmitt : 송신) : GPIO14
- RX(Receive : 수신) : GPIO15

☐ SPI(Serial Peripheral Interface)

- 1:N 연결, N이 높아질수록 물리적 연결이 많아져야함
- 동시에 송수신 가능, I2C 대비 속도가 빠르다

	SPI 0	SPI 1
MOSI (Master Out Slave In)	GPIO 10	GPIO 20
MISO (Master In Slave Out)	GPIO 9	GPIO 19
SCK (clock)	GPIO 11	GPIO 21
CE 0	GPIO 8	GPIO 18
CE 1	GPIO 7	GPIO 17
CE 2		GPIO 16

☐ I²C(Inter Integrated Circuit)통신

○ N개 master, M개 slave (최대 127개) 구조

○ Clock 기능을 사용하기 때문에 전송시간 자유로우나 긴 data에는 부적합

데이터 통로 (SDA, Serial Data Line)	제어 클럭 (SCL, Serial Clock line)
GPIO 2	GPIO 3

⑧ 아두이노 vs 라즈베리파이 GPIO

구분	아두이노 UNO	라즈베리파이 5
GPIO 갯수	디지털 14개, 아날로그 6개	40
아날로그 입력	지원 (ADC 내장)	지원안함
PWM 출력	디지털 핀 이름 앞에 ~ 붙은 6개 (3, 5, 6, 9, 10, 11)	하드웨어 지원 : 4개 나머지는 소프트웨어 지원 가능
전압	5V	5V , 3.3V
멀티태스킹	단일작업	멀티태스킹 가능
인터페이스	SPI, I2C, UART 지원	SPI, I2C, UART 지원
운영체제	없음	리눅스 기반 OS

2) GPIOZero 라이브러리

① 특징

- ☐ 라즈베리파이에서 GPIO 핀을 쉽게 제어할 수 있도록 도와주는 파이썬 라이브러리
- ☐ LED, 버튼, 모터, 센서 등 다양한 하드웨어를 간단한 코드로 컨트롤할 수 있습니다.
- ☐ 라즈베리파이 OS에 기본 내장되어 있어 따로 설치할 필요 없음.

② 홈페이지 : <https://gpiozero.readthedocs.io/en/latest/index.html>

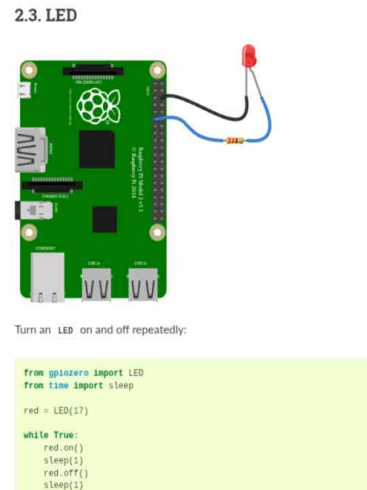
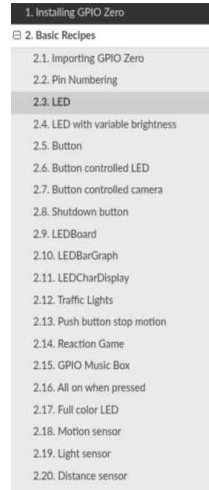
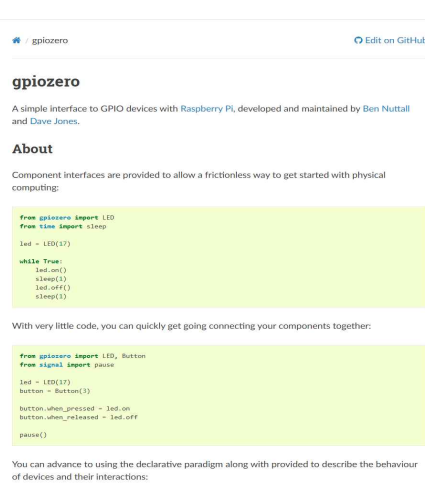
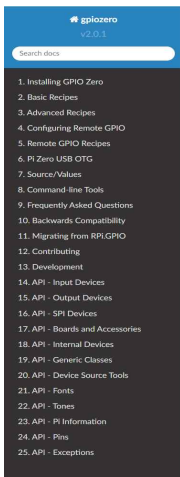
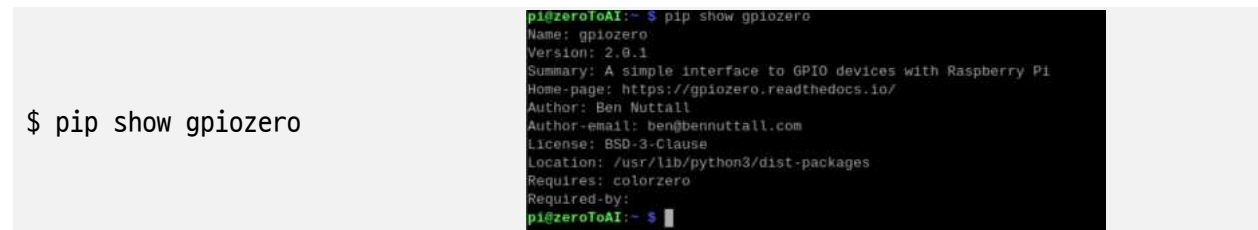


그림 28 gpiozero 홈페이지

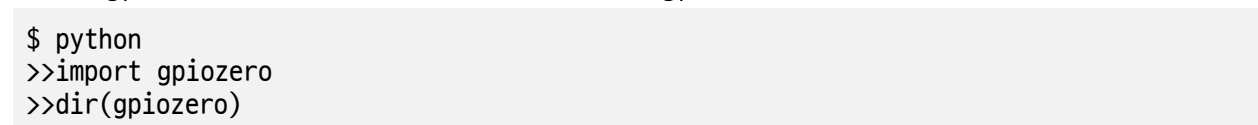
그림 29 2.3 LED 회로도와 예제 코드

☐ gpiozero 버전 확인



☐ 관련 property & method

○ gpiozero 사용 가능한 클래스 확인 : `dir(gpiozero)`



- 글자 앞 뒤로 `_` (밑줄이 2개) 있는 것은 파이썬 특수 property(변수)나 method(함수)들
- `_` (밑줄) 1개가 붙은 것은 사용자가 사용 가능한 property(변수)나 method(함수)들
- a~z 순으로 이름 있는 것이 property(변수)나 method(함수)들

○ LED 클래스의 사용 가능한 메소드들 확인 : `dir(LED)`



○ 메서드 사용은 객체 생성 시 만든 '**객체명.메서드명**' 사용, 구분자 dot(.)
예) LED 객체 뒤에 `on()/off()` 메서드 사용.

```

from gpiozero import LED          # gpiozero 라이브러리에서 LED 클래스 불러옴

led = LED(17)                     # LED 클래스로 led 객체 생성

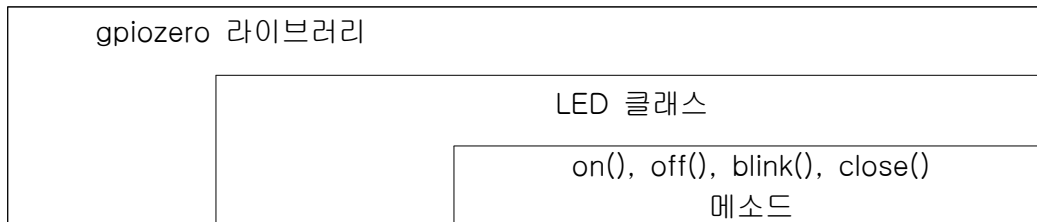
led.on()                          # led 객체에 on 메서드 사용
led.off()                         # led 객체에 off 메서드 사용

```

이후 사용하는 클래스에 dir(import한 클래스명) 사용하여 클래스에 따른 사용가능한 변수와 메서드를 확인하는 것이 매우 중요합니다.

③ 사용 클래스 및 주요 메서드

- gpiozero 라이브러리 안에 LED, DistanceSensor, Button, Motor 등의 많은 클래스가 있으며 각각의 클래스안에는 공통으로 사용될수 있거나 독립적인 많은 method(함수)가 있다.



클래스	기능	주요 메서드
LED	led 제어	on(), off(), blink(), close()
TonalBuzzer	부저 제어	play(), stop()
DistanceSensor	초음파센서 제어	when_in_range(), when_out_of_range() wait_for_range(), wait_for_out_of_range(), close()
LineSensor	라인 트레이서 제어	when_line(), when_no_line() wait_for_line(), wait_for_no_line()
Motor	모터 제어	forward(), backward(), reverse(), stop()
Robot	모터 제어	

3) 코드 구성 요소

○ 메서드 사용은 객체 생성 시 만든 '객체명.메서드' 사용 (구분자 점(dot, .))

```
# file: code_1.py                                # code_1 모듈 파일

# 내장 라이브러리
from time import sleep                            # time 라이브러리에서 sleep 함수 불러오기
from signal import pause                          # signal 라이브러리에서 pause 함수 불러오기

# 외부 라이브러리
from gpiozero import LED                          # gpiozero 라이브러리에서 LED 클래스 불러오기

led1 = LED(17)                                    # LED 클래스로 led1 객체 생성
led2 = LED(27)                                    # LED 클래스로 led2 객체 생성

count = 0                                         # count 변수

print('Press Ctrl+C to exit')

def blink_led():                                  # blink_led() 함수 정의
    led1.on()                                     # led1 객체의 on 메서드 호출
    led2.on()                                     # led2 객체의 on 메서드 호출
    sleep(1)                                      # sleep() 함수 호출
    led1.off()                                    # led1 객체의 off 메서드 호출
    led2.off()                                    # led2 객체의 off 메서드 호출
    sleep(1)

print('Press Ctrl+C to exit')
print('-'*30)

led1.source = blink_led                          # led1 객체의 source 속성에 blink_led 함수 할당
led2.source = blink_led                          # led2 객체의 source 속성에 blink_led 함수 할당

pause()
```

구분	예시 코드	설명
라이브러리(패키지)	gpiozero	파이썬에서 특정 기능을 모아둔 코드 집합 GPIO 제어 기능을 제공
모듈(module)	signal	라이브러리 안에 포함된 개별 파일 time, signal
클래스(class)	Button	라이브러리 안에서 특정 기능을 정의한 설계도 LED(), Button(), Buzzer(), Motor() 등등
객체	led1 led2	클래스로부터 생성된 실제 객체 GPIO(17)번과 연결된 led1 객체 GPIO(27)번과 연결된 led2 객체
함수	blink_led()	특정 동작을 수행하는 코드들 정의하고 불러서 동작함
메서드	on() / off() / sleep()	객체가 가진 동작(함수)을 의미
속성	source	객체가 가진 특성(속성)을 의미

② gpiozero vs RPi.GPIO

구분	gpiozero	RPi.GPIO
사용 편리성	고수준 API 제공 초보자들도 사용하기 쉬움	저수준 API 제공 직접 핀 설정등으로 어려움
설정방식	자동 설정 LED(17)	수동 설정 GPIO.setup(17, GPIO.OUT)
이벤트 처리	가능 button.when_pressed = 실행 함수	가능 GPIO.add_event_detect()
PWM 지원	지원	지원
멀티스레딩	자동 지원	직접 설정 필요
외부 라이브러리 지원	지원	지원하지 않음
특징	라즈베리파이 OS 지원	많은 예제가 있음.
대상	초보자 및 간단한 프로젝트	고급 사용자 및 세밀한 제어 필요

초음파 센서를 이용해서 거리를 측정하세요.

타 교재 사용 라이브러리 : RPi.GPIO

```
import RPi.GPIO as GPIO
import time
import sleep

GPIO.setmode(GPIO.BCM)
TRIG = 23
ECHO = 24

GPIO.setup(TRIG, GPIO.OUT)
GPIO.setup(ECHO, GPIO.IN)

GPIO.output(TRIG, False)

while True:
    GPIO.output(TRIG, True)
    sleep(0.00001)
    GPIO.output(TRIG, False)

    pulse_start = 0
    pulse_end = 0

    while GPIO.input(ECHO) == 0:
        pulse_start = time()

    while GPIO.input(ECHO) == 1:
        pulse_end = time()

    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * 17150

    distance = distance / 100

    print(f'Distance : {distance :.3f} m')
    sleep(1)
```

본 교재 사용 라이브러리 : gpiozero

```
from gpiozero import DistanceSensor
from time import sleep

ultra = DistanceSensor(24, 23)

while True:

    print(f'Distance : {ultra.distance :.3f} m')
    sleep(1)
```

gpiozero: ultra.distance 한 단어로 끝.

RPi.GPIO: 펄스를 쏘고(Trigger), 받고(Echo), 물리 공식(17150)까지 직접 계산해야 함.

DC motor 2개를 전진 3초, 정지 1초, 후진 2(후진 속도는 0.5)초, 시키시오

타 교재 사용 라이브러리 : RPi.GPIO

```
import RPi.GPIO as GPIO
from time import sleep

# 1. 핀 번호 설정
GPIO.setmode(GPIO.BCM)
M1_FWD = 6
M1_BWD = 13
M2_FWD = 12
M2_BWD = 16

# 2. 핀 출력 설정
GPIO.setup(M1_FWD, GPIO.OUT)
GPIO.setup(M1_BWD, GPIO.OUT)
GPIO.setup(M2_FWD, GPIO.OUT)
GPIO.setup(M2_BWD, GPIO.OUT)

# 3. PWM 설정 (속도 조절을 위해 필수)
# gpiozero는 이걸 내부에서 자동으로 처리함
pwm1_f = GPIO.PWM(M1_FWD, 100) # 100Hz
pwm1_b = GPIO.PWM(M1_BWD, 100)
pwm2_f = GPIO.PWM(M2_FWD, 100)
pwm2_b = GPIO.PWM(M2_BWD, 100)

# 4. PWM 시작 (0%로 초기화)
pwm1_f.start(0)
pwm1_b.start(0)
pwm2_f.start(0)
pwm2_b.start(0)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    print('Motor1 forward, Motor2 forward')
    # 전진: 정방향 핀 100, 역방향 핀 0
    pwm1_f.ChangeDutyCycle(100)
    pwm1_b.ChangeDutyCycle(0)
    pwm2_f.ChangeDutyCycle(100)
    pwm2_b.ChangeDutyCycle(0)
    sleep(3)

    print('Motor1 stop, Motor2 stop')
    # 정지: 모든 핀 0
    pwm1_f.ChangeDutyCycle(0)
    pwm1_b.ChangeDutyCycle(0)
    pwm2_f.ChangeDutyCycle(0)
    pwm2_b.ChangeDutyCycle(0)
    sleep(1)

    print('Motor1 backward, Motor2 backward')
    # 후진 (속도 0.5): 역방향 핀 50, 정방향 핀 0
    pwm1_f.ChangeDutyCycle(0)
    pwm1_b.ChangeDutyCycle(50) # speed=0.5
    pwm2_f.ChangeDutyCycle(0)
    pwm2_b.ChangeDutyCycle(50) # speed=0.5
    sleep(2)
```

본 교재 사용 라이브러리 : gpiozero

```
from gpiozero import Motor
from time import sleep

# 모터 설정
Motor1 = Motor(forward=6, backward=13)
Motor2 = Motor(forward=12, backward=16)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    # 전진 3초
    print('Motor1 forward, Motor2 forward')
    Motor1.forward()
    Motor2.forward()
    sleep(3)

    #정지 1초
    print('Motor1 stop, Motor2 stop')
    Motor1.stop()
    Motor2.stop()
    sleep(1)

    # 후진 2초
    print('Motor1 backward, Motor2 backward')
    Motor1.backward(speed=0.5)
    Motor2.backward(speed=0.5)
    sleep(2)
```

4. 파이썬 프로그래밍 환경

1) 로컬 환경 소스 준비

본 교재의 모든 소스 파일 다운로드를 진행합니다. 터미널에 작업합니다.

- ① 터미널을 열고 아래 명령어를 입력합니다.

```
$ git clone https://github.com/zeroToaiot/smart_factory.git
```

- ② 작업 위치로 이동

```
$ cd smart_factory/  
pi@zeroToAI: ~smart_factory $
```

- ③ 확인 : ls 명령어

```
pi@zeroToAI: ~smart_factory $ ls
```

- ☐ pi : 접속 사용자 아이디
- ☐ zeroToAI : 라즈베리파이 호스트명
- ☐ ~ : 사용자 홈 디렉터리, /home/pi 임
- ☐ smart_factory : 현재 위치한 smart_factory 디렉터리
- ☐ \$: 프롬프트
- ☐ ls : 파일 목록 보여주는 명령어

2) 개발환경 - 라즈베리파이(Thonny)

IDE(Integrated Development Enironment)는 통합개발환경 소프트웨어를 말하며 보통 프로그래밍을 할 수 있는 환경이며 GUI 환경과 CLS 환경으로 구분된다. 본 교재에서는 라즈베리파이에 기본 설치된 [Thonny Python IDE]를 이용하겠습니다.

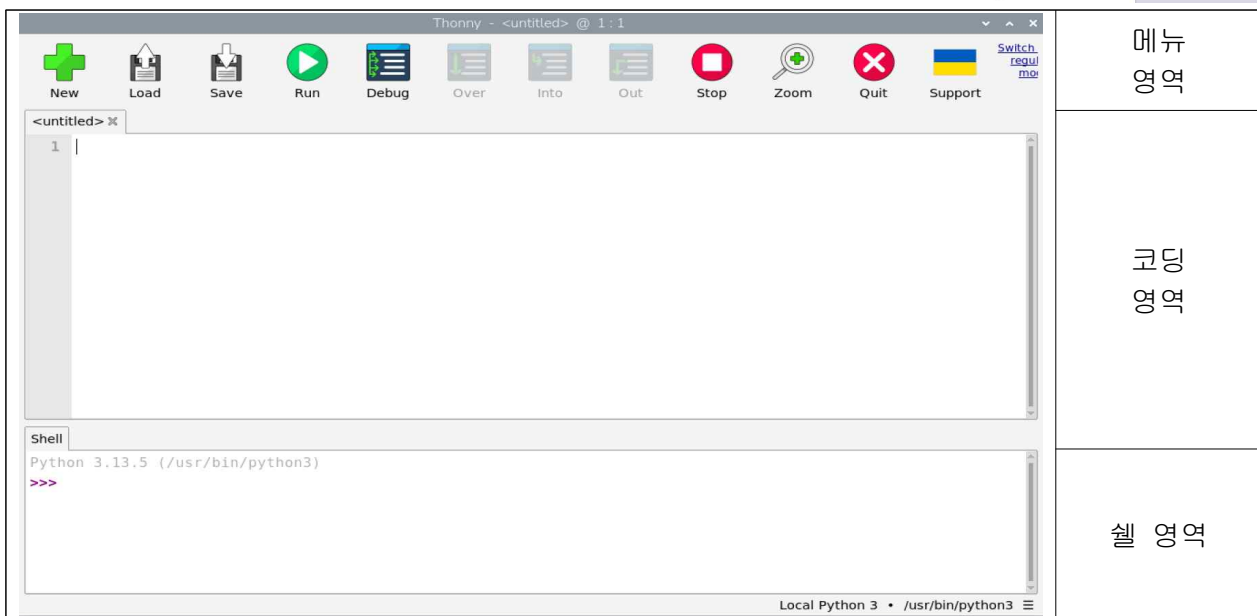
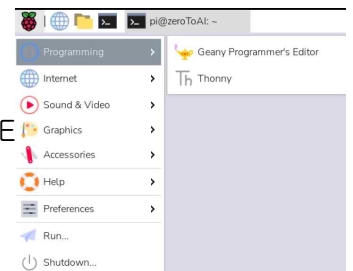
	GUI	CLS 텍스트 에디터
기본설치	Thonny python	nano, vi
추가설치 필요	jupyter notebook, pycharm, Visual Studio code, spyder, python IDLE	vim, emacs

jupyter notebook(설치 필요용량 : 2.7G), pycharm, spyder, python IDLE 등 설치하는 라즈베리파이 OS 에서 사용되어지는 sd card의 용량에 따라 본인이 선택하여 설치하면 된다.

① 라즈베리파이 - [Programming] - [Thonny] 클릭

Thonny는 Thonny 이름의 개발자가 만든 무료 오픈소스 파이썬 IDE 입니다. 라즈베리파이 OS에 기본적으로 설치되어 있습니다.

② 화면 구성



☐ 메뉴

New : 새로운 파일(탭) 생성	Run : 현재 작업 내용 실행
Load : 기존 작업했던 파일을 불러오기	Stop : 현재 실행 내용 중지
Save : 현재 작업 내용 저장	Quit : Thoonny IDE 종료

☐ 프로그램 코딩 영역

; 실제 코드를 입력하시면 됩니다.

☐ 셸

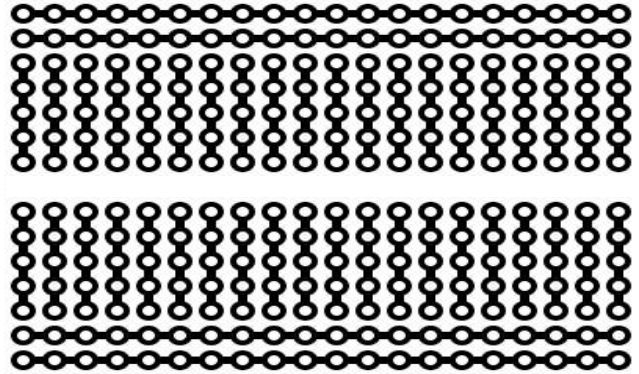
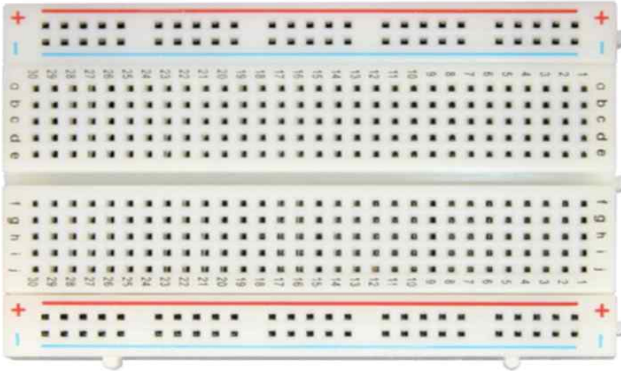
- 실행 창으로 실행 결과나 오류들이 표시
- 실행 정지 단축키(ctrl + c)를 누를 때 셸영역에서 누름.

3) 브레드보드(빵판)

회로를 연결할 때 사용한다. 납땜을 하지 않아도 되기 때문에 시제품이나 실험용을 많이 사용한다.

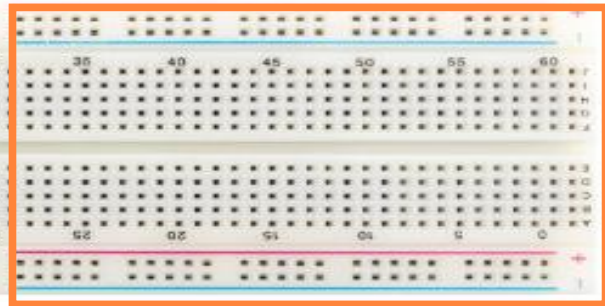
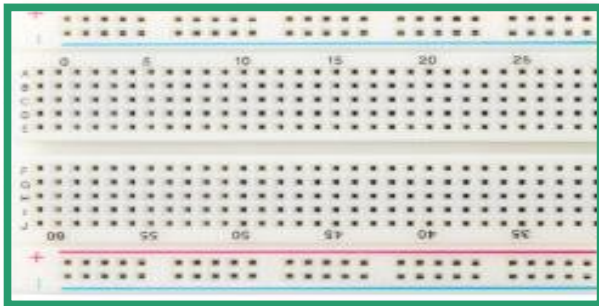
① 400홀

- ☐ 중앙을 기준으로 위, 아래는 구분되어져 있음.
- ☐ 가장자리 줄들은 각자 한줄로 연결되어 있다.
- ☐ 오른쪽 회로도의 가운데는 5개씩 세로로 연결되어 있다.



- ☐ 빨강 라인(+) 라즈베리파이의 VCC와 연결
- ☐ 파랑 라인 (-) : 라즈베리파이의 GND(ground, 접지)와 연결

② 브레드보드 830핀 (56.5×165.5×8.5mm)



양쪽 가장자리의 왼쪽 영역과 오른쪽 영역은 연결되어 있지 않습니다.

4) 저항 기초

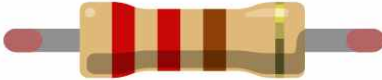
- 전류의 흐름을 방해하는 정도
- 색 띠나 숫자로 값을 표시
- 단위기호 : Ω (옴, ohms)
- 옴의 법칙 : $R = \frac{V}{I}$
- 기초

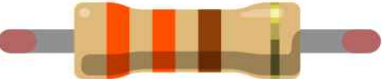


색깔	수	승수	오차율
검정	0	10^0	
갈색	1	10^1	$\pm 1\%$
빨강	2	10^2	$\pm 2\%$
주황	3	10^3	
노랑	4	10^4	
초록	5	10^5	$\pm 0.5\%$
파랑	6	10^6	$\pm 0.25\%$
보라	7	10^7	$\pm 0.1\%$
회색	8	10^8	
흰색	9	10^9	
금색		10^{-1}	$\pm 5\%$
은색		10^{-2}	$\pm 10\%$

② 저항값 읽기

- ☐ 4색 저항값 읽기

첫째숫자	둘째 숫자	10 승수	저항값 오차
빨강	빨강	갈색	금색
2	2	10^1	5%
		첫째+둘째숫자 × 10의 승수 ± 오차	
		$22 \times 10^1 = 220\Omega \pm 5\%$	

첫째숫자	둘째 숫자	10 승수	저항값 오차
주황	주황	갈색	금색
3	3	10^1	5%
		첫째+둘째숫자 × 10의 승수 ± 오차	
		$33 \times 10^1 = 330\Omega \pm 5\%$	

첫째숫자	둘째 숫자	10 승수	저항값 오차
갈색	검정	빨강	금색
1	0	10^2	5%
		첫째+둘째숫자 × 10의 승수 ± 오차	
		$10 \times 10^2 = 1000\Omega = 1K\Omega \pm 5\%$	

□ 5색 저항값 읽기

첫째숫자	둘째 숫자	셋째숫자	10 승수	저항값 오차
빨강	파랑	검정	주황	은색
2	6	0	1000	10%
			첫째,둘째숫자,셋째숫자 × 10승수 ± 오차	
			$260 \times 10^3 = 260,000\Omega = 260K\Omega \pm 10\%$	

③ 저항 선택하기

□ 저항값 계산

일반적인 적색 LED의 경우 1.8V의 전압강하가 있고 적정전류는 25mA이다. 그리고 라즈베리파이는 5V, 3V3 출력 전압을 지원하고 있다. 옴의 법칙을 이용하여 필요한 저항값을 계산해 보자.

○ 예 : 라즈베리파이는 5V를 사용하고 LED의 전압강하는 1.8V, 적정전류는 0.025A 일경우 계산상 128Ω이 필요하다.

$$R = \frac{V}{I} = \frac{\text{출력단자전압} - \leq D\text{전압강화}}{\text{적정 전류}} = \frac{5 - 1.8}{0.025} \geq 128\Omega$$

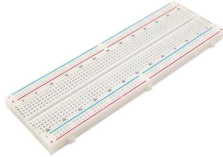
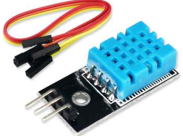
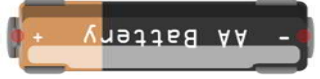
100Ω은 128Ω보다 저항값이 적으니 안되고, 다음이 220Ω이다, 128Ω보다 크고 쉽게 구할 수 있는 220Ω을 사용하면 된다. 이보다 매우 큰 저항을 사용하면 밝기가 약하다.

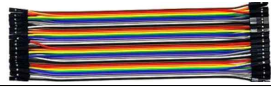
○ 저항 종류 : 보통 쉽게 구할 수 있는 보편적인 저항은 아래와 같다.

{ 100, 220, 470, 1000, 2200, 4700, 10000 }

3V3 구성 요소에는 5V를 사용하면 안되며, 특히 모터등은 GPIO 핀에 직접 연결하지 말고 대신 H 브리지 회로 또는 모터 컨트롤러 보드를 사용하여야 한다.

5) 실습 키트 전체 구성품 한눈에 보기

재료명	이미지	수량	재료명	이미지	수량
라즈베리파이		1	브레드보드 (830홀)		1
T-Cobbler		1	연결 케이블		1
LED (5mm)		3	푸시버튼		3
피에조 능동 부저		1	피에조 수동 부저		1
초음파센서 (HC-SR04P)		2	FND 7-segment		
적외선 동작 감지 HC-SR505		1	온습도센서 DHT11		1
가변저항 Potentiometer		1	MCP3008		1
수위센서		1	I ² C LCD		1
파이 카메라		1	카메라 홀더 (아크릴)		
모터드라이버 SN754410		2	DC모터		4
건전지 (1.5V)		4	건전지 홀더 (4구 직렬)		1

저항	220Ω		5	점퍼 케이블	암암		
	330Ω		8		암수		
	10kΩ		1		수수		

구분	분류	부품	갯수	gpiozero 라이브러리 사용 클래스
센서	1. LED	LED	1	LED
	2. 부저	능동 부저	1	Buzzer
	3. 부저	수동 부저	1	TonalBuzzer
	4. 초음파	HC-SR04P	2	DistanceSensor
	5. 모터 드라이버 H-bridge	SN754410	2	-
	6. 모터 드라이버 H-bridge	SN754410	2	-
	7. DC motor	DM148	4	Motor, Robot
부품	8. 브레드보드	브레드보드(830, 400홀)	1	
	9. 디스플레이	I2C LCD	1	디스플레이
	10. 자동차 프레임	4WD Rc카	1	4축용 프레임
	11. 바퀴	66mm	4	RC카용
	12. 건전지 홀더 4구	리튬이온 용	1	건전지 홀더
	13. 건전지	AA 건전지 1.5V	4	전원공급
	14. 보조배터리	보조배터리	1	라즈베리파이 5V 전원 공급
	15. 케이블	-	1	보조배터리와 라즈베리파이 연결

① 조립시 필요 물품 및 도구

명	이미지	수량	용도	기타
납땀기 및 기타 일체		1	DCmotor 점퍼케이블 연결을 위한 납땀 용도 (미리 납땀되어 있는 DC모터에는 필요없습니다.)	
드라이버		1	RC Car 프레임 고정 및 dc motor 연결 등	

용로즈		1	물품 및 전선 잡기 및 점퍼연결선 절단 등	
양면테이프		-	부품 프레임 부착	
절연테이프		-	DC모터 절연	
연결선		-	브레드보드 연결 및 센서 연결 등	점퍼케이블로 대체가능

6) 라즈베리파이와 브레드보드(830) 기초 연결

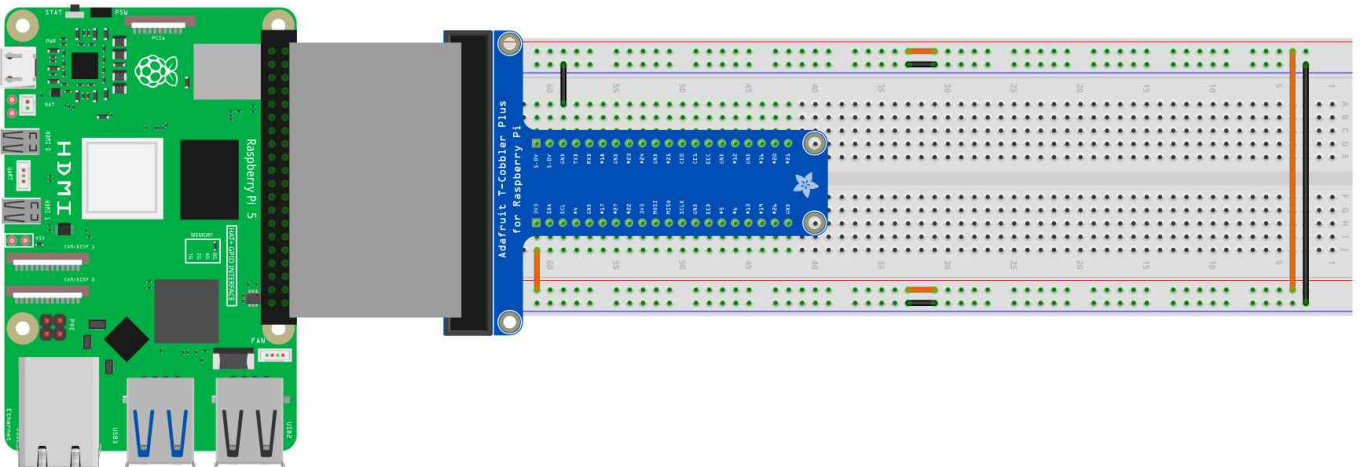


그림 76 basic_830.png

- ☐ 브레드보드(830)에 T Cobbler를 연결
주위 : 브레드 보드의 빨강라인이 위에 위치해야 함.
- ☐ 라즈베리파이의 40개의 GPIO와 연결케이블을 연결 한 후 T Cobbler에 연결
- ☐ 라즈베리파이의 3.3V (물리적핀번호 1번) 브레드보드 아래쪽 빨강 라인에 연결
- ☐ 라즈베리파의 GND (물리적 핀번호 6번), 브레드보드 위쪽 파랑 라인에 연결
- ☐ 브레드보드 830홀 중앙에 빨강, 파랑 라인을 연결
- ☐ 브레드 보드 맨 오른쪽에 반대편 빨강, 파랑 라인을 연결

Chapter 2. 스마트 팩토리 환경 모니터링

1. 센서 기초 : LED와 부저, 버튼

1) LED(Light Emitted Diode)

- ① 발광 다이오드, 전기에너지를 빛 에너지로 변환
- ② 구조 : 긴 선, GPIO 핀에 연결
짧은 선, GND에 연결
- ③ LED(5mm) : 220Ω ~ 330Ω의 저항을 사용
- ④ 연결 source : (예 GPIO 17번에 연결)



```
from gpiozero import LED
```

```
led = LED(17)
```

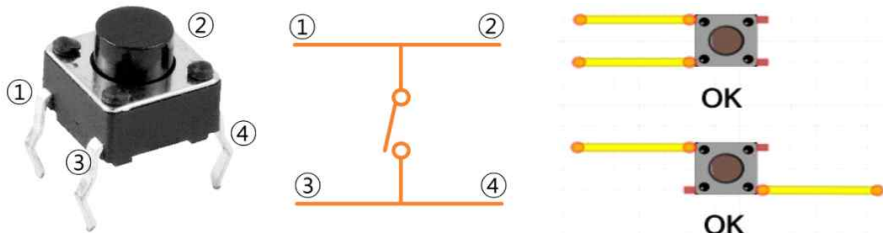
⑤ 동작 method

- ☐ on() : 켜다, 예) led.on()
- ☐ off() : 끄다, 예) led.off()
- ☐ blink() : 1회 깜빡거리다.
- ☐ pulse() : 천천히 연속해서 깜빡임. 기본값은 1초, pwm가 True일 때 동작.

2) 버튼 , Tactile Switch

① 구조

- ☐ 1과2, 3과4는 각각 연결되어 있음



② 연결(아래 중 하나의 방법만 사용)

- ☐ 1 또는 2를 GPIO 핀에 연결시 3 또는 4를 GND에 연결
- ☐ 3 또는 4를 GPIO 핀에 연결시 1 또는 2를 GND에 연결

③ 연결 source : (예 GPIO 23번에 연결)

```
from gpiozero import Button
```

```
btn = Button(23)
```

④ 동작 method

- is_pressed() : 버튼이 눌러지는 여부, true/false 값 반환
- wait_for_press(timeout=None) : 버튼 눌러졌는지 여부
 - timeout : 기본값이 None일 경우 버튼 눌러질때까지 대기
- wait_for_release(timeout=None) : 버튼을 뗄 때까지 대기
- when_pressed() : 버튼이 눌러지면 동작
- when_released() : 버튼 눌렀다 띄면 동작

3) 부저

① 개요

- ☐ 전자석의 코일에 전류를 보낸 전기적 신호로 진동판을 진동시켜 소리를 출력
- ☐ 마그네틱부저, 피에조부저, 기계식 부저등이 있음

② 피에조부저(압전세라믹 부저)

- 진동판만 연결된 단순 구조
- 일정 주파수를 입력시켜 다양한 음을 낼 수 있음
- 가격이 저렴

☐ 수동 부저(Passive Buzzer)

- 내부에 진동 회로가 없음,
- 소리를 내기 위해 PWM 신호 또는 특정 주파수의 입력이 필요
- 주어진 신호의 주파수에 따라 다양한 소리 가능
- 주파수 제어하기 위해 마이크로컨트롤러나 추가 회로 필요
- 예: 멜로디 생성이나 다양한 알림음을 표현할 때 사용.

☐ 능동 부저(Active Buzzer)

- 내장 진동 회로가 있어 부저 자체가 동작음 발생
- 입력 전압만 연결하면 작동하며, 단순 ON/OFF 신호로 제어 가능
- 사용이 간단하며, 일정한 주파수로 고정된 소리를 냄
- 예: 경고음이나 알람 장치에 주로 사용.



③ 연결 source : (예 GPIO 13번에 연결)

```
from gpiozero import Buzzer
```

```
bz = Buzzer(13)          # bz 객체 생성
```

④ 동작 method

☐ Buzzer

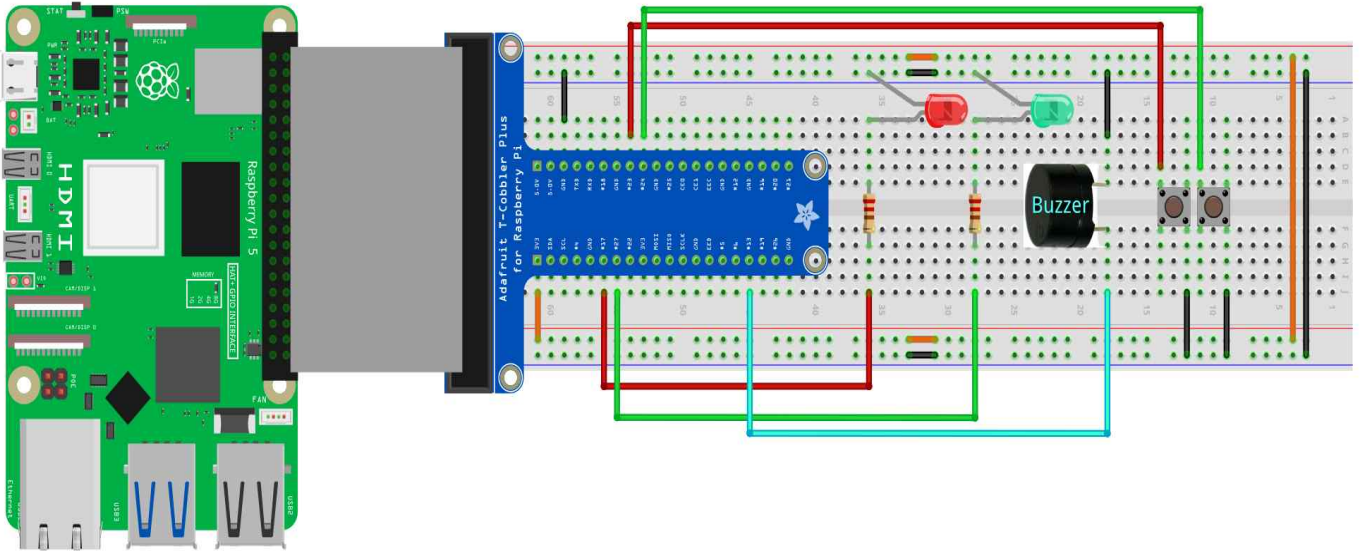
- on() : 동작음 발생 시작
- off() : 동작음 끄
- beep() : 1초단위로 경고음 무한 발생.

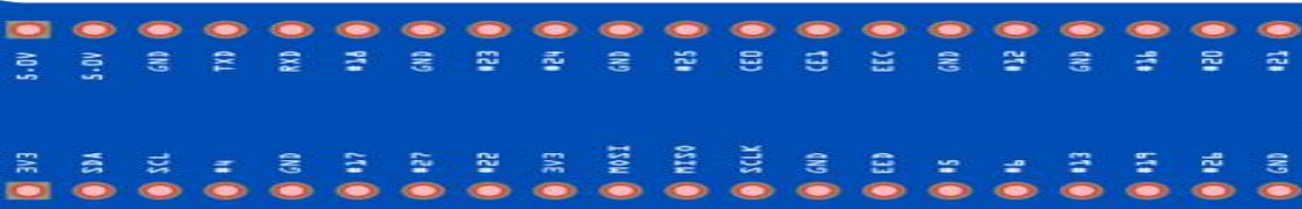
4) 회로도 및 연결정보

① 연결정보

분류	부품	갯수	라이브러리	이미지
1. LED	LED	2	LED	
2. 저항	220Ω	2	-	
3. 버튼	button	2	Button	
4. 부저(능동)	buzzer	1	Buzzer	

② 회로도



																버튼 1	버튼 2						
5V	5V	GND	TX	RX	GPIO 18	GND	GPIO 23	GPIO 24	GND	GPIO 25	CE0	CE1	EEC	GND	GPIO 12	GND	GPIO 16	GPIO 20	GPIO 21				
2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32	34	36	38	40				
																							
1	3	5	7	9	11	13	15	17	19	21	23	25	27	29	31	33	35	37	39				
3V3	SDA	SCL	GPIO 4	GND	GPIO 17	GPIO 27	GPIO 22	3V3	MOSI	MISO	SCLK	GND	EED	GPIO 5	GPIO 6	GPIO 13	GPIO 19	GPIO 26	GND				
					RED LED	Green LED											부 저						

5) 기본 실습

① LED

☐ led1, led2를 1초 간격으로 동시에 on/off 해보자.

```
# file name : led_1.py
# 두 개의 LED 점등 프로그램

from gpiozero import LED
from time import sleep

led1 = LED(17)
led2 = LED(27)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    led1.on()
    led2.on()
    sleep(1)

    led1.off()
    led2.off()
    sleep(1)
```

☐ led1, led2를 0.5초 간격으로 교대로 on/off 해보자.

```
# file name : led_2.py
# 두 개의 LED 교대로 깜빡임

from gpiozero import LED
from time import sleep

led1 = LED(17)
led2 = LED(27)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    led1.on()
    led2.off()
    sleep(0.5)

    led1.off()
    led2.on()
    sleep(0.5)
```

② Button

☐ 디바운스 처리

- 디바운스란 스위치를 누르거나 떼는 순간에 접점이 여러 번 붙었다 떨어지는 바운스 현상을 방지하기 위해 처리하는 방법
- 소프트웨어적으로 처리
 - bounce_time=0.1 : 바운싱을 방지하기 위해 0.1초의 디바운스 시간 부여

☐ 2개의 버튼에서 버튼을 누르면 각 버튼에 대해 출력메시지를 출력해보자.

```
# file name : button_1.py
# 2개 버튼 눌렀을 때 메시지 출력

from gpiozero import Button
from signal import pause

btn1 = Button(23, bounce_time=0.1)
btn2 = Button(24, bounce_time=0.1)

def btn1_pressed():
    print('Button 1 was pressed')

def btn2_pressed():
    print('Button 2 was pressed')

print('Press Ctrl+C to exit')
print('-'*30)

btn1.when_pressed = btn1_pressed
btn2.when_pressed = btn2_pressed

pause()
```

③ 부저

☐ 0.3초 경고음 발생, 0.2초 쉼. 무한 반복하도록 하자.

```
# file name: buzzer_beep.py
# 능동 부저 경고음 발생 프로그램

from gpiozero import Buzzer
from time import sleep

bz = Buzzer(13)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    bz.on()
    sleep(1)
    bz.off()
    sleep(1)

    bz.beep(on_time=0.3, off_time=0.2)
    sleep(1)
```

6) 버튼을 통한 LED 및 부저 제어 실습

① 목표 : 버튼을 누르면(시스템 가동) LED가 켜지고, 다시 누르면(시스템 정지) LED가 꺼지면서 부저가 '빠-' 하고 울리는 '공장 설비 수동 조작 패널' 만들기.

② 동작 프로세스

- ☐ 버튼 2개 (가동/정지 분리): 시스템 가동용(Start) 버튼과 시스템 정지용(Stop) 버튼을 물리적으로 분리합니다. 한 번 누르면 켜지고 다시 누르면 꺼지는 토글(Toggle) 방식도 가능하지만, 공장 환경에서는 비상 상황 시 작업자가 고민 없이 '정지' 버튼을 즉각 누를 수 있도록 스위치를 분리하는 것이 표준입니다.
- ☐ LED 2개 (상태 표시용): 시스템이 작동 중임을 알리는 녹색(Green) LED 1개와, 설비가 정지되었음을 직관적으로 보여주는 적색(Red) LED 1개를 사용하여 현장의 상태 표시반(신호등) 역할을 구현합니다.
- ☐ 부저 1개 (경보용): 전원만 인가되면 즉각적으로 '빠-' 하는 단일 경고음을 내는 피에조 능동 부저 1개를 사용합니다.

```
# file name: led_button_buzzer.py
# LED, 버튼, 부저 테스트 프로그램

from gpiozero import LED, Button, Buzzer
from signal import pause

# LED 핀 번호 설정
ledRed = LED(17)
ledGreen = LED(27)

# 버튼 핀 번호 설정
btnRed = Button(23, bounce_time=0.05)
btnGreen = Button(24, bounce_time=0.05)

# 부저 핀 번호 설정
bz = Buzzer(13)

print('Press Ctrl+C to exit')
print('-'*30)

def btnRed_pressed():
    print('Red button pressed')
    ledRed.on()
    ledGreen.off()
    bz.beep(on_time=0.3, off_time=0.2, n=None)

def btnGreen_pressed():
    print('Green button pressed')
    ledGreen.on()
    bz.off()
    ledRed.off()

btnRed.when_pressed = btnRed_pressed
btnGreen.when_pressed = btnGreen_pressed

print('Press Ctrl+C to exit')
print('-'*30)

pause()
```

2. 아날로그 데이터 읽기 : ADC와 아날로그 센서

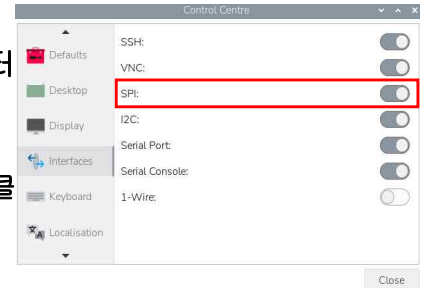
1) ADC(Analog Digital Converter)

① 필요성

- ☐ 라즈베리파이는 아날로그 데이터값을 직접 읽을 수 없기 때문에 아날로그 데이터를 디지털로 변환이 필요
- ☐ 아두이노는 ADC를 내장하고 있지만 라즈베리파이는 ADC가 내장되어 있지 않음

② 라즈베리파이 SPI 통신

- ☐ SPI 통신을 통해 라즈베리파이와 MCP3008간의 데이터 통신
- ☐ 라즈베리파이 SPI 통신 활성화
 - [시작] - [Preferences] - [Raspberry Pi Configuration] 클릭
 - [Interfaces] 탭 - SPI 활성화



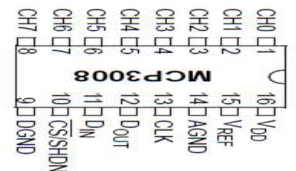
2) MCP (Multi-Chip Package)

① 기능

- ☐ 아날로그 센서의 입력값을 디지털 값으로 변환 전송
- ☐ MCP IC칩들은 SPI 통신으로 데이터를 전송

② MCP-3008

- ☐ 10bit 8채널(single-ended inputs) ADC
- ☐ 라즈베리파이와 SPI 통신을 통해 데이터 송수신
- ☐ 칩이 살짝 들어간 부분이 있는 쪽이 1번 핀쪽
- ☐ 기본 회로 연결



라즈베리파이			MCP3008	
BCM 핀번호	40Pin 물리핀 번호	기능	핀번호	기능
3V3	1	3V3	16	V _{DD}
3V3	1	3V3	15	V _{REF}
GND	39	GND	14	AGND
GPIO 11	23	SCLK	13	CLK
GPIO 09	21	MISO	12	D _{OUT}
GPIO 10	19	MOSI	11	D _{IN}
GPIO 08	24	CE0	10	CS/SHDN
GND	39	GND	9	D _{GND}

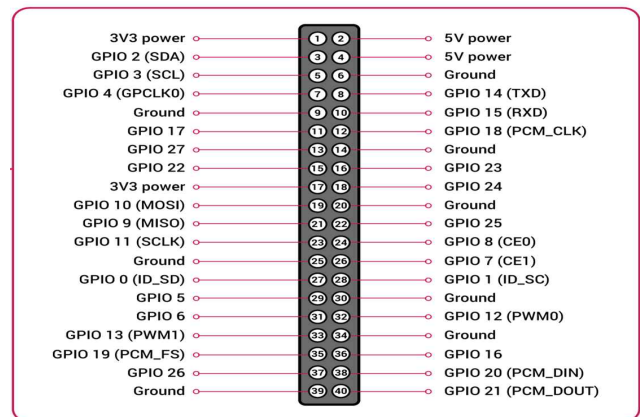


그림 91 라즈베리파이 GPIO핀 번호

③ 측정값의 변환 공식

- ☐ 0~7채널에서 측정된 전압(V)에 따른 VREF에 변환되는 값

MCP3008은 10bit 데이터 처리로 아날로그 0~3.3V = 0~5V = 0~1023 값을 가진다.

예를 들어 아날로그 센서 측정값이 1V라면 MCP에 연결된 전압값에 따라 실제 변환값은 아래와 같다.

○ 3V3 이용시 : $\frac{1V}{3.3V} \times 1024(2^{10bit}) = 310.3$

○ 5V 이용시 : $\frac{1V}{5V} \times 1024(2^{10bit}) = 204.8$

④ 기본 연결

아날로그 센서들 연결 하는 channels

ch 7	ch 6	ch 5	ch 4	ch 3	ch 2	ch 1	ch 0	channel
8	7	6	5	4	3	2	1	핀번호
								MCP3008 앞
9	10	11	12	13	14	15	16	핀번호
DGND	CS/SHDN	D _{IN}	D _{OUT}	CLK	AGND	V _{DD}	V _{DD}	기능
GND	GPIO 8	GPIO 10	GPIO 9	GPIO 11	GND	3V3	3V3	GPIO연결
	CE0	MOSI	MISO	SCLK				기능

□ 회로도

○ SPI 통신을 위한 라즈베리파이와 연결

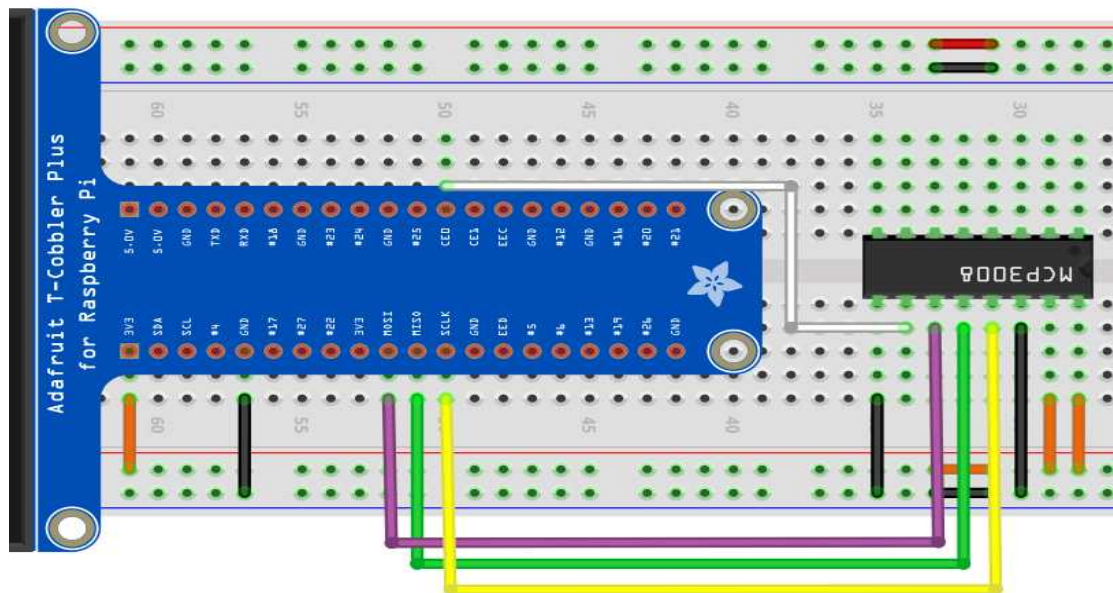


그림 92 ch03_Analog_01_ADC_MCP3008 회로도

⑤ 클래스 : MCP3008

```
from gpiozero import MCP3008

MCP3008(channel=0)
MCP3008(channel=0, clock_pin=11, mosi_pin=10, miso_pin=9, select_pin=8)
```

⑥ 변수 property

○ value : 채널에서 입력된 값을 0 ~ 1사이 값으로 조정하여 표시

3) 스마트 팩토리 적용 실습

① 가변저항을 활용한 LED 밝기 조절 다이얼

□ 개요

- 가변저항 다이얼을 돌려 작업장의 LED의 밝기를 조절.

□ PWDLED

○ 디밍(Dimming)

- PWM(Pulse Width Modulation)로 LED에 들어오는 전력을 조절하여 밝기를 조절
- 고속의 스위칭으로 High와 Low 신호의 비율을 조절하여 LED의 밝기, 모터의 회전 등을 조절하는 방법

```
from gpiozero import PWMLED
```

```
led = PWMLED(17)
```

○ 동작 method

- pulse() : LED를 0 ~ 100까지 밝기를 변화시킨다.
- close() : 사용 중단

□ 가변저항

○ 개요

- 가변저항(영어: potentiometer 포텐서미터)은 전자회로에서 저항값을 임의로 바꿀 수 있는 저항기.
- 주로 오디오 장비나 동작 감지기 같은 센서에서 감도, 밸런스, 입력, 출력 등을 조절하는데 사용됩니다.

○ 구조 및 연결

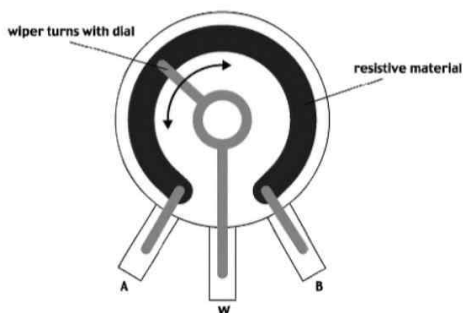


그림 93



그림 94

핀	연결
A, B	각각 vcc, gnd로 연결 (바뀌 연결되도 상관없음.)
w (가운데)	signal.

○ 저항물질(resistive material)

가장자리에 둥글게 연결된 띠는 탄소로 이루어져 전기가 흐르며 저항의 역할을 한다. 양쪽 단자(A, B)로 연결되어 있다. 극성은 없기 때문에 A, B를 +, - 어느중에 연결하여도 상관없다.

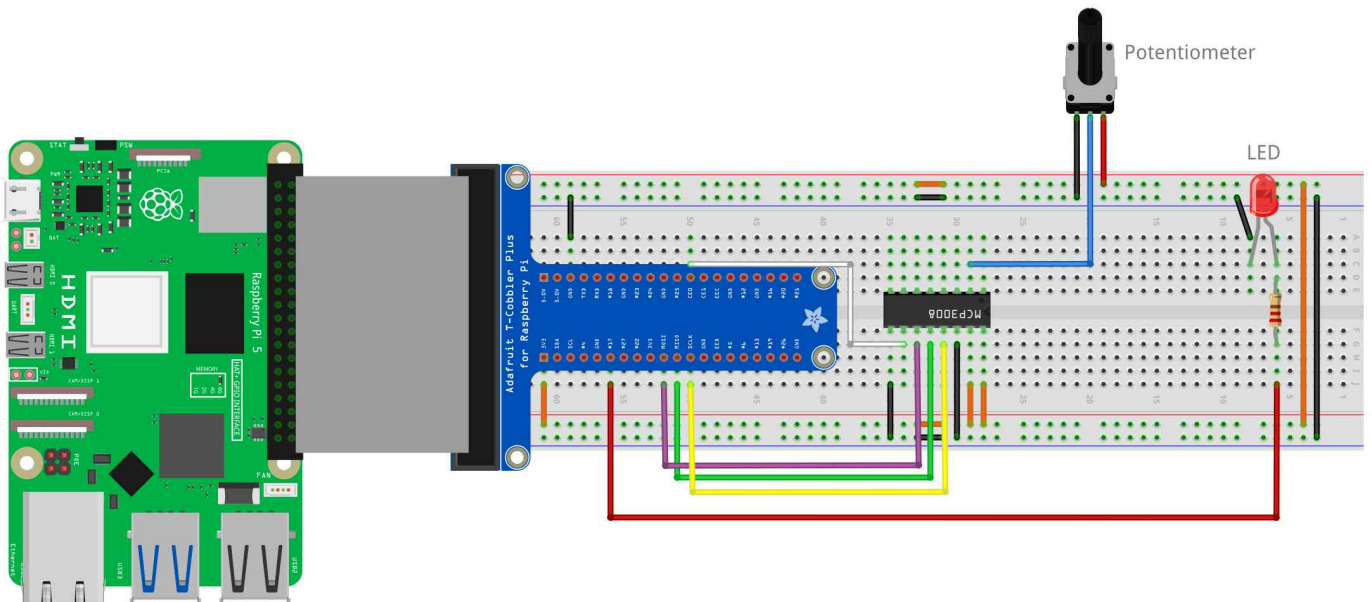
○ 회전막대(wiper-turns with dial)

회전하면 회전막대 끝이 저항물질에 연결되면 연결되며 저항의 길이가 변화가 생기며 길이가 길어질수록 저항값이 커진다.

☐ 연결정보 및 회로도

sensor		연결
	GND	GND
	sig	MCP 3208 channel 1
	+	3.3V

sensor		라즈베리파이
LED	+	GPIO 17
	-	GND



☐ source code

```
# file: potentiometer_led_1.py
# 가변 저항값에 따라 LED 밝기 조절

from gpiozero import MCP3008, PWMLED
from time import sleep

mcp = MCP3008(channel=1)
led = PWMLED(17)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    print(f'{mcp.value :.3f}')
    led.value = mcp.value

    sleep(0.1)
```

② 수위 센서를 활용한 설비 냉각수 수위 측정 및 경고

□ 개요

- 냉각수 수위가 10% 미만으로 떨어지면 LED를 on 시키자.

□ 수위 센서(Water Sensor)

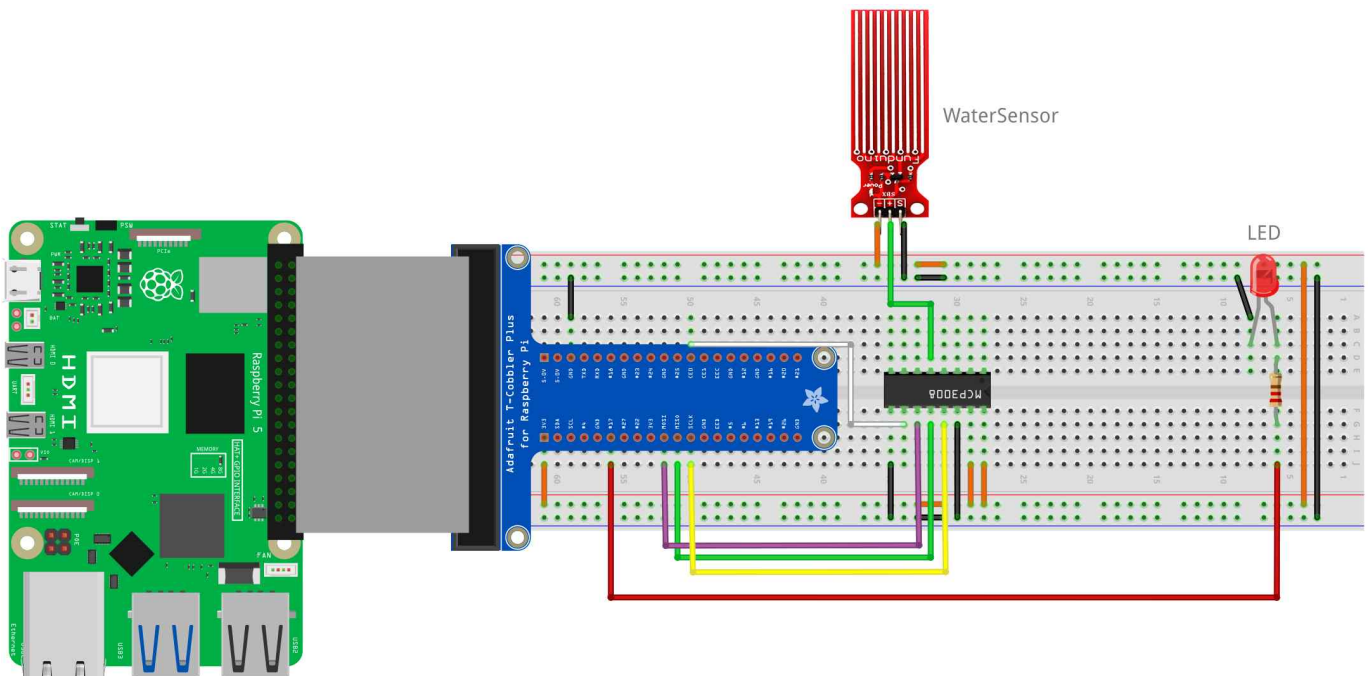
- 액체의 높이를 감지하는 센서
- 수위 센서는 접촉형과 비접촉형으로 구분
- 응용 분야 : 물탱크 수위 모니터링, 배수 관리, 정수처리 시스템, 산업용 액체 관리, 선박의 연료 상태 측정 등 다양한 산업에서 활용
- 동작원리



센서		동작원리
		- 니켈이 선 형태로 아래로 쪽 코딩되어 있고 끝부분에는 떨어져 있다. 평상시에는 연결되어 있지 않아 높은 저항을 가짐. - 물은 전기를 잘 흐르게 함으로 물이 차오른다면 아래 부분의 니켈선은 병렬로 연결되어지는 형태가 된다. - 저항이 감소하는 정도를 측정

□ 연결정보 및 회로도

sensor		라즈베리파이 / ADC	
	SIG	MCP3008 channel 4	
	VCC	3.3V	
	GND	GND	
sensor		라즈베리파이	
LED	+	GPIO 17	
	GND	GND	



□ source code

```
# file: water_led.py
# 물 수위 센서 값에 따라 LED 밝기 조절

from gpiozero import MCP3008, LED
from time import sleep

# MCP3008 채널 4에 수위 센서
mcp = MCP3008(channel=4)

# GPIO 17번에 연결된 경고용 LED
led = LED(17)

print('냉각수 수위 모니터링 시작.. Press Ctrl+C to exit')
print('-'*30)

while True:
    value = mcp.value
    print(f'현재 수위(비율) : {value:.3f}')

    # 수위 센서 값이 10% 미만이면 LED 켜기
    if value < 0.1:
        led.on()
    else:
        led.off()

    sleep(0.5)
```

3. 초음파 센서를 활용한 거리 측정 및 경고

1) 개요

① 초음파 센서

- ☐ trigger(송신부)에서 일정 간격의 초음파 펄스를 방출하고 전방 대상물에 부딪혀 돌아온 신호를 echo(수신부)에서 받아 이에 대한 시간차를 기반으로 거리를 산출.
- ☐ 전방에 장애물의 유무 및 물체와의 거리, 속도 등을 측정
- ☐ 거리 계산
 - 이동거리(m) = 속도(m/s) × 소요시간(s)
 - 초음파 속도 : $340m/s$
 - 거리(m) = $\frac{\text{초음파 왕복 소요시간}(s) \times 340m/s}{2}$



그림 102 HC-SR04P

2) 클래스 : DistanceSensor()

① 특징

- ☐ 따로 계산하는 코드 필요 없음. 알아서 계산해서 거리값만 반환
 - 예 : echo는 GPIO 24, Trig는 GPIO 23

```
from gpiozero import DistanceSensor

#ultra = DistanceSensor(echo=24, trigger=23)
ultra = DistanceSensor(24, 23)
```

② 동작 method

- wait_for_in_range(timeout=None) : 센서가 감지상태(distance < threshold_distance)가 될 때까지 대기
 - 물체가 설정된 임계거리(threshold_distance) 안으로 들어올 때까지 프로그램 멈추고 기다리고 있음.
 - timeout이 None이면 무한정 대기
- wait_for_out_of_range(timeout=None) : 물체가 임계거리 밖으로 나갈때까지 프로그램 멈춤
- when_in_range() : 기본 거리(threshold_distance) 안이 되면. 즉 감지가 되면 이벤트 동작
- when_out_of_range() : 기본 거리 벗어나면 이벤트 동작

실시간 처리	순차적 처리
when_in_range() when_out_of_range()	wait_for_in_range() wait_for_out_of_range()
이벤트 기반 처리	블로킹 방식 처리

③ 변수 property

- distance : 측정값
- value : 연결된 장치의 값
 - 범위 : 0 ~ 1

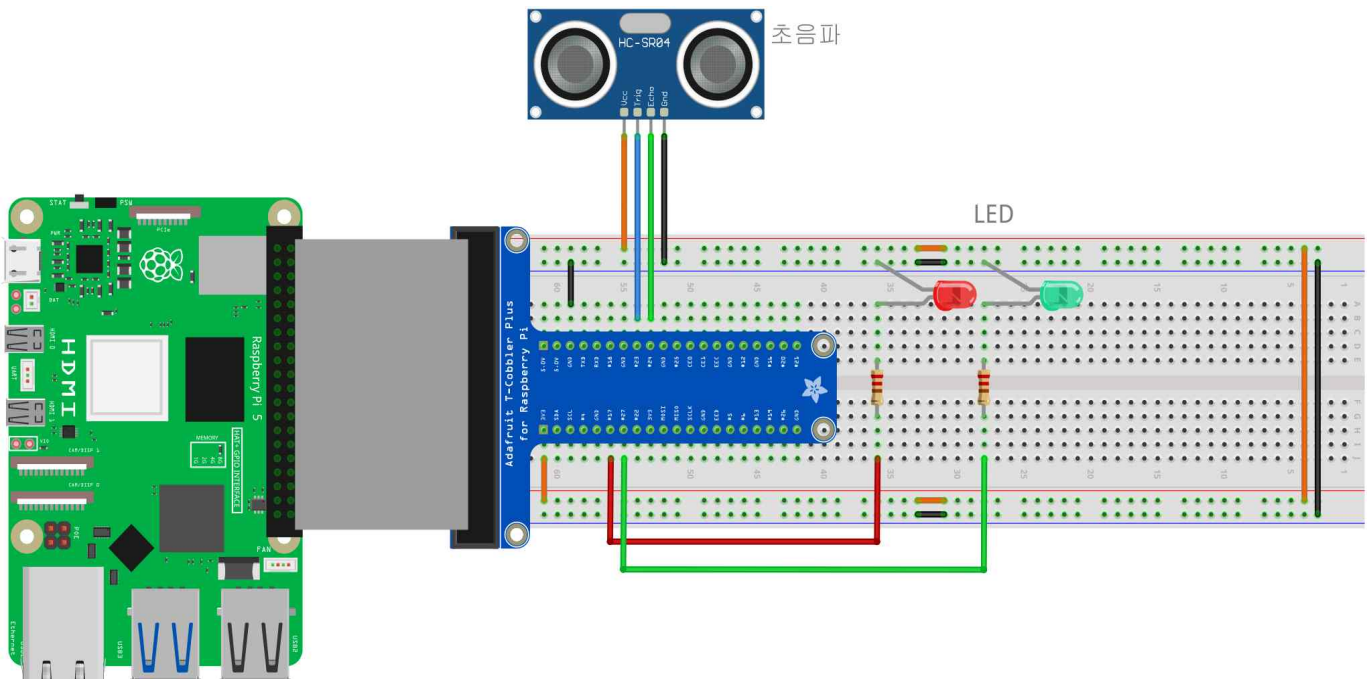
3) 연결정보 및 회로도

① 연결정보

라즈베리파이	sensor(HC-SR04P)		sensor		BCM 핀번호
3.3V	VCC		LED	red +	GPIO 17
GPIO 23	Trig			green +	GPIO 27
GPIO 24	Echo			GND	GND
GND	GND				

② 회로도

- ☐ **주의** : 구형 HC-SR04는 5V 전용입니다. 최신 HC-SR04P는 3.3V~5V 겸용이지만, 라즈베리파이의 안전(GPIO 3.3V 제한)을 위해 반드시 라즈베리파이의 3.3V 핀에 연결해서 사용합니다.



```
# file name : DistanceSensor_1.py
# 거리 센서 값 출력

from gpiozero import DistanceSensor
from time import sleep

#ultra = DistanceSensor(echo=24, trigger=23)
ultra = DistanceSensor(24, 23)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    print(f'Distance : {ultra.distance :.3f} m')

    sleep(1)
```

4) 스마트 팩토리 적용 실습

- ① 목표 : 작업자가 위험 구역(20cm 이하)에 진입하거나 벗어나는 '이벤트(상태 변화)'가 발생할 때만 즉각적으로 반응하여 거리를 표시하고 경고등(Red LED)과 안전등(Green LED)을 제어해 보자

② source code

```
# file name : DistanceSensor_led.py
# 거리 센서 값에 따라 LED on/off
# 거리 센서 값이 0.3m 이하일 때 red_led on, green_led off
# 거리 센서 값이 0.3m 초과일 때 red_led off, green_led on

from gpiozero import DistanceSensor, LED
from signal import pause

red_led = LED(17)
green_led = LED(27)
ultra = DistanceSensor(24, 23, max_distance=2.0, threshold_distance=0.3)

red_led.off()
green_led.off()

def object_detected():
    print('Object detected')
    print(f'Distance: {ultra.distance:.3f} m')
    red_led.on()
    green_led.off()

def object_not_detected():
    print('Object not detected')
    print(f'Distance: {ultra.distance:.3f} m')
    red_led.off()
    green_led.on()

print('Press Ctrl+C to exit')
print('-'*30)

ultra.when_in_range = object_detected
ultra.when_out_of_range = object_not_detected

pause()
```

Press Ctrl+C to exit

```
-----
Object not detected
Distance: 0.450 m
Object not detected
Distance: 0.420 m
Object detected
Distance: 0.280 m
Object detected
Distance: 0.250 m
```


4. 인체 감지 및 보안 : PIR 모션 센서를 활용한 작업자 인식

1) 개요

① 적외선 인체감지센서 (PIR:Passive Infrared Sensor)

- ☐ 사람이나 동물의 몸에서 뿜어져 나오는 미세한 적외선(열, $9\mu\text{m} \sim 11\mu\text{m}$)의 변화를 감지하여 움직임을 파악합니다. 센서 앞을 사람이 지나가면 신호를 발생시킵니다.
- ☐ 초음파 센서와의 차이점: 초음파 센서는 앞에 있는 것이 벽이든 사람이든 거리를 정확히 재지만, PIR 센서는 오직 '열을 가진 물체의 움직임'만 감지합니다. 가만히 서 있으면 감지하지 못합니다.
- ☐ 스마트 팩토리 응용 분야: 물류 창고의 에너지 절감형 자동 조명, 야간 무인 공장 침입 탐지 경보, 위험 설비 주변의 작업자 인체 감지.



그림 105 HC-SR501

② HC-SR505

- ☐ 감지 거리 : 2 ~ 3 m
- ☐ 출력값 : 디지털값(0 or 1) 움직임 감지 : 1, 움직임 감지 없음 : 0
- ☐ 최소 전압(vcc) : 4.5V 이상
- ☐ 내부 전압을 낮춰주는 레귤레이터가 자체 내장, 출력은 3.3V로 안정적
- ☐ 특징 : 약 8초의 고정 딜레이(Delay) 기능이 내장. .
즉, 움직임이 감지되고 나서 움직임이 멈추더라도 약 8초 동안은 계속 '감지됨(True)' 상태를 유지하다가 꺼집니다. (이는 사람이 잠시 멈춰있을 때 조명이 바로 꺼져 버리는 불편함을 막아주는 유용한 기능입니다.)



그림 106 HC-SR505

2) 클래스 : MotionSensor()

```
from gpiozero import MotionSensor  
  
pir = MotionSensor(25)
```

① 동작 method

- wait_for_motion() : 모션이 감지될때까지 대기
- wait_for_no_motion() : 모션이 감지되지 않을 때 까지 대기.
- when_motion() : 비활성상태에서 활성상태로 변경될 때 실행,
- when_no_motion() : 활성상태에서 비활성상태로 변경될 때 실행

② 변수 property

- motion_detected : True / False 반환, 임계값 초과시 True 그렇지 않으면 False
- pin : motion sensor 연결 핀 번호 반환
- value : 움직임 감지하면 1, 그렇지 않으면 0

3) 연결정보 및 회로도

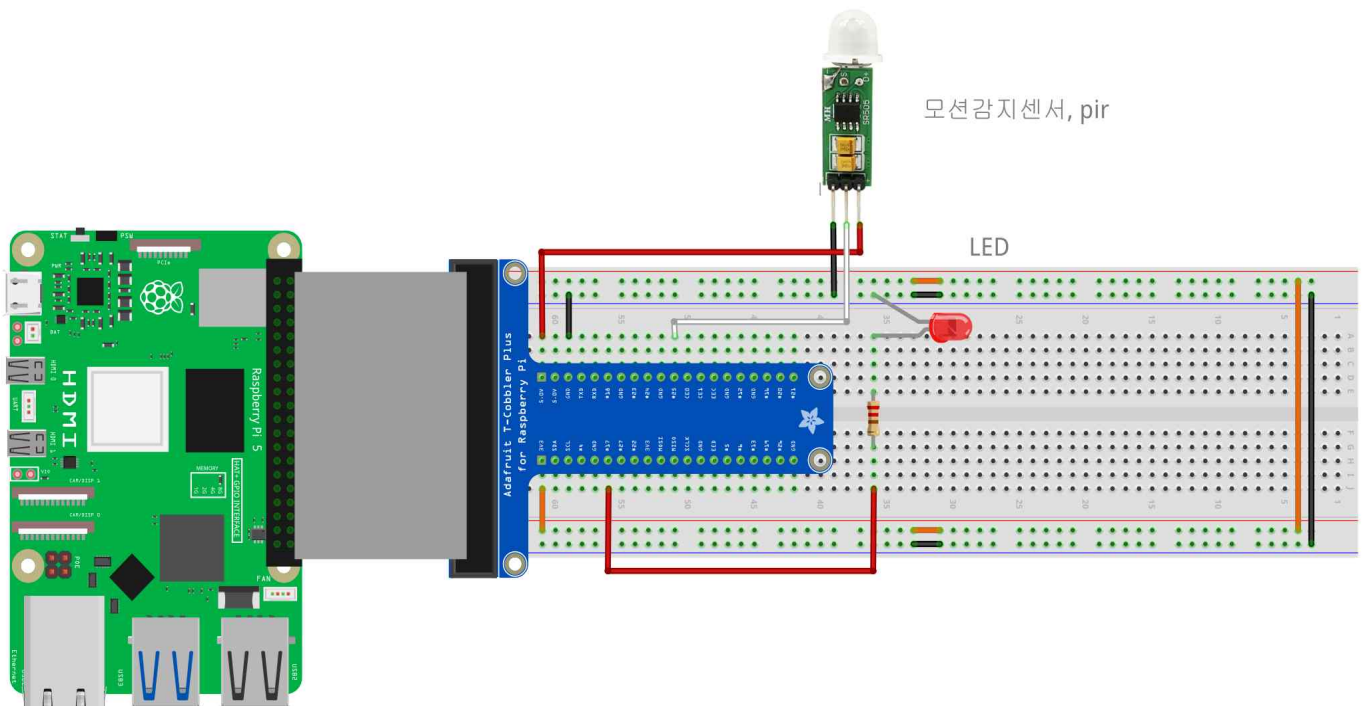
① 연결정보

- ☐ HC-SR505 센서는 구동을 위해 5V 전압이 필요하지만, 센서가 움직임을 감지하고 라즈베리파이로 보내는 출력(OUT) 신호는 안전한 3.3V로 고정되어 나옵니다. 따라서 저항 없이 라즈베리파이 GPIO에 직접 꽂아도 완벽하게 안전합니다!

라즈베리파이	sensor		sensor	BCM 핀번호
GND	GND		red +	GPIO 17
GPIO 25	SIG		GND	GND
5V	VCC			

- ☐ + : 라즈베리파이의 5V 직접 연결
- ☐ - : 라즈베리파이의 GND 연결
- ☐ out : 라즈베리파이의 GPIO, 여기서는 GPIO 25번에 연결

② 회로도



```
# file: MotionSensor_led.py
# 동작 감지 센서 값에 따라 LED on/off
# 작업자 동작 감시시 LED on, 작업자 미동작시 LED off

from gpiozero import MotionSensor, LED
from signal import pause

pir = MotionSensor(25)
led = LED(17)

print('Press Ctrl+C to exit')
print('-'*30)

pir.when_motion = led.on
pir.when_no_motion = led.off

pause()
```

4) 스마트 팩토리 적용 실습 - 위험 구역 경고 시스템

- ① 목표 : 위험 구역에 사람의 움직임이 감지되면 Red LED가 켜지고 경고 안내문이 출력하고 움직임이 없으면 Green LED와 정상 안내문을 출력해보자.

```
# file: MotionSensor_led_2.py
# 동작 감지 센서 값에 따라 LED on/off
#

from gpiozero import MotionSensor, LED
from signal import pause

pir = MotionSensor(25)
red_led = LED(17)
green_led = LED(27)

red_led.off()
green_led.on()

def motion_detected():
    red_led.on()
    green_led.off()
    print('움직임 감지: red_led ON, green_led OFF')

def no_motion_detected():
    red_led.off()
    green_led.on()
    print('움직임 감지 안됨: red_led OFF, green_led ON')

print('Press Ctrl+C to exit')
print('-'*30)

pir.when_motion = motion_detected
pir.when_no_motion = no_motion_detected

pause()
```

□ Tip (고장 났다고 할 때):

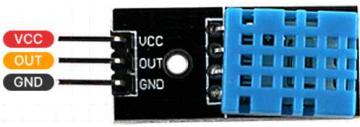
실습을 하다보면 "손을 치웠는데도 불이 안 꺼져요! 고장 났나 봐요!"라고 느낄 수 있습니다. 이건 "HC-SR505 센서 내부에는 움직임이 멈춰도 약 8초간 불을 유지해 주는 딜레이 기능이 내장되어 있어서 정상적으로 작동하는 것"입니다. 실제 화장실이나 복도의 센서등도 사람이 나간 후 일정 시간 뒤에 꺼지는 것과 완벽히 같은 원리입니다.

5. 작업장 환경 측정 : 온습도 센서(DHT11)를 활용한 온도 측정

1) 개요

① 온습도 센서(Humidity and Temperature Sensor)

DHT11 모듈은 내부에 정전식 습도 센서와 써미스터를 포함하고 있으며, 이를 통해 습도와 온도를 측정합니다. 센서는 데이터를 디지털 신호로 변환하여 마이크로컨트롤러로 전송합니다.

부품명	DHT-11	DHT11 모듈
이미지		
온도측정범위	$-20^{\circ}\text{C} \sim 50^{\circ}\text{C} \pm 2^{\circ}\text{C}$	
습도측정범위	$5\% \sim 95\%\text{RH} \pm 5\text{RH}$	
출력값	디지털	
전원	$3.3\text{V} \sim 5\text{V}$	

2) 라이브러리 : adafruit-circuitpython-dht

① 개요

- ☐ DHT 시리즈 온습도 센서를 제어하기 위해 Adafruit에서 제공하는 Python 라이브러리
- ☐ source : https://github.com/adafruit/Adafruit_CircuitPython_DHT
- ☐ DHT11, DHT22, AM2302와 같은 센서를 지원

② 필수 패키지 설치

- ☐ 시스템 패키지 업데이트

```
$ sudo apt update
$ sudo apt upgrade -y
```

- ☐ GPIO 제어용 라이브러리 설치

```
$ sudo apt install libgpiod3 -y
```

- y : 설치 사전 동의
- 설치 버전 : (2.2.1-2). (2025.11월 기준)

- ☐ adafruit-blinka 패키지 설치

```
$ pip3 install adafruit-blinka --break-system-packages
```

- adafruit-blinka : CircuitPython 라이브러리를 Raspberry Pi에서 쓸 수 있게 해줌
- --break-system-packages : 강제로 전역 설치, 강제 설치 권장하지 않으나 i2c LCD 테스트를 하기 위한 용도로 사용합니다. 가상환경에서 사용을 권장합니다.

- ☐ python 패키지 설치

```
$ pip3 install adafruit-circuitpython-dht --break-system-packages
```

③ 주요 함수

- ☐ adafruit_dht.DHT11(pin number)
 - DHT11 및 DHT22 센서 객체를 생성
 - pin number : 센서가 연결된 BCM GPIO 번호
 - 예 : 21번 핀에 연결, dht 객체 생성

```
import board
import adafruit_dht

dht = adafruit_dht.DHT11(board.D21)
```

○ 연결 후 동작이 되지 않을 경우

- Adafruit의 adafruit_dht 라이브러리는 원래 CircuitPython 보드(Feather, Metro, ESP32 등)를 위해 만들어 졌고, 이 라이브러리에서 DHT 센서를 읽을 때는 pulseio라는 모듈을 사용
- pulseio는 핀의 펄스 길이(High/Low 시간)를 아주 정밀하게 측정하는 기능을 제공
- DHT11/22 센서는 데이터를 1-wire 방식으로 보내 펄스 길이로 0과 1을 구분
- 라즈베리파이의 CircuitPython 환경에는 pulseio 모듈이 지원되지 않음.
- 라즈베리파이에서 adafruit_dht.DHT11(board.D21)처럼 그냥 쓰면 에러가 나기도 함.

```
import board
import adafruit_dht

dht = adafruit_dht.DHT11(board.D21, use_pulseio=False)
```

☐ dht.temperature : 온도 값

☐ dht.humidity : 습도 값

- dht 센서에서 온도(dht.temperature)를 읽어 temp 변수에 저장
- dht 센서에서 습도(dht.humidity)를 읽어 hum 변수에 저장

```
temp = dht.temperature
hum = dht.humidity
```

④ 에러 상황시

☐ Error reading sensor: A full buffer was not returned. Try again.

- 센서 인식 못함. 다시 실행하면 됨.

☐ Unable to set line 21 to input

- 센서가 이미 실행되고 있다는 의미.
- 아래 명령어 실행한 후 프로그램 다시 실행

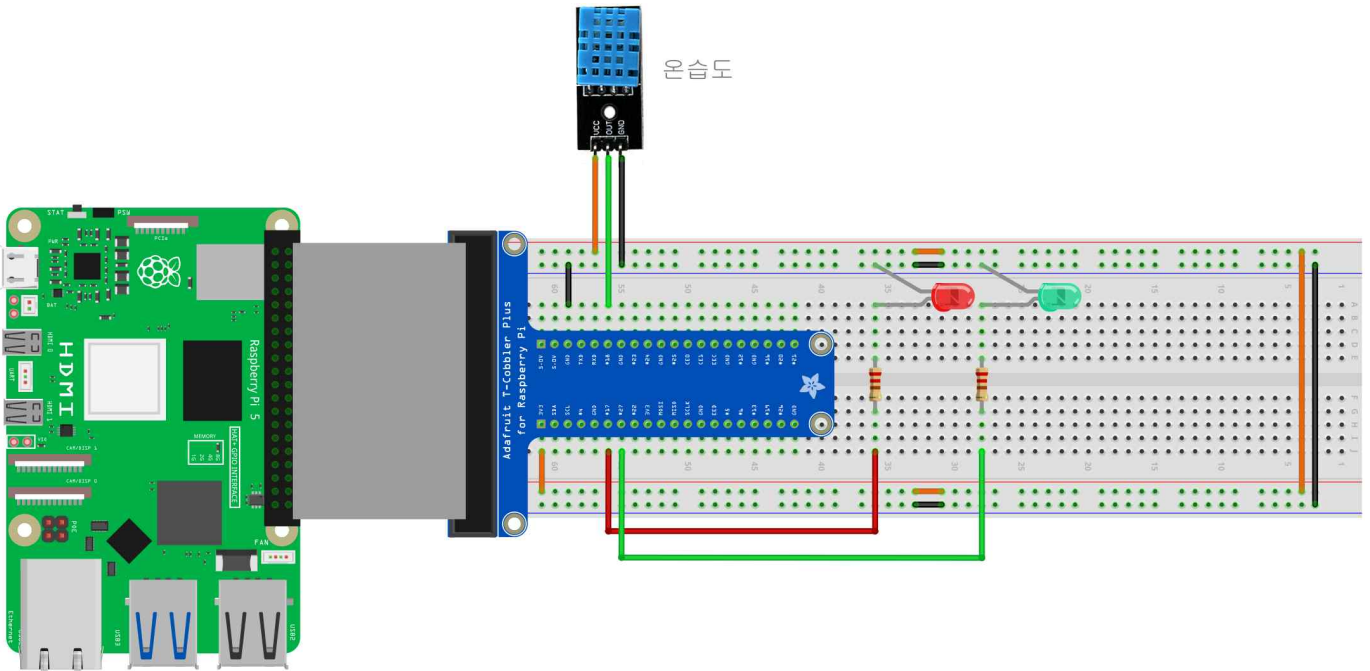
```
$ sudo pkill -f python3
```

3) 연결정보 및 회로도

① 연결정보

라즈베리파이	sensor		sensor		BCM 핀번호
3.3V	VCC		LED	red +	GPIO 17
GPIO 21	OUT		LED	green +	GPIO 27
GND	GND			GND	GND

② 회로도



③ 온도와 습도를 출력해보자.

```
# file: hum_temp_1.py
# 온도, 습도 측정

import board, adafruit_dht
from time import sleep

dht = adafruit_dht.DHT11(board.D21, use_pulseio=False)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    temp = dht.temperature
    print(f'Temperature: {temp:.1f}C')

    hum = dht.humidity
    print(f'Humidity: {hum:.1f}%')

    print(f'Temperature : {temp:.1f}C, Humidity : {hum:.1f}%')
    print('-'*30)

    sleep(2)
```


4) 스마트 팩토리 적용 실습

① 목표

정밀 반도체 공정이나 식품 가공 공장에서는 온도와 습도가 모두 특정 범위 안에 유지되어야만 불량품이 발생하지 않습니다. 온도가 20~28℃ 사이이고, 동시에 습도가 40~60% 사이일 때만 초록색 LED(정상)를 켜고, 이 범위를 하나라도 벗어나면 빨간색 LED(경고)를 켜는 '항온항습 모니터링 시스템'을 만들어 봅니다.

```
# file: dht11_led.py
# 스마트 팩토리 환경 모니터링 시스템

import board, adafruit_dht
from time import sleep
from gpiozero import LED

# 1. 객체 생성
dht = adafruit_dht.DHT11(board.D21, use_pulseio=False)
red_led = LED(17)
green_led = LED(27)

def check_temperature(temp):
    if temp is not None:
        if temp >= 28:
            print(f'Temperature: {temp:.1f}C, [경고] 온도가 너무 높습니다.')
            return False
        elif temp <= 20:
            print(f'Temperature: {temp:.1f}C, [경고] 온도가 너무 낮습니다.')
            return False
        else:
            print(f'Temperature: {temp:.1f}C, [정상] 최적 온도입니다.')
            return True
    else:
        print('온도 데이터가 없습니다.')
        return False

def check_humidity(hum):
    if hum is not None:
        if hum >= 60:
            print(f'Humidity: {hum:.1f}%, [경고] 습도가 너무 높습니다.')
            return False
        elif hum <= 40:
            print(f'Humidity: {hum:.1f}%, [경고] 습도가 너무 낮습니다.')
            return False
        else:
            print(f'Humidity: {hum:.1f}%, [정상] 최적 습도입니다.')
            return True
    else:
        print('습도 데이터가 없습니다.')
        return False

print('스마트 팩토리 환경 모니터링 시작 (Press Ctrl+C to exit)')
print('-'*30)

try:
    while True:
        try:
            temp = dht.temperature
```

```

        hum = dht.humidity

        temp_ok = check_temperature(temp)
        hum_ok = check_humidity(hum)

        if temp_ok and hum_ok:
            green_led.on()
            red_led.off()
        else:
            green_led.off()
            red_led.on()

    except RuntimeError as error:
        print(f"센서 읽기 오류 (재시도 중): {error.args[0]}")

    print('-'*30)
    sleep(2)

except KeyboardInterrupt:
    print("\n프로그램을 사용자가 강제 종료했습니다.")

finally:
    print('센서 종료')
    dht.exit()

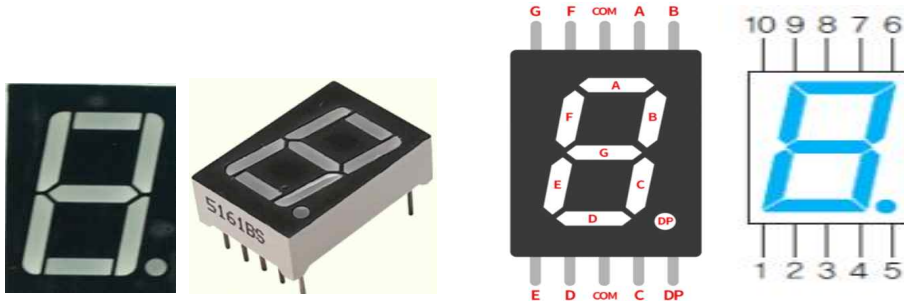
```

6. 수동 불량품 카운터 (버튼 + FND)

1) 개요

① FND (Flexible Numeric Display)

- ☐ LED의 조합으로 숫자를 표시하는 장치
- ☐ 7개의 LED를 사용하기 때문에 7-segment 라고도 함.
- ☐ 숫자 뿐만 아니라 간단한 기호나 16진수까지 표현 가능



② 구조

- ☐ 윗면 중앙(8번), 아랫면 중앙(3번) : GND 연결
- ☐ 7-segment 8개(1, 2, 4, 5, 6, 7, 9, 10)에 각각 330Ω 저항을 연결 후 라즈베리파이의 GPIO 핀에 각각 연결

2) 클래스 : LEDCharDisplay

① 함수원형 및 parameters

```
LEDCharDisplay(pins, dp=None, initial_value=0.0, active_high=True )
```

- pins : a ~ g까지 연결된 GPIO 핀 번호를 순서대로 입력
- dp : dot pont(.) 연결된 GPIO 핀번호, 기본값은 None
- initial_value(float) : 초기값설정,
 - float : 0~1사이의 실수값으로 LED 초기 밝기 지정
 - 생략시 0 (기본값) : LED가 off 상태로 시작
- active_high : FND 방식이 cathode/anode 에 따라 설정
 - 생략 : True, Cathode방식, 대부분 제품에 해당
 - active_high=False : Anode 방식일 경우 사용
- 예

```
from gpiozero import LEDCharDisplay
```

```
display = LEDCharDisplay(20, 21, 19, 13, 6, 16, 12, dp=26)  
display.value = str(3)          #숫자 3이 표시
```

- display.value = '3', 3의 작은 따옴표(' ') 생략하면 에러.. 숫자를 문자 형태로 표시

② 변수 property

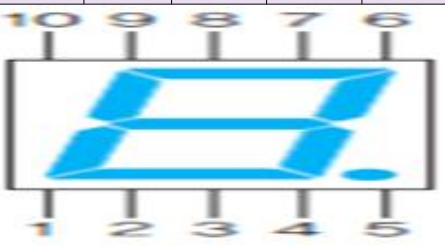
- value : 표시할 글자.

3) 연결정보 및 회로도

① 부품

재료명 모델명	이미지	재료명	이미지	수량
FND 5011AS		저항 330Ω		8

② 연결정보

gpio	12	16	GND	20	21
핀	g	f	GND	a	b
칩					
	10	9	8	7	6
핀	e	d	GND	c	dp
gpio	6	13	GND	19	26

기능	연결선 색	GPIO
a	빨강	20
b	주황	21
c	노랑	19
d	초록	13
e	파랑	6
f	남색(cyan)	16
g	보라	12
dot	갈색	26

sensor		라즈베리파이
버튼	+	GPIO 17
	GND	GND

- ☐ abcdefg 연결선을 무지개 색순으로 표현.
- ☐ a 연결은 빨간 연결선, b는 주황 연결선, c는 노랑 연결선 순으로 표현
- ☐ 무지개 남색은 cyan 연결선으로 표시, dp는 갈색으로 표시

③ 회로도

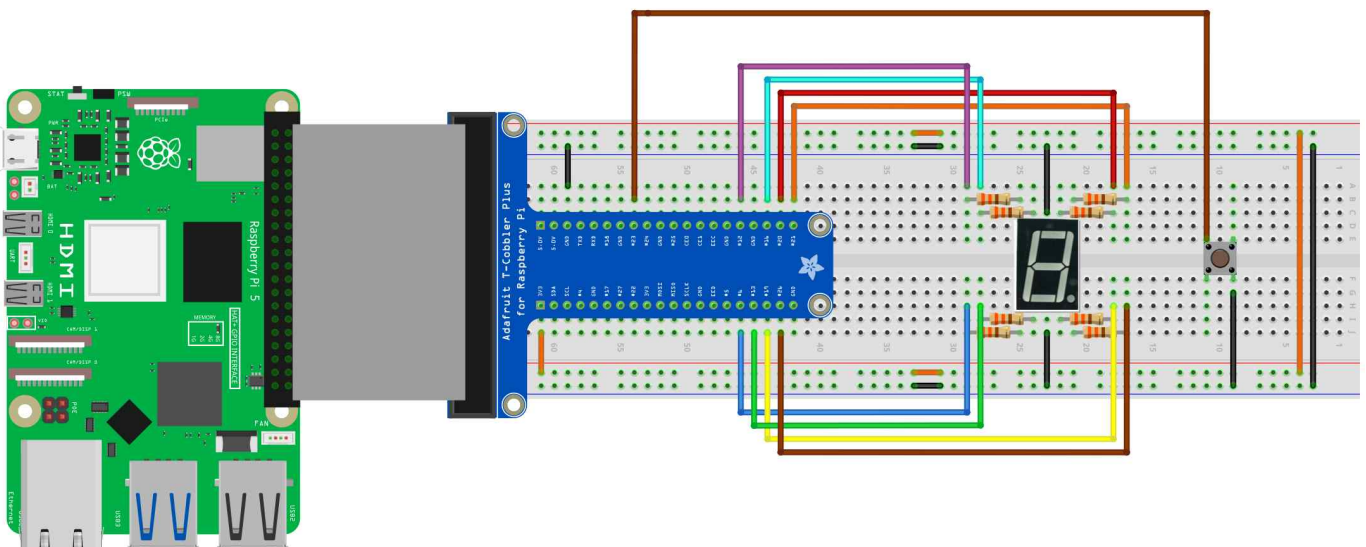


그림 119 fnd_button.png

○ 저항끼리 서로 붙지 않도록 세밀하게 연결하세요.

④ 동작 테스트

☐ fnd에 8FAH를 표시해보자.

```
# file name : fnd_1.py
# FND 표시

from gpiozero import LEDCharDisplay
from signal import pause

display = LEDCharDisplay(20, 21, 19, 13, 6, 16, 12, dp=26)
#display = LEDCharDisplay(20, 21, 19, 13, 6, 16, 12, dp=26, active_high=False)

print('Press Ctrl+C to exit')
print('-'*30)

display.source_delay = 1
display.source = '8FAH'

pause()
```

- display.source_delay = 1 : 1초 지연. 생략시 첫글자 8이 표시되지 않음.
숫자표시 준비시간
- 문자 표시할 때 반대로 켜지는 경우에 active_high=False 활용

☐ 7-segment에 0에서 9까지 카운트다운을 1초 단위로 표시해보자

○ while() 활용

```
# file name : fnd_3.py
# 0부터 9까지 표시

from gpiozero import LEDCharDisplay
from time import sleep

display = LEDCharDisplay(20, 21, 19, 13, 6, 16, 12, dp=26)

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    for i in range(10):
        display.value = str(i)
        print(f'Display: {i}')
        sleep(1)
```

- str() 함수 : 숫자를 문자형으로 변환

4) 스마트 팩토리 적용 실습

- ① 목표 : 컨베이어 벨트에서 품질 검수를 하는 작업자가 불량품을 발견할 때마다 '검수 버튼'을 누르면, FND 전광판의 숫자가 1씩 증가하는 '불량품 누적 카운터'를 만들어 봅니다. 숫자는 0부터 9까지 증가하며, 9를 넘으면 다시 0으로 초기화됩니다.

```
# file name : fnd_counter.py
# FND 카운터 프로그램
# 버튼을 누를 때마다 카운트가 올라갑니다.
# 카운터가 9를 넘으면 0으로 초기화합니다.

from gpiozero import Button, LEDCharDisplay
from signal import pause

display = LEDCharDisplay(20, 21, 19, 13, 6, 16, 12, dp=26)
btn_plus = Button(23)

count = 0
display.value = str(count)

print("스마트 팩토리 불량품 카운터 가동 (Press Ctrl+C to exit)")
print("버튼을 누를 때마다 카운트가 올라갑니다.")
print('-'*30)

def add_count():
    global count
    count = count + 1

    # 카운터가 9를 넘으면 0으로 초기화
    if count > 9:
        count = 0
        print('카운터가 9를 넘었습니다. 0으로 초기화합니다.')

    # FND 표시
    display.value = str(count)
    print(f'불량품 카운트: {count}')

btn_plus.when_pressed = add_count

pause
```

7. 온습도(DHT11) 센서를 활용한 온습도 디스플레이

1) 개요

① 액정 표시 장치 : Liquid Crystal Display

- ☐ 얇은 액정판 아래 조명을 비추는 장치로서 액정판의 전류 흐름을 제어하여 문자나 그림을 표시
- ☐ Liquid Crystal이란 액체처럼 유체의 성질을 가지면서 고체처럼 광학적 성질을 가지는 물질
- ☐ 액정은 고체 표면에 액정이 특정 방향으로 정렬할 수도 있어, 전기적 신호를 가해 원하는 방향으로 배열하여 빛을 투과시키거나 차단시킬 수 있음

② 종류

☐ 16×2 LCD

- 가장 일반적인 LCD로 한줄에 5×8 dot 크기의 문자(1byte)를 16개 표현 가능한 2줄의 LCD
- 5V 전원공급, 3.3V 전원공급도 가능
- 기본적으로 PIN1, PIN2, PIN15, PIN16으로 LCD를 구동하며 A and K로 구동.
- 16×2 LCD는 16개의 핀이 있고 전원을 빼더라도 라즈베리파이의 GPIO 12개가 필요
- 라즈베리파이의 GPIO의 핀을 많이 사용하는 단점.



[그림 120] WH1602W 16×2 LCD



그림 121 I2C LCD 앞면과 뒷면

☐ I2C LCD

- Inter-Integrated Circuit LCD
- ‘아이-투-씨’라 읽으시면 됩니다. 공식문서에는 ‘아이-스퀘어-씨(i-square-c)’라 씁니다.
- 전원과 GND를 제외하면 2개의 GPIO핀(GPIO 2, GPIO 3) 만 사용
 - VCC : 라즈베리파이 5V
 - GND ➡ 라즈베리파이 GND
 - SDA ➡ 라즈베리파이 GPIO 2 (SDA)
 - SCL ➡ 라즈베리파이 GPIO 3 (SCL)

③ I²C 통신 연결

☐ 개념

- 마이크로프로세서끼리 연결, 마이크로프로세서와 센서 연결 후 데이터 통신
- 하나의 마스터와 하나 이상의 슬레이브(이론상 127개)와 시리얼 통신 가능.
- 동기 양방향 2선식 bus
- 데이터 오류에 대한 안정성은 없다.
- 데이터 안정을 위해서는 smbus 라이브러리를 활용함.

☐ 신호

- 데이터 통신(양방향) : SDA (GPIO 2)
- 클럭 신호 : SCL (GPIO 3)

④ 라즈베리파이 환경설정

- GUI환경에서 I2C 통신 - enable



2) 라이브러리 : RPLCD

① 개요

- ☐ RPLCD (Raspberry Pi Liquid Crystal Display)
- ☐ 라즈베리파이에서 HD44780 컨트롤러 기반의 문자 LCD(예: 16x2, 20x4)를 제어하기 위한 파이썬 라이브러리(한글 지원 안됨)

② 연결 방식

- ☐ I2C 방식 : PCF8574와 같은 I/O 확장 칩을 통해 단 2개의 GPIO 핀(SDA, SCL)만으로 LCD를 제어. (가장 널리 쓰이는 방식)

③ 필수 패키지

- ☐ SMBus(System Management Bus) 라이브러리
 - 파이썬 코드에서 I2C 장치와 직접 통신할 수 있게 함
 - I²C는 기본적으로 2선(SDA, SCL) 직렬 통신 규격
 - 주요 용도 : LCD, RTC, DHT온습도, MPU6050 관성장치 등 같은 I²C 기반 장치 제어
- ☐ i2c-tools 라이브러리
 - I²C 디바이스 진단/테스트용 리눅스 유틸리티 모음
 - 설치 후 제공되는 명령어
 - i2cdetect : 연결된 I²C 장치의 주소 스캔 (★)
 - i2cget : 특정 장치 레지스터 값 읽기
 - i2cset : 특정 장치 레지스터에 값 쓰기
 - i2cdump : 장치의 전체 레지스터 덤프

☐ 설치

```
$ sudo apt update
$ sudo apt upgrade -y
$ sudo apt install python3-smbus i2c-tools -y
```

④ RPLCD 라이브러리 설치

- ☐ 강제 설치 : --break-system-packages 옵션 사용

```
$ pip install RPLCD --break-system-packages
```

```
pi@zeroToAI:~ $ pip install RPLCD --break-system-packages
Defaulting to user installation because normal site-packages is not writeable
Collecting RPLCD
  Downloading rplcd-1.4.0-py3-none-any.whl.metadata (6.2 kB)
  Downloading rplcd-1.4.0-py3-none-any.whl (48 kB)
Installing collected packages: RPLCD
Successfully installed RPLCD-1.4.0
```

- 강제 설치는 권장하지 않습니다만 테스트용으로만 사용합니다. 프로젝트나 개발 시에는 가상환경 활용을 권장합니다.

☐ I²C LCD 연결 장치 주소 확인

; lcd를 연결한 후 아래 명령으로 연결 여부 확인

```
$ sudo i2cdetect -y 1
```

```

pi@zeroToAI:~$ sudo i2cdetect -y 1
    0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
10:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
20:  -- -- -- -- -- -- -- 27 -- -- -- -- -- --
30:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
40:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
50:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
60:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
70:  -- -- -- -- -- -- -- -- -- -- -- -- -- --
pi@zeroToAI:~$

```

파악된 주소는 27. 16진수이기에 표기 및 사용은 '0x27'로 하시면 됩니다.

⑤ 동작 method

□ 기본 제어 (Basic Control)

LCD의 화면을 관리하는 기본적인 함수들입니다.

함수	기능	예
clear()	LCD 화면의 모든 텍스트를 지우고 커서를 맨 앞(0, 0)으로 이동	lcd.clear()
home()	화면의 텍스트는 지우지 않고, 커서만 맨 앞(0, 0)으로 이동	lcd.home()
close()	LCD 사용을 종료하고 GPIO 핀 등 사용한 리소스를 정리 clear=True로 설정하면 화면을 지우면서 종료	lcd.close() lcd.close(clear=True)

□ 텍스트 및 커서 (Text & Cursor)

화면에 글자를 쓰거나 커서의 위치와 모양을 제어합니다.

함수	기능	예
write_string()	- 현재 커서 위치부터 문자열을 씀 - WrWn (또는 WnWr) 포함하면 줄바꿈을 지원	lcd.write_string('Hello World') lcd.write_string('Line 1\r\nLine 2')
cursor_pos()	- 커서 위치를 지정한 (행, 열)로 이동 (행과 열은 0부터 시작)	lcd.cursor_pos = (1, 0) (두 번째 줄 첫 번째 칸 이동)
cursor_visible	- 커서의 표시 여부를 설정 - True : 밑줄(_) 모양의 커서 나타남 - False : 커서가 사라짐	lcd.cursor_visible = False
blink	- 커서의 깜빡임 여부를 설정 - True : 커서가 깜빡임 (cursor_visible이 True일때만 가능)	lcd.blink = True

□ 백라이트 제어 (Backlight Control)

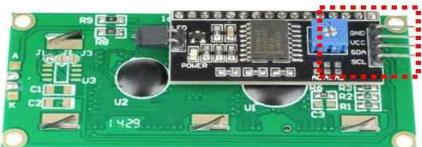
I2C 방식 LCD 모듈의 백라이트를 켜고 끕니다. (GPIO 병렬 연결 방식에서는 작동하지 않을 수 있습니다.)

○ backlight_enabled = True / False (속성)

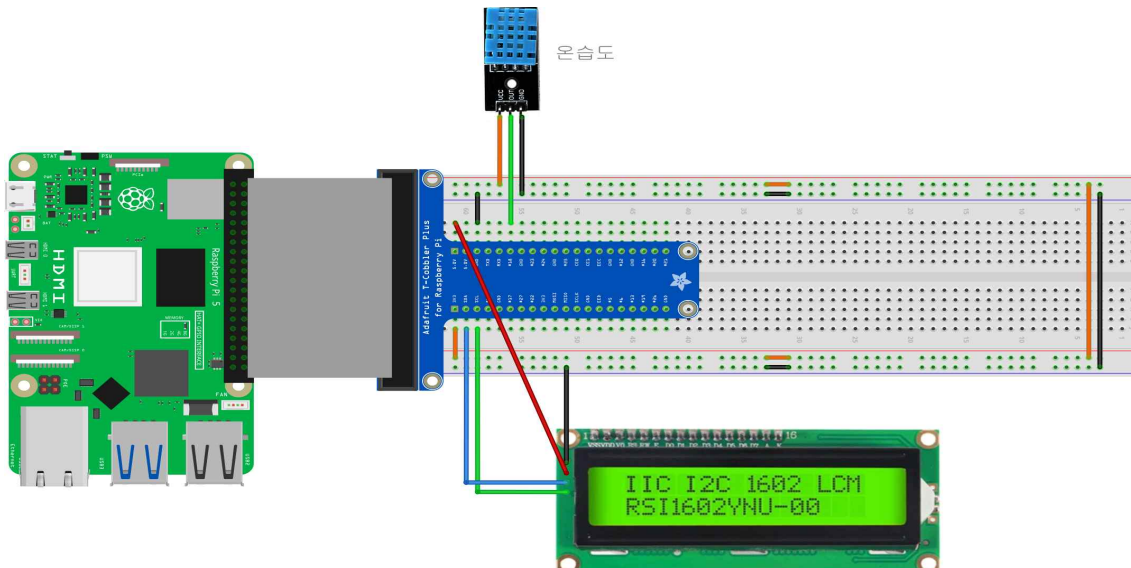
- True로 설정하면 백라이트를 켜고, False로 설정하면 끕니다.
- 예시: lcd.backlight_enabled = True (백라이트 켜기)

3) 연결정보 및 회로도

① 연결정보

sensor		라즈베리파이
	-GND	GND
	-VCC	5V
	-SDA	GPIO 2
	-SCL	GPIO 3

② 회로도



4) 실습해보기

① 첫번째줄에는 "Zero To AI !!", 두번째 줄에는 "Physical AI !!" 를 출력해보자.

```
# file name : lcd_1.py
# LCD 표시

from RPLCD.i2c import CharLCD
from signal import pause

# 1. I2C 설정 및 초기화
lcd = CharLCD(i2c_expander='PCF8574',
              address=0x27,
              port=1,
              cols=16,
              rows=2,
              charmap='A00')

lcd.clear()
lcd.backlight_enabled = True

# 2. LCD 화면에 텍스트 출력
lcd.cursor_pos = (0, 0)
lcd.write_string('Zero To AI !!')

lcd.cursor_pos = (1, 0)
lcd.write_string('Physical AI !!')
```

```
print('LCD 출력 완료. (Press Ctrl+C to exit)')
print('-'*30)

# 3. 프로그램 대기 (글자가 계속 떠 있도록 유지)
pause()
```

- i2c_expander='PCF8574' : I²C 방식의 LCD 백팩 모듈(확장 칩),
- A00: HD44780 LCD 컨트롤러의 표준 문자 세트, 영문/숫자/기본 기호만 표시, 한글은 지원 안함.

5) 응용해보기

① 현재 날짜, 시간을 1초 단위로 표시해보자.

☐ 1번째 줄에는 오늘 날짜, 2번째 줄에는 현재 시간을 출력해보자.

```
# file: lcd_3.py

from time import sleep
from datetime import datetime
from RPLCD.i2c import CharLCD

lcd = CharLCD(i2c_expander='PCF8574',
              address=0x27,
              port=1,
              cols=16,
              rows=2,
              charmap='A00')

# initialize LCD
lcd.clear()
lcd.backlight_enabled = True # backlight on

print('Press Ctrl+C to exit')
print('-'*30)

while True:
    now = datetime.now()

    date_str = f'Date {now.strftime("%Y-%m-%d")}'
    time_str = f'Time {now.strftime("%H:%M:%S")}'

    lcd.cursor_pos = (0, 0)
    lcd.write_string(date_str)

    lcd.cursor_pos = (1, 0)
    lcd.write_string(time_str)

    sleep(1)
```

6) 스마트 팩토리 적용 실습

- ① 목표 : 3절에서 다룬 온습도 센서(DHT11)와 이번 절에서 배운 I2C LCD를 결합하여, 작업장 내 온도와 습도를 실시간으로 표시하는 '스마트 팩토리 환경 종합 대시보드'를 만들어 봅니다.

```
# file: lcd_dht11.py
# 온도 센서 DHT11 모듈을 사용하여 온도와 습도를 측정하고 LCD 화면에 출력하는 프로그램

import board, adafruit_dht
from RPLCD.i2c import CharLCD
from time import sleep

# 1. 객체 생성
dht = adafruit_dht.DHT11(board.D21, use_pulseio=False)

lcd = CharLCD(i2c_expander='PCF8574',
              address=0x27,
              port=1,
              cols=16,
              rows=2,
              charmap='A00')

# 2. 초기화
lcd.clear()
lcd.backlight_enabled = True

print('Press Ctrl+C to exit')
print('-'*30)

try:
    while True:
        try:
            temp = dht.temperature
            hum = dht.humidity

            if hum is not None and temp is not None:
                # \xDF는 LCD에 출력되는 온도(°) 기호입니다.
                lcd.cursor_pos = (0, 0)
                lcd.write_string(f'Temp : {temp:.1f} \xDFC  ')
                lcd.cursor_pos = (1, 0)
                lcd.write_string(f'Hum  : {hum:.1f} %  ')

                print(f'Temperature : {temp:.1f}C, Humidity : {hum:.1f}%')
                print('-'*30)
            else:
                lcd.cursor_pos = (0, 0)
                lcd.write_string('Sensor Error  ')
                lcd.cursor_pos = (1, 0)
                lcd.write_string('Retrying...  ')
                print('Failed to read sensor data.')

        except RuntimeError as error:
            # 에러가 발생해도 멈추지 않고 2초 대기 후 다시 측정
            print(error.args[0])

        sleep(2)

except KeyboardInterrupt:
    print("\n프로그래를 종료합니다.")

finally:
    # 5. 종료 시 화면 지우고 센서 핀 자원 반환
    lcd.clear()
    lcd.backlight_enabled = False
    dht.exit()
```

Chapter 3. 원격 접속을 통한 피지컬AI 기초

1. 원격 접속 환경 구축 (라즈베리파이 엣지 설정)

1) 원격 제어용 소프트웨어 준비 및 핫스팟 설정

작업용 일반 컴퓨터(노트북)에 Raspberry Pi Imager, PuTTY, VNC Viewer 세 가지 프로그램을 다운로드하고 설치합니다.

① 환경설정 : 준비물 체크

하드웨어	소프트웨어
일반 컴퓨터(노트북 또는 데스크톱 PC) Micro SD 카드(16GB 이상) SD 카드 리더기 무선 LAN 카드(wi-fi 동글) ★ 데스크톱 PC일 경우	Raspberry Pi Imager PuTTY VNC Viewer

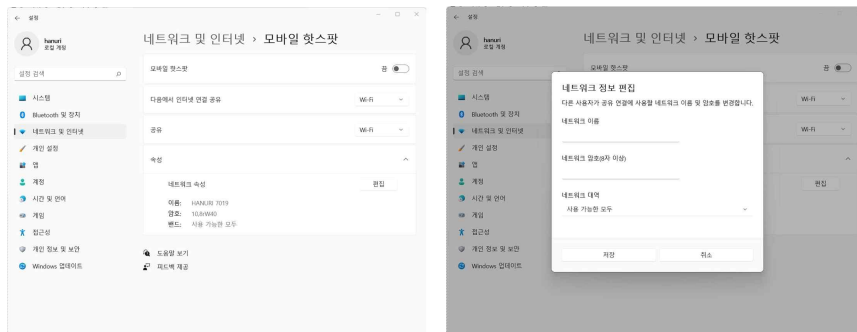
② 모니터 없는 접속을 위한 필수 관문 : 핫스팟 네트워크 묶기

모니터 없이 원격 접속(Headless)을 하려면 라즈베리파이와 내 컴퓨터가 '반드시 같은 와이파이 네트워크'에 연결되어 있어야 합니다. 이를 위해 내 컴퓨터나 스마트폰을 무선 공유기처럼 만드는 '핫스팟(Hotspot)' 기능을 사용합니다.

□ 방법 1. 노트북(윈도우) 모바일 핫스팟 설정(추천 ★)

가장 권장하는 방식입니다. 노트북의 핫스팟을 켜면 라즈베리파이가 노트북으로 직접 접속하므로, 나중에 라즈베리파이의 IP 주소를 윈도우 화면에서 바로 확인할 수 있어 매우 편리합니다.

- [시작] - [설정] - [네트워크 및 인터넷] - [모바일 핫스팟] 켜기
- 네트워크 속성에서 편집 클릭 후 '네트워크 이름', '비밀번호'를 입력 후 저장.
(비밀번호는 반드시 8자리 이상)
- 여기서 설정한 네트워크 이름, 비밀번호는 다음 단계인 라즈베리파이 OS 설치시 꼭 필요하므로 메모해 둡니다. (예 : 이름 ; hanuri, 비밀번호 : 12345678)

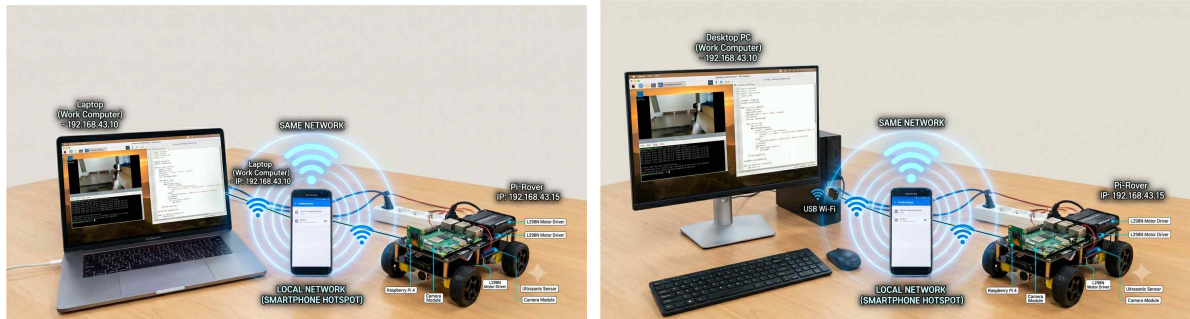


- 유선 인터넷만 연결된 데스크톱 컴퓨터의 경우, USB 무선 LAN 카드(Wi-Fi 동글)를 꽂으면 노트북과 똑같이 모바일 핫스팟 기능을 켤 수 있습니다.)

□ 방법 2. 스마트폰 모바일 핫스팟 설정(대안)

노트북에서 핫스팟을 켤 수 없는 환경이라면 스마트폰을 활용합니다.

- 스마트폰의 설정에서 '모바일 핫스팟'을 켭니다.
- 노트북의 와이파이를 스마트폰 핫스팟에 연결합니다.
- 잠시 후 라즈베리파이도 동일한 스마트폰 핫스팟에 연결되도록 세팅하여, 세 기기(스마트폰, 노트북, 라즈베리파이)가 하나의 네트워크로 묶이게 만듭니다.



모니터 없이 라즈베리파이 연결

로봇이 달리려면 HDMI 케이블을 뽑아야 한다. 노트북 · 라즈베리파이 · 스마트폰을 하나의 망으로 묶는다.



□ 왜 학교 와이파이를 그냥 안 쓰나요?

대학 강의실이나 공공기관의 와이파이는 보안이 매우 강력한 '엔터프라이즈 와이파이'이거나, 해킹 방지를 위해 기기들끼리 통신을 차단하는 'AP 격리(Isolation)' 기능이 켜져 있는 경우가 많습니다. 이 환경에서는 라즈베리파이 원격 접속이 100% 차단됩니다. 따라서 우리의 PC를 개인용 미니 공유기(핫스팟)로 만들어 이 문제를 우회하는 것입니다.

2) [실습] 모니터 없이 설치하기 : 헤드리스(Headless) 원격 접속 방법

① 이미지(Imager) 다운로드 및 설치

☐ 다운로드

- 라즈베리파이 공식 홈페이지
(<https://www.raspberrypi.com/software/>) 접속.
- 자신의 PC 운영체제에 맞는 'Raspberry Pi Imager'를 다운로드하고 설치하여 실행함.
(파일명 : imager_2.0.10.exe, 202600731 기준)

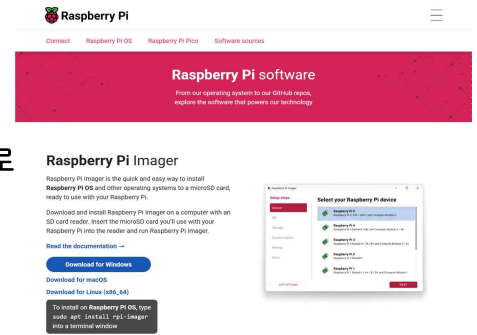
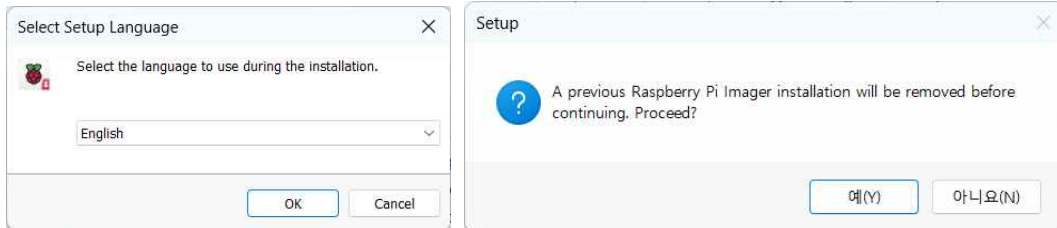


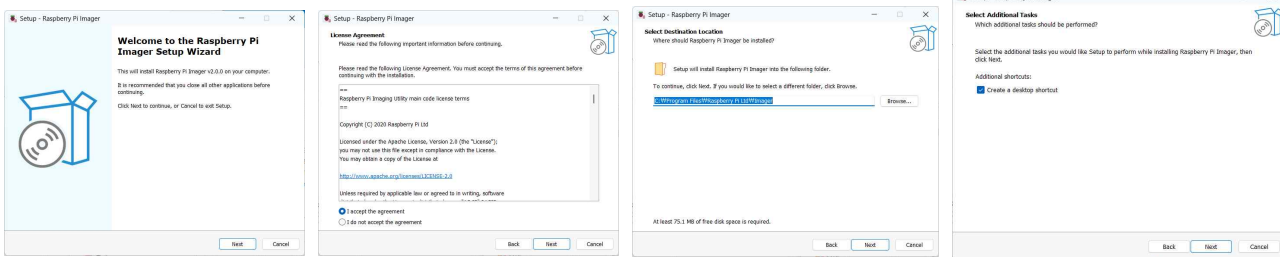
그림 131 라즈베리파이공식 홈

☐ 설치

- 언어 선택, English, Next 클릭, 이전 버전이 있으면 삭제..예(Y) 선택
- 바탕화면에 Imager 아이콘 확인

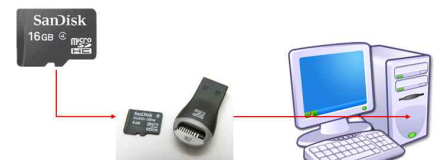


- 크게 신경쓰실꺼 없습니다. 그냥 Next...



② micro SD card 및 SD card reader 준비

- ☐ micro SD card : 최소 16GB, 추천 32GB 이상
- ☐ SD 메모리 카드 연결
 - SD카드를 USB 젠더에 삽입
 - USB 젠더를 컴퓨터 USB 포트에 삽입
- ☐ 일부 노트북은 Micro SD card를 직접 읽을 수 있음. 확인해보세요.



③ Micro SD card에 OS 설치 (Imager 굽기)

□ 1. 윈도우 - [시작] - Raspberry Pi - Raspberry Pi Imager 실행
혹은 바탕화면 Imager 실행

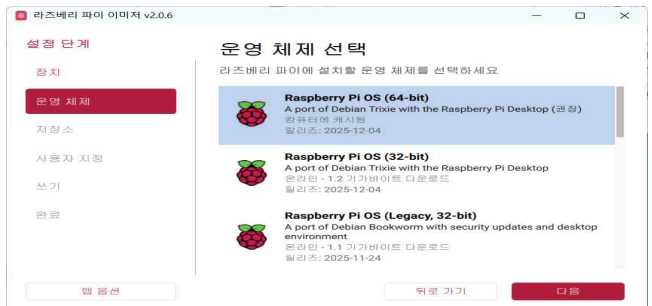
□ 2. 장치 선택

○ 사용 라즈베리파이 모델 선택



□ 3. 운영체제 선택

; 64bit 선택(매우 중요) PyTorch, YOLO 사용시 32비트는 에러날 수 있음.



□ 4. 저장장치 선택



□ 5. OS 커스터마이제이션

○ 호스트 이름 입력

- 예 zeroToAI, 공백없이 영어로 다른 이름 가능. 보통 본인의 이니셜, raspberry, pipi 등등



□ 6. OS 커스터마이제이션

○ 지역 선택

- Seoul(South Korea)
- Ais/Seoul



□ 7. OS 커스터마이제이션

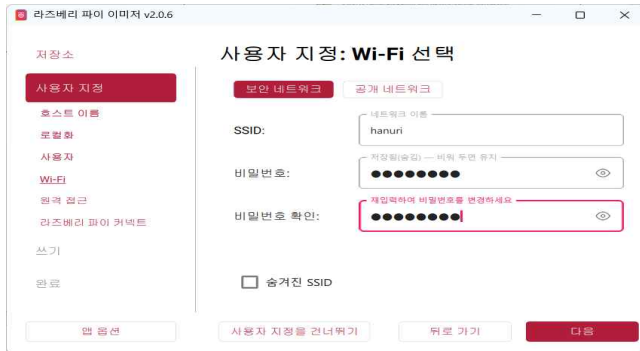
○ 사용자 이름 선택

- 너무 길거나 어려운 비밀번호는 자제
- 예) 아이디 : pi, 비밀번호 : 12345678



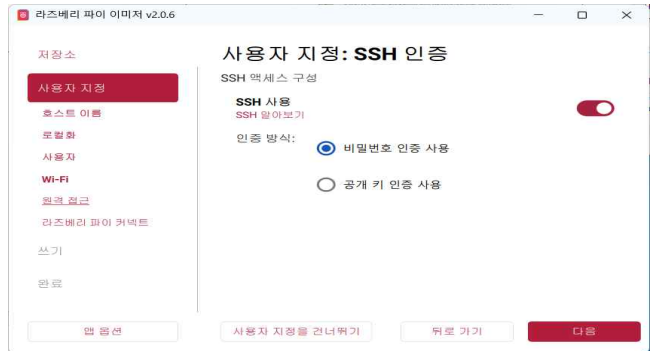
□ 8. OS 커스터마이제이션

- 사용자 지정 : Wi-Fi 선택
- SSID : 위에서 지정한 모바일 핫스팟 ID, 비밀번호
(예 : hanuri / 12345678)



□ 9. OS 커스터마이제이션

- SSH 사용여부 - 활성화(★ 필수)
- 외부 원격접속 허용
- SSH 접속(putty 접속시 필수)



□ 10. OS 커스터마이제이션

- 라즈베리파이 커넥트

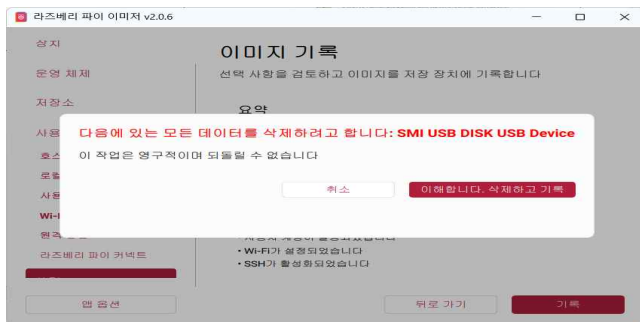
- 원격 접속 통계, 라즈베리파이OS 관련 통계 수집 등 기록 전송 여부, **비활성**



□ 11. 이미지 기록 내용 요약 - 내용 확인



□ 12. 내용물 삭제 경고



□ 13. 완료



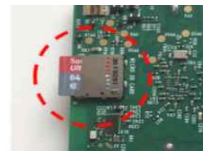
□ 윈도우에서 디스크 삽입 창이 뜨면 취소..

Micro SD card를 라즈베리파이에 장착

- SD카드를 라즈베리파이 뒷면의 소켓에 삽입
- 그림처럼 sd card의 로고가 보이도록 삽입

④ 부팅 (전원 케이블 연결)

- 전원 케이블을 연결한 후 바로 '⑦번 모바일 핫스팟 활성화'를 진행하세요.
- 부팅이 시작되고 라즈베리파이 LED가 깜박거립니다.



⑤ 모바일 핫스팟 활성화

- ☐ 시작메뉴 - 마우스 오른쪽 버튼 - 네트워크 연결 - 모바일 핫스팟 활성화

⑥ 와이파이 연결 (모바일 핫스팟)

- ☐ 활성화 후 잠시 기다리면 라즈베리파이(호스트명)으로 접속, 예 zeroToAI
- ☐ 가장 중요한 것은 IP 주소 확인

⑦ putty 다운로드 및 설치, 실행

- ☐ putty 접속의 유일한 목적은 '라즈베리파이의 vnc 기능 활성화'
- ☐ putty를 다운로드 및 설치(<https://www.putty.org/>)
- ☐ 시작메뉴 - PuTTY 클릭 후 위에서 확인한 IP 입력 후 Open 클릭

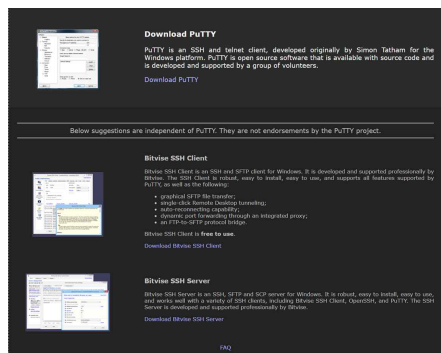


그림 154 putty 사이트

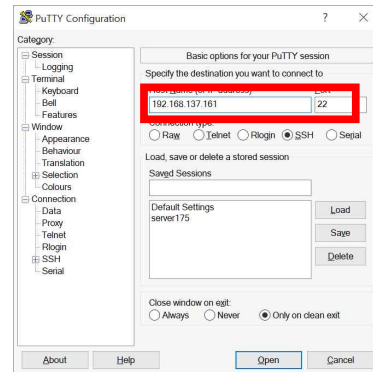


그림 155 putty 접속화면

- ☐ 접속 화면이 나오면 ID와 비밀번호 입력 후 엔터키.. 접속
 - 라즈베리파이 OS 설치시 만들었던 ID, 비밀번호 입력

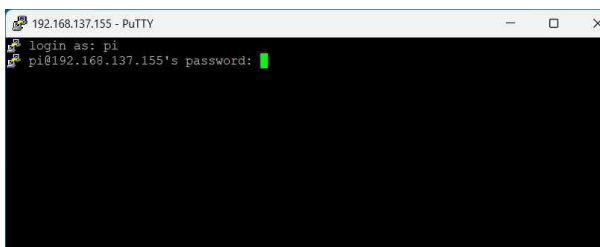


그림 157 putty 접속 터미널 화면

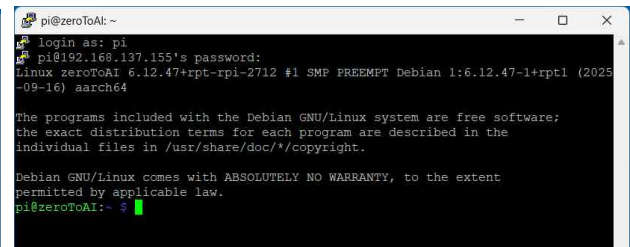


그림 158 접속 성공 화면

⑧ 라즈베리파이 vnc 활성화(putty 창에서 작업)

- ☐ 라즈베리파이 환경설정 실행 : sudo raspi-config
 - \$ 프롬프트를 의미함. 입력하지 않음.

```
$ sudo raspi-config
```

- ☐ 키보드 화살표(↓)를 이용해 3 Interface Options 이동, 엔터 (그림 39)
- ☐ 키보드 화살표(↓)를 이용해 I3 VNC 이동, 엔터 (그림 40)

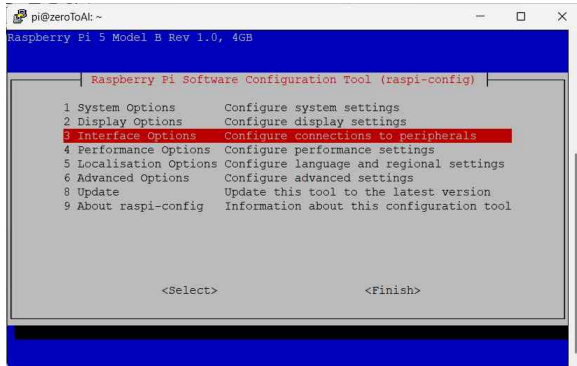


그림 159 환경설정 초기화면

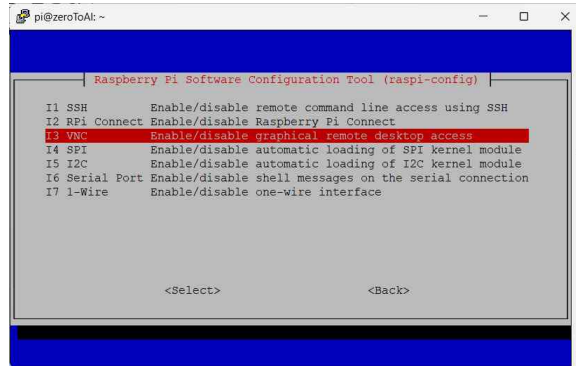


그림 160 환경설정 - VNC 선택

- ☐ VNC server 활성화 물어보면 <YES> 엔터, 활성화 됨 <OK> 엔터

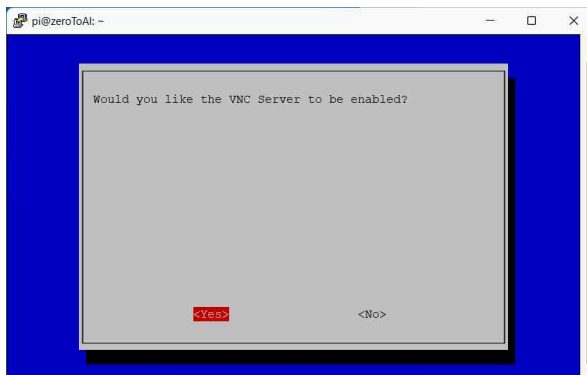


그림 161 VNC 활성화

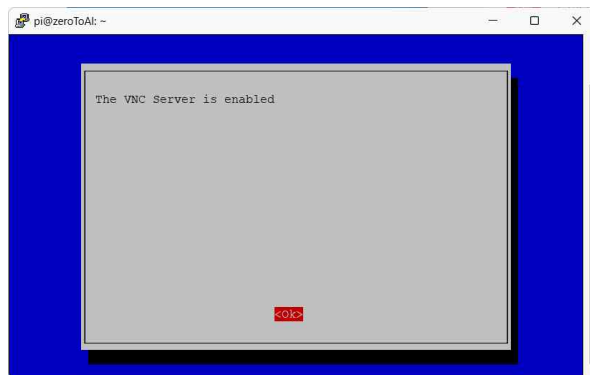


그림 162 VNC 활성화 성공

- ☐ 탭을 이용(두번 누름)해서 <Finish> 클릭하면 초기 화면으로 나옵니다.

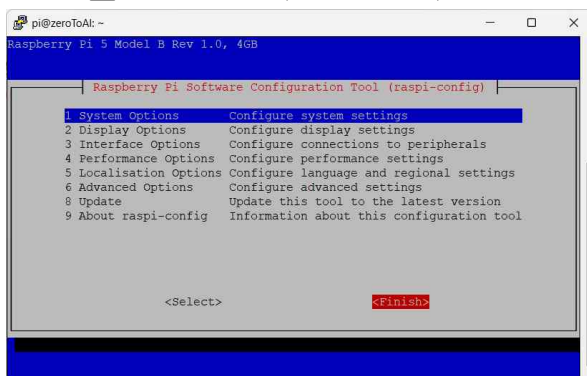


그림 163 환경설정 finish

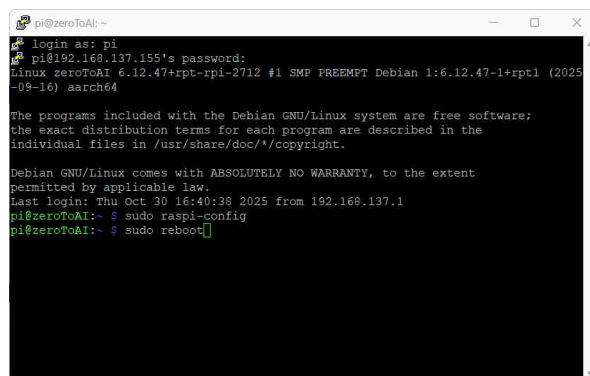


그림 164 재부팅 reboot

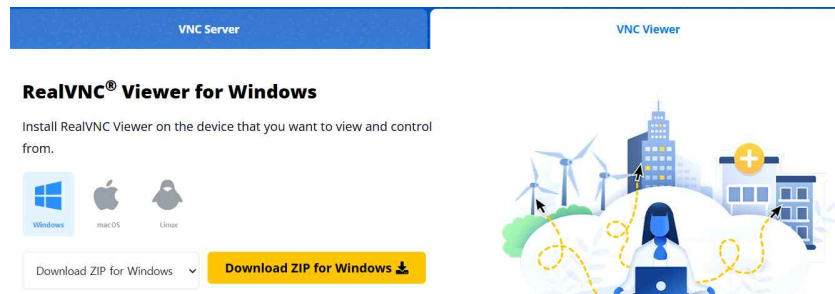
⑨ 재부팅

- ☐ 터미널 창에 `sudo reboot`를 입력하고 엔터, putty 창을 닫습니다.

⑩ vnc viewer 설치, 라즈베리파이 접속 및 확인

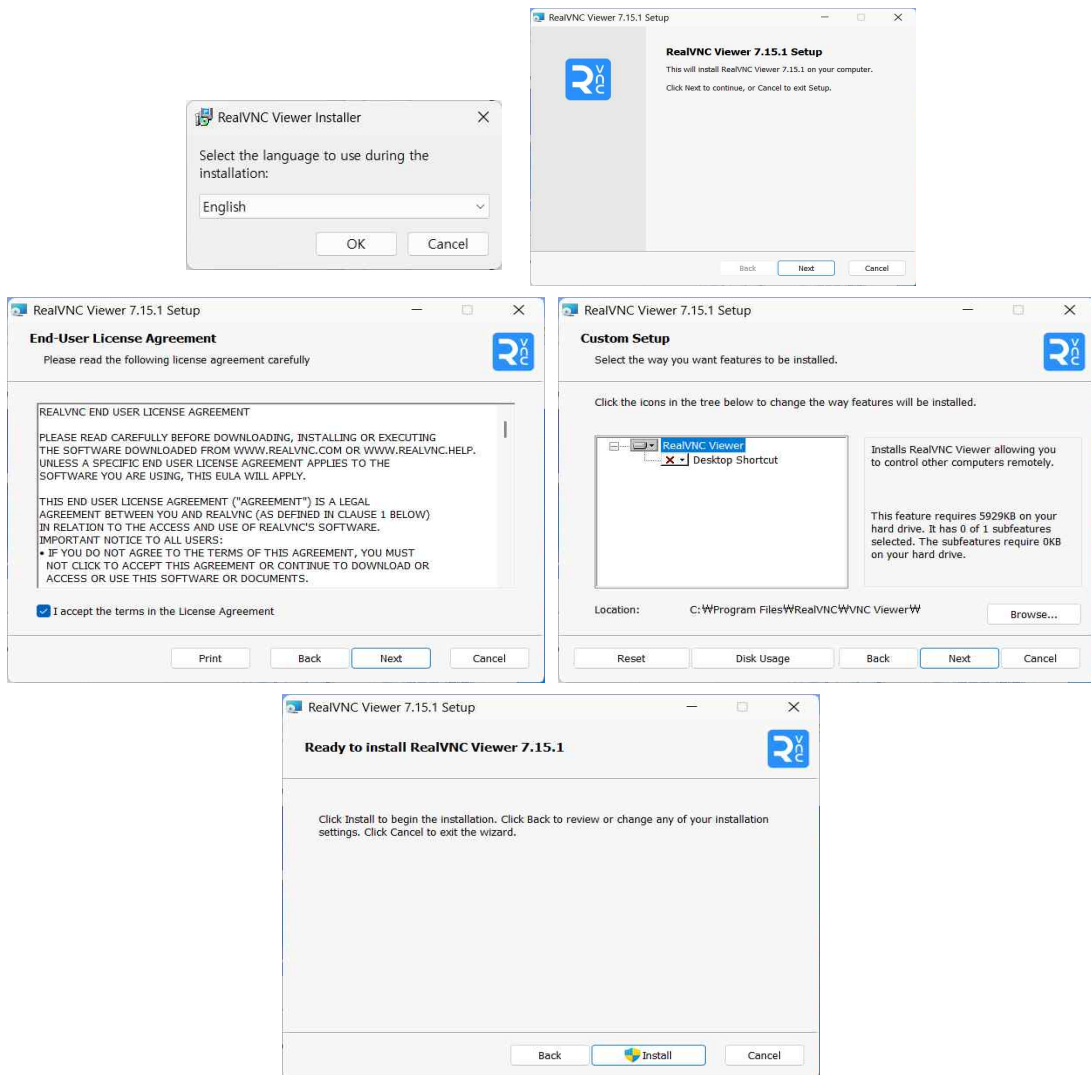
□ 컴퓨터에서 VNC Viewer 다운로드

- 사이트 : <https://www.realvnc.com/en/connect/download/viewer>
- 컴퓨터 운영체제에 맞게 다운로드 후 실행, 무료 회원가입 필요
- 파일명 : RealVNC-Connect-Viewer-8.4.2-Windows (2026.07 기준)



□ 설치

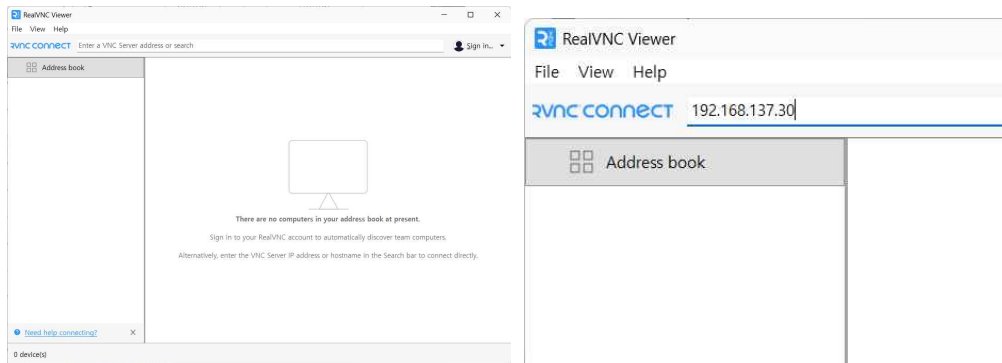
- 특별히 선택하실 것 없습니다. 초반에 라이선스 활용에 동의만 체크.



VNC Viewer 회원가입 관련 최신 동향 (참고)

RealVNC 정책이 변경되어 최근 버전에서는 무료(Lite) 계정 가입을 유도하는 창이 뜹니다. 이제부터는 무료 회원가입 필수입니다.

- 작업용 컴퓨터에서 VNC Viewer로 접속
 - 실행 : 시작메뉴 - VNC Viewer
 - 상단에 라즈베리파이 IP 주소 입력 후 엔터

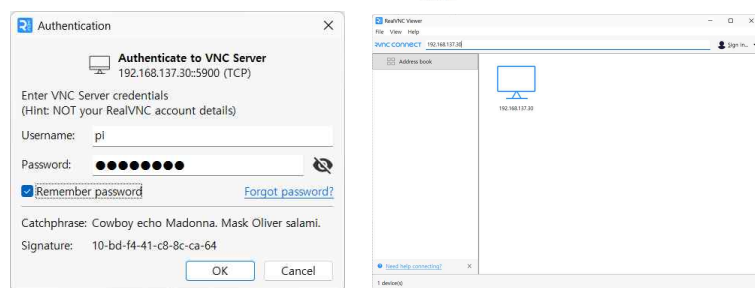


- 아이디와 비밀번호 입력, 비밀번호 기억 체크 (라즈베리파이 OS 만들 때 ID, 비밀번호)



- 예 : 아이디 : pi, 비밀번호 : 12345678
- ☒ Remember password, 다음 접속시부터 아이디와 비밀번호 묻지 않고 바로 접속

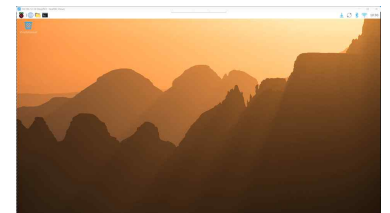
- 이후부터 접속은 vnc Viewer의 저장된 PC (아이콘) 을 더블클릭하시면 됩니다.



- 접속성공 창 의 왼쪽 상단에 ip 주소와 viewer 이 보임

- 라즈베리파이 접속

- 윈도우에서 창처럼 크기 조절, 최대, 최소, 닫기 등이 가능.
- 접속하시면 라즈베리파이 초기화면이 보임.



□ 다음 실습 시간에 접속이 안 될 때
 핫스팟 환경에서는 라즈베리파이를 켜다가 며칠 뒤에 다시 켜면 IP 주소의 맨 끝자리가 변경될 수 있습니다. VNC Viewer로 접속이 안 된다면 당황하지 말고, 윈도우의 '모바일 핫스팟' 설정 창을 다시 열어 오늘 라즈베리파이가 새로 할당받은 IP 주소가 무엇인지 확인한 후 그 주소로 다시 접속하면 됩니다.

3) VNC 원격 데스크톱 연결과 OS 화면구성 기초

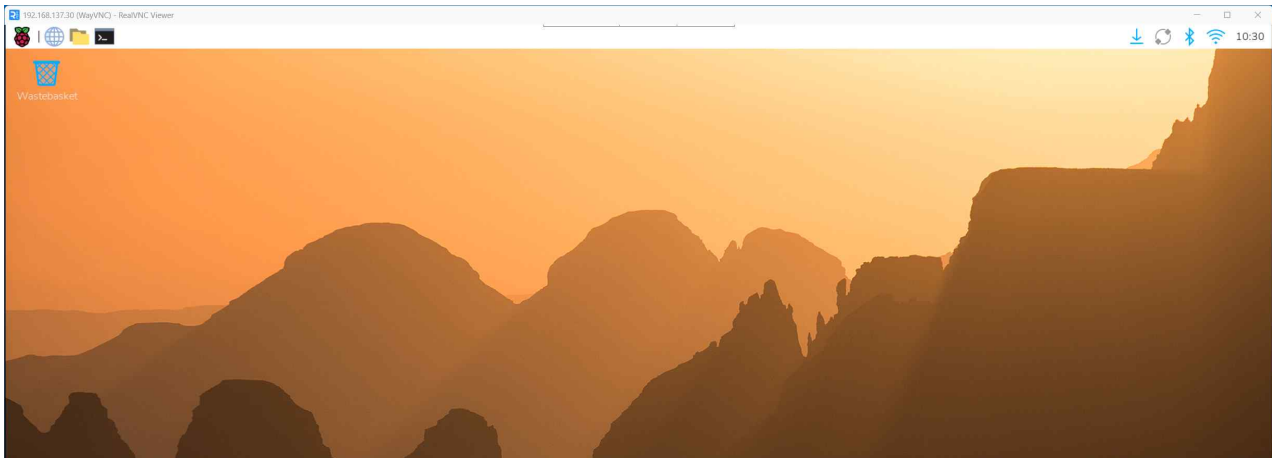


그림 178 라즈베리파이os 초기화면

① 시작메뉴 및 기본 메뉴

☐ 화면 상단 왼쪽

시작 메뉴	Web Browser	파일 탐색기	터미널
라즈베리파이 모든 기능 제어	Chromium 웹브라우저 인터넷 접속	파일 관리 (검색, 삭제, 이동 등)	명령어 입력 및 실행
Programming > Internet > Sound & Video > Graphics > Accessories > Help > Preferences > Run... Shutdown...			

② 상황 표시

☐ 화면 상단의 오른쪽.

				11:23
시스템 업데이트가 있다는 표시	라즈베리파이 커넥터 접속 해제	블루투스 기능 활성화 되어 있음	와이파이 접속 되어 있음.	현재 로컬 시간

③ 라즈베리파이 OS Shutdown Options

- ☐ Shutdown : OS 종료
- ☐ Reboot : OS 재 시작
- ☐ Lock Screen : 화면 잠금
- ☐ Logout : 로그아웃



④ 업데이트 및 업그레이드

□ 상황표시줄에서  Install Updates 클릭 혹은 터미널 창에 아래 명령어 입력

```
sudo apt update          # 업데이트 목록만 가져옴으로 금방 끝남.
sudo apt upgrade -y      # 실제 업데이트 함으로 시간이 오래 걸림.
```

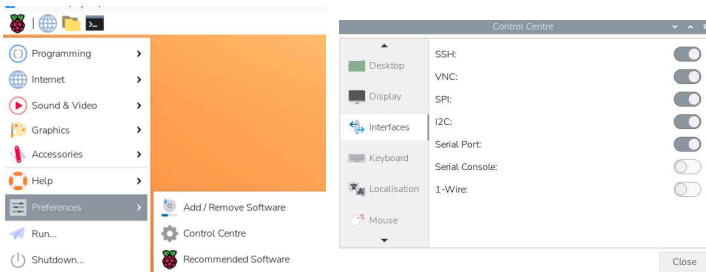
□ upgrade 는 가급적 와이파이 상태에서 진행하세요.

⑤ 라즈베리파이 시작메뉴

 Programming	>	 Programming	>	개발 도구 및 프로그래밍 환경	 Geany Programmer's Editor  Thonny
 Internet	>	 Internet	>	인터넷 프로그램 (브라우저, 네트워크 설정 등)	 Chromium Web Browser  Firefox
 Sound & Video	>	 Sound & Video	>	멀티미디어 도구 (미디어 플레이어 등)	 VLC Media Player
 Graphics	>	 Graphics	>	그래픽 편집 도구 (GIMP, 드로잉 프로그램 등)	 Image Viewer
 Accessories	>	 Accessories	>	유틸리티 프로그램 (계산기 파일 관리자, 텍스트 편집기 등)	 Archiver  Calculator  Document Viewer  File Manager  Keyboard layout viewer  Main Menu Editor  Raspberry Pi Diagnostics  Raspberry Pi Imager  SD Card Copier  Task Manager  Terminal  Text Editor
 Help	>	 Help	>	도움말 및 문서 (라즈베리파이 공식문서 등)	 Bookshelf  User Guide
 Preferences	>	 Preferences	>	시스템 설정 (디스플레이, 네트워크 등)	 Add / Remove Software  Control Centre  Recommended Software
 Run...		 Run...		프로그램 직접 실행 (명령어 입력 후 실행)	 Run... Enter the command you want to execute: Cancel OK
 Shutdown...		 Shutdown...		시스템 종료 옵션 (종료, 재부팅, 대기모드 등)	 Shutdown Options Shutdown Reboot Lock Screen Logout

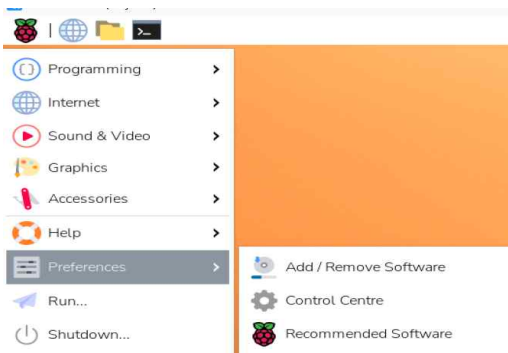
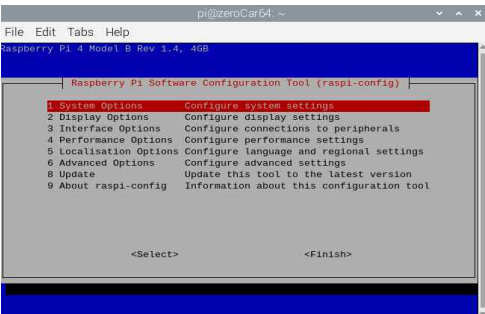
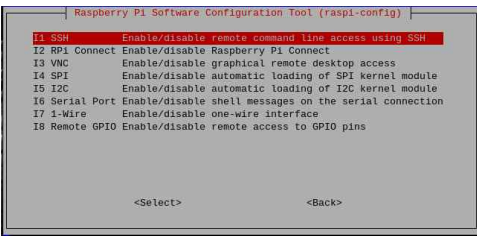
⑥ 라즈베리파이 OS 환경설정

☐ 화면 왼쪽 맨 위 시작메뉴  - Preference - Control Centre - **Interfaces**



- ☐ SSH : 외부 원격접속 활성화 (vs code에서 SSH 접속시 필수)
- ☐ VNC : VNC 원격접속 활성화 (VNC viewer 미러링 접속시 필수)
- ☐ SPI : 동기식 직렬 통신 프로토콜. (아날로그 센서 및 MCP3008 사용시 필수)
- ☐ I2C : SCL/SDA 통신 활성화 (I2C LCD 사용시 필수)
- ☐ Serial Port 활성화 : 라즈베리파이 GPIO 14(TX), 15(RX)을 통해 시리얼 통신을 가능
- ☐ Serial Console : 불활성. 라즈베리파이 터미널을 UART를 통해 외부에서 접근하게 함.
- 활성화 시키면 UART 포트를 로그인 셸이 점유해서 다른 시리얼 장치와 통신을 사용하지 못함.

☐ 환경 설정 모드

	GUI 모드	CUI 모드
특징	- 마우스 사용 - 윈도우와 비슷	- 키보드 사용 - 초기화면의 터미널을 이용 혹은 원격접속 등으로 접속시 사용하는 방식
Control Centre		터미널 창에 아래 명령어 입력 <pre>\$sudo raspi-config</pre> 
Interfaces		

☐ Desktop 탭

- 바탕화면 변경
- Desktop에서 Picture(이미지 선택), Layout(이미지 크기)

☐ Mouse 탭

- 마우스 감도 조절
- 마우스 Speed 및 더블 클릭 속도 조절
- Left-handed : 왼손용 마우스(좌우 클릭 위치 변경)

☐ Screens 탭

- Screens : 모니터 해상도 조절
- + / - : 모니터에 표시되는 글자 크기 조절
- Apply : 변경한 내용 적용

☐ System 탭

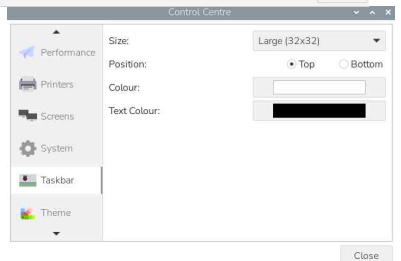
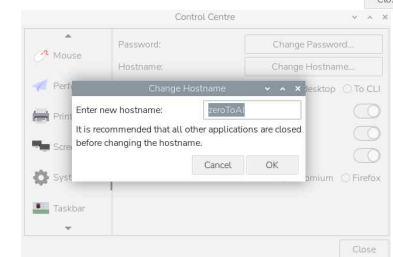
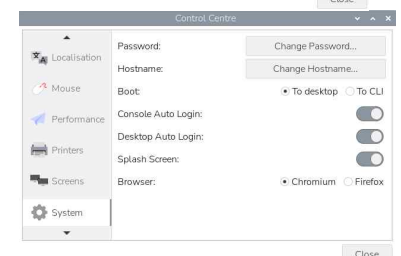
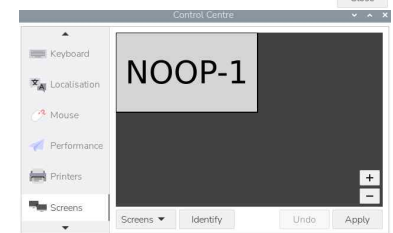
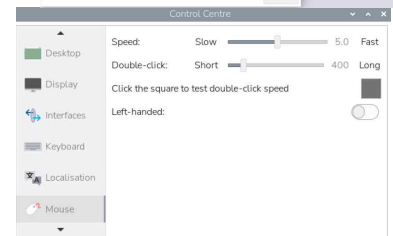
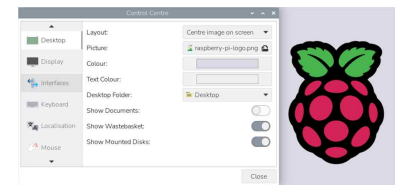
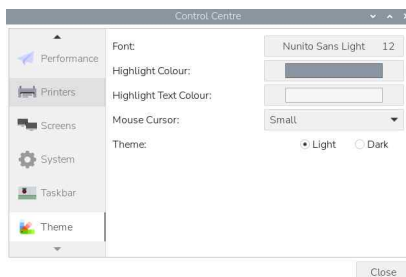
- 호스트 및 비밀번호 확인 및 변경(★)
- desktop Auto Login : 부팅 후 자동 로그인, 활성화
- 기본 브라우저 : Chromium, Firefox 둘 중 선택

☐ Taskbar 탭

- 작업표시줄 크기 및 위치 변경
- 마우스 커서 크기 변경

☐ Theme

- Font : 기본 폰트 변경
- Mouse Cursor : 마우스 커서 크기 조절
- Theme : 전체 테마 변경(Light / Dark)



4) 원격 작업 소스 준비

본 교재의 모든 소스 파일 다운로드를 진행합니다. 터미널에 작업합니다.

① 터미널 열기

- ☐ 화면 왼쪽 상단의 터미널  을 클릭,
- ☐ 아래 명령어 입력 후 엔터



```
git clone https://github.com/zeroToAIoT/smart_factory.git
```

② 작업 위치로 이동

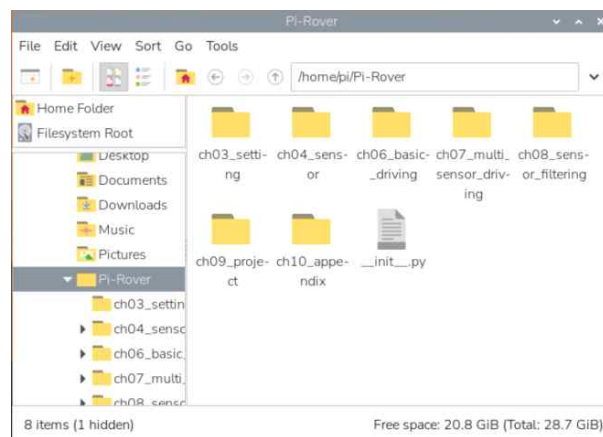
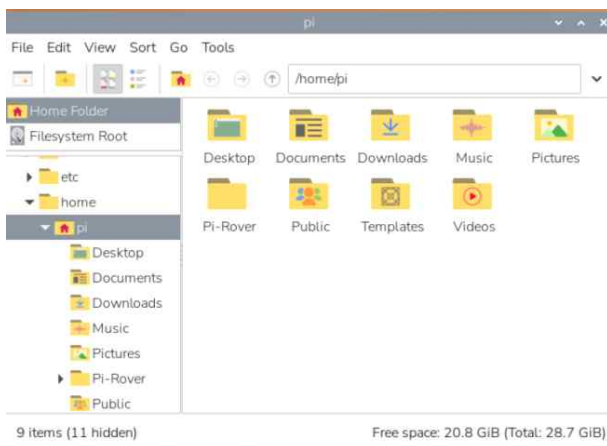
```
$ cd smart_factory/  
pi@zeroToAI: ~/smart_factory $
```

③ 확인 : ls 명령어

```
pi@zeroToAI: ~/smart_factory $ ls
```

- ☐ pi : 접속 사용자 아이디
- ☐ zeroToAI : 라즈베리파이 호스트명
- ☐ ~ : 사용자 홈 디렉터리, /home/pi를 의미
- ☐ smart_factory : 현재 위치한 smart_factory 디렉터리
- ☐ ls : 파일 목록 보여주는 명령어
 - 챔터 이름이 보입니다. 각 디렉터리에 소스코드가 들어 있습니다.

④ 라즈베리파이 파일 탐색기 활용



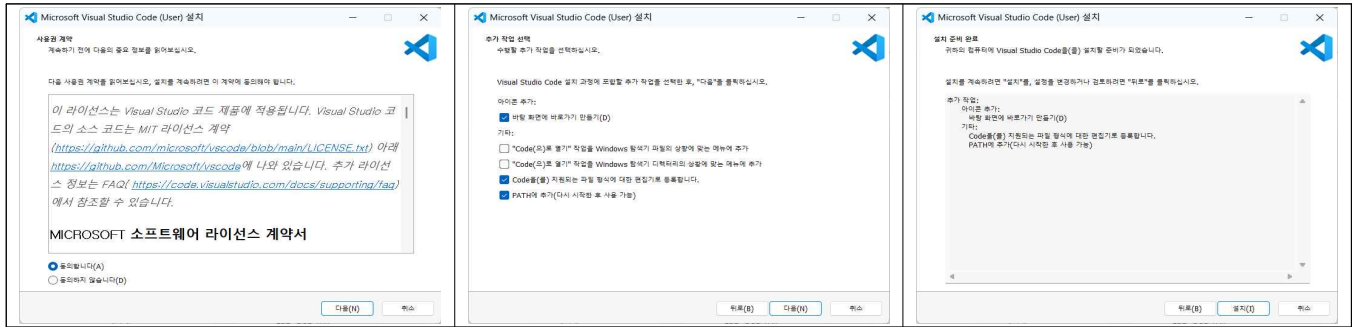
2. 본격적인 원격 개발의 시작 : VS Code기반 개발 환경 구축

1) VS Code 특징 및 설치

① 특징

- ☐ 일반 PC(노트북)에서 VS Code의 Remote SSH를 이용하여 라즈베리파이에 접속하여 코딩 및 실행
- ☐ 로컬 환경에서 작업하는 것처럼 라즈베리파이의 파일 시스템에 접근하고, 코드를 편집, 실행할 수 있음.

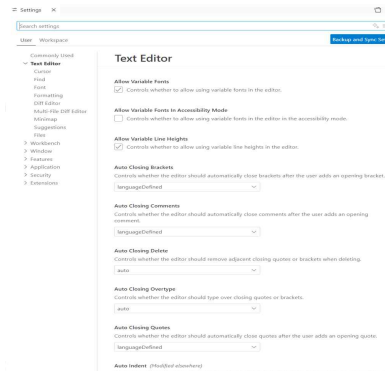
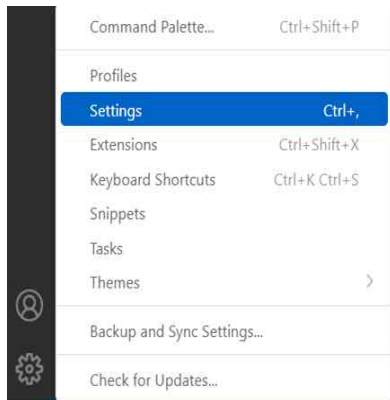
② vs code 설치, 다운로드 : <https://code.visualstudio.com/download>



③ vs code 기본 사용

☐ 설정

○ 화면 왼쪽 아래 톱니바퀴 - Setting



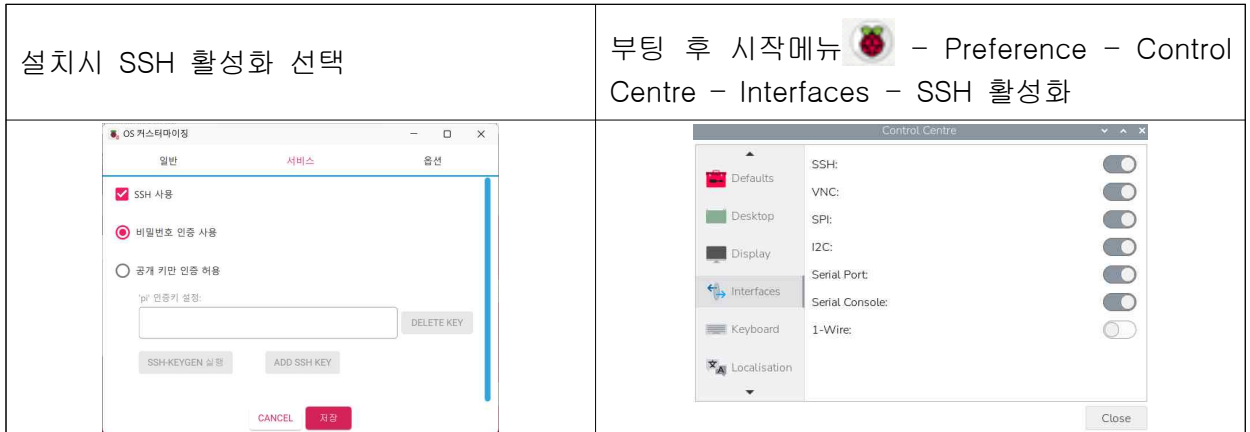
☐ 사이드 메뉴

메뉴	명칭	기능
	Explorer (Ctrl + Shift + E)	폴더 및 파일 탐색기
	search (Ctrl + Shift + F)	검색, 프로젝트 전체에서 작성한 파일이나 폴더 찾을때
	Source Control (Ctrl + Shift + G)	git과 깃허브 기능 제공
	Run and Debug (Ctrl + Shift + D)	파일 실행 및 디버그
	Remote Explorer	원격 접속 컴퓨터 목록 확인 (★) 작업용 컴퓨터에서 AGV로 SSH 접속 처음에는 안 보임. Remote-SSH 확장 프로그램 설치 후 보임.
	Extensions (Ctrl + Shift + X)	vs code에 추가 확장 프로그램 설치

2) 접속 환경 설정 및 SSH 연결

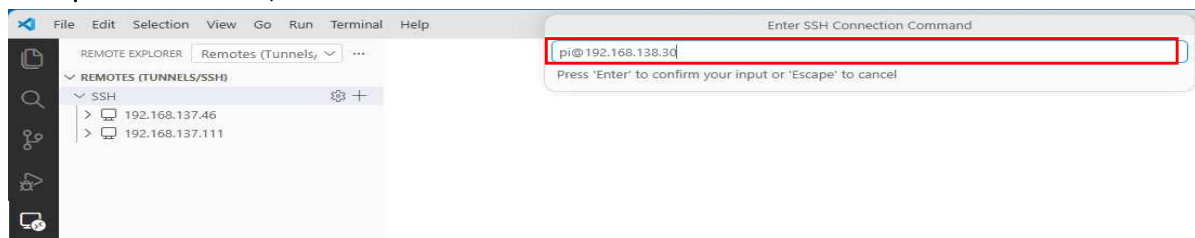
① 원격 접속 하기

- ☐ 라즈베리파이에서 먼저 SSH 접속이 활성화 되어 있어야 합니다.
 - 라즈베리파이 OS Imager 셋팅시 SSH를 이미 활성화 했음.
 - 혹은 이 부분을 놓쳤다면 아래 오른쪽 그림처럼 다시 하면 됨.

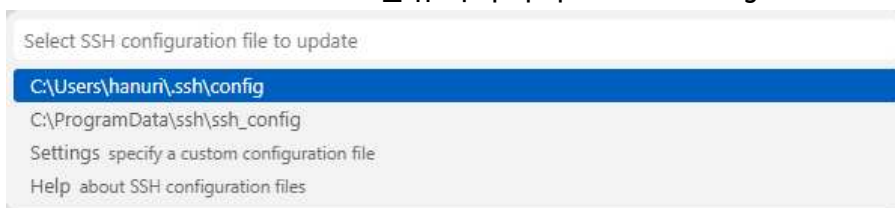


② vs code에서 SSH 접속

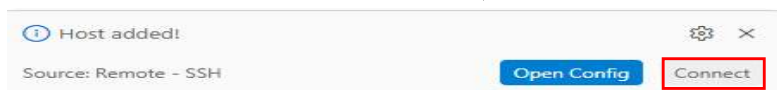
- ☐ 접속 IP 주소 예 : 192.168.137.30
- ☐  에서 + 클릭하거나 왼쪽 하단  클릭 하거나
- ☐ Enter SSH Connection Command에 [pi@192.168.137.216](#)
 - pi : 접속 계정, 라즈베리파이 OS 이미지 설정시 만든 계정



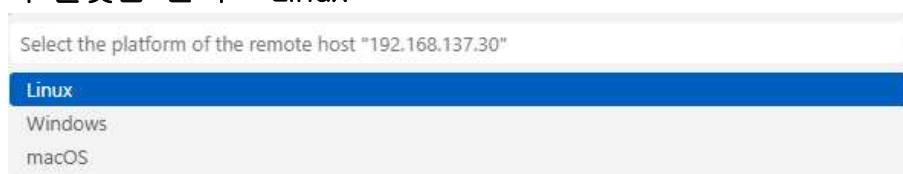
- ☐ 환경설정 파일 선택 : C:\Users\컴퓨터아이디\.ssh\config



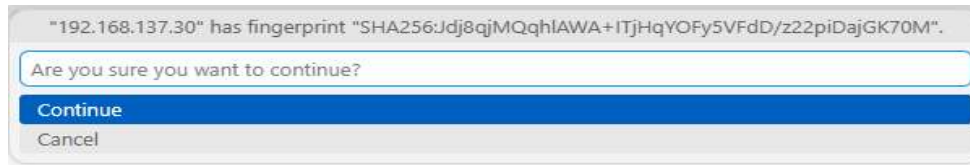
- ☐ vs code의 오른쪽 하단에서 Connect 선택, 접속



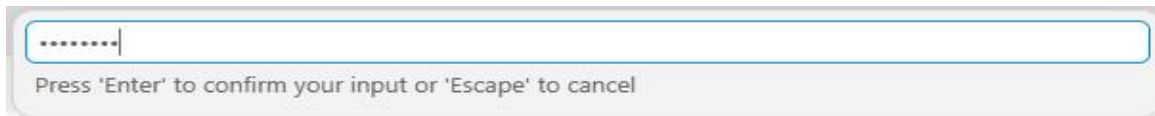
- ☐ 원격접속 플랫폼 선택 : Linux



- ☐ 원격접속용 비밀키 생성, Continue 선택, 처음 한번만 실행됨



- ☐ 라즈베리파이 pi 계정 접속용 비밀번호, (예: 12345678)

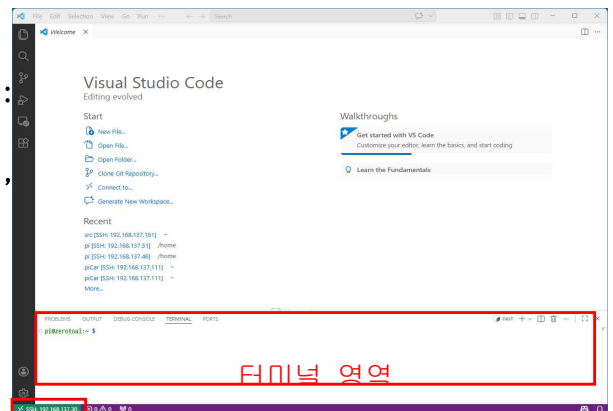


- ☐ vs code의 오른쪽 아래. 접속중



- ☐ 접속 완료

- 왼쪽 하단 녹색 바탕의 SSH : 192.168.137.30
- 터미널 : 파이썬 프로그램 실행, 명령어 입력, 실행 결과 확인 등



3) 원격 환경에 실행 버튼 달기

① 확장 프로그램 설치

- ☐ python 관련 개발 확장 프로그램 설치

- VS Code 좌측의 확장(Extensions) 탭 (단축키: Ctrl+Shift+X).
- 검색창에 python 입력,,, python by microsoft 선택



그림 251 확장프로그램_pythonbymicrosoft

- ☐ Remote - SSH 확장 프로그램 설치(SSH 접속시 자동 설치됨)

- VS Code 좌측의 확장(Extensions) 탭 (단축키: Ctrl+Shift+X).
- 검색창에 Remote - SSH를 입력
- 검색된 Remote - SSH 확장 프로그램을 찾아 Install 버튼을 눌러 설치

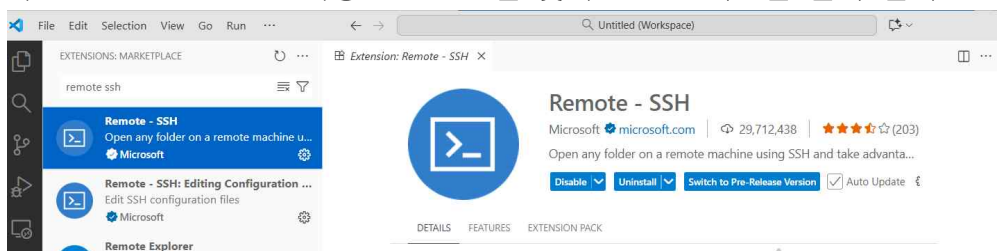


그림 252 vscode_exp_ssh

- Remote Development라는 확장 프로그램 팩을 설치해도 Remote-SSH가 포함되어 있어 동일하게 사용 가능

□ 추가 설치

- Python by Microsoft (필수)
 - 기능 : 코드 실행, 디버깅, 자동완성, 실행 버튼 활성화 등
 - 설치 버튼 : Install in SSH:192.168.x.x 클릭
- Pylance by Microsoft (필수) : 빠르고 정확한 코드 분석, 타입 체크, 인텔리센스 향상
- Python Environments (선택) : 가상환경 관리에 유용

② 소스 파일 다운 및 열기

- 왼쪽 사이드 메뉴에서 [Explorer]  클릭 > Open Folder > AGV 선택 후 OK 클릭

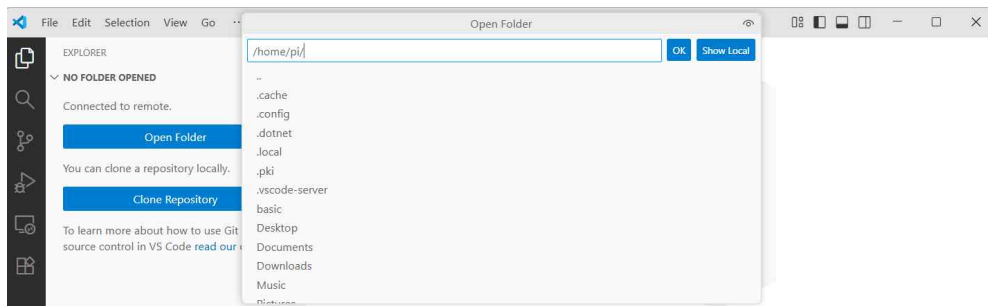


그림 254 open_folder_basic

- 상단에 비밀번호 입력창이 나옵니다. 비밀번호(예 : 12345678) 입력 후 엔터
- 처음으로 디렉터리 열 때 신뢰성에 대해 물어봅니다.
꼭 ☒ Trust the authors of all files in the parent folder 'pi' 체크하고 Yes, I Trust the authors 버튼 클릭. 다음부터 비밀번호를 물어보지 않습니다.

- 왼쪽화면에 챗터별로 소스가 있습니다. 클릭하시면 오른쪽 메인창에 보입니다.

- welcome 파일은 닫으세요.
- smart_factory 내용으로 캡처.. 꼭..

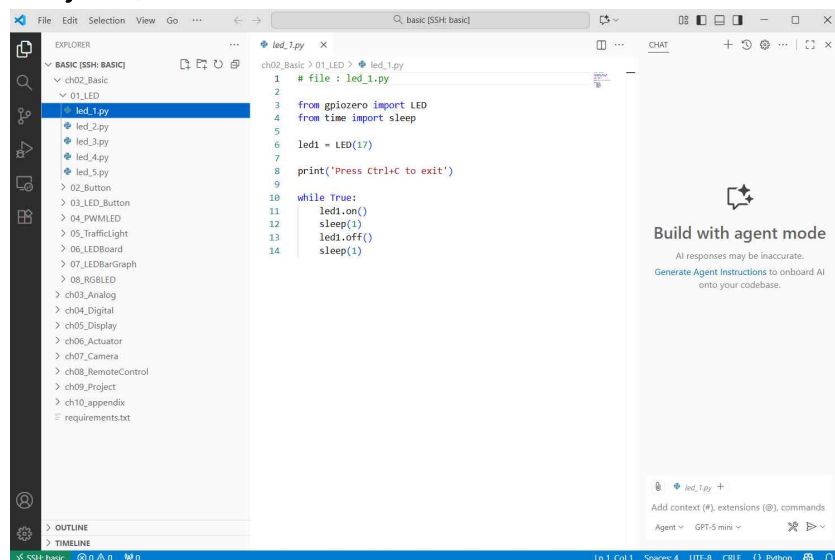


그림 255 소스 파일

4) 고임 방지를 위한 독립 구역 : 가상 환경(venv) 구축 및 패키지 설치

라즈베리파이 OS 12버전 이후부터는 시스템 안정성을 위해, pip 설치를 OS 전역(Global)에 함부로 설치하지 못함(PEP 668 정책). 따라서 우리는 AGV 프로젝트만을 위한 독립적인 공간, 즉 가상환경(venv)을 만들고 그곳에 필요한 라이브러리 설치하고 실행, vs code에서 진행.

① vs code로 SSH 접속

> SSH: 192.168.137.216

☐ vs code 맨 아래 왼쪽에서 접속 상태 확인, IP주소는 다를수 있음.

② 사용자 폴더 이동, smart_factory 프로젝트를 진행할 폴더로 이동

```
$ cd ~ # ~ 사용자 폴더로 이동
$ cd smart_factory # smart_factory 폴더 이동
```

③ 가상환경 생성 : smart_factory_venv (교재에서 딱 1번만 생성.)

```
$ python3 -m venv smart_factory_venv --system-site-packages
```

☐ --system-site-packages 옵션은 매우 중요 이 마법의 옵션을 붙여주면, 우리가 앞서 설치했던 패키지(i2c-tools 등)를 가상환경 안에서도 그대로 사용

④ 가상 환경 활성화(사용할때마다 실행 ★)

```
$ source ~/smart_factory/smart_factory_venv/bin/activate
```

- 가상 환경이 활성화되면 프롬프트 앞에 (smart_factory_venv)와 같이 가상 환경 이름이 표시
- 이 작업은 라즈베리파이를 켜다 켜 때마다, 혹은 새 터미널을 열 때마다 코드 실행 전에 반드시 수행해야 함

☐ 터미널 프롬프트 해석하기

```
(smart_factory_venv) pi@zeroToAI: ~/smart_factory $
```

- (smart_factory_venv) pi@zeroToAI: ~/smart_factory \$: 가상환경 이름 (보여야 정상)
- (smart_factory_venv) pi@zeroToAI: ~/smart_factory \$: 현재 사용자 계정 이름
- (smart_factory_venv) pi@zeroToAI: ~/smart_factory \$: 라즈베리파이 기기 이름(호스트명)
- (smart_factory_venv) pi@zeroToAI: ~/smart_factory \$: 사용자 홈 폴더 (/home/pi)
- (smart_factory_venv) pi@zeroToAI: ~/smart_factory \$: 현재 위치한 폴더
- 이후 표기는 아래와 같이 프롬프트 앞에 가상환경 이름만 표기.

```
(smart_factory_venv) $
```

⑤ 가상환경 비활성화 : deactivate

```
(smart_factory_venv) $ deactivate
```

⑥ 생성한 가상환경 폴더 완전 삭제

이름이 잘못되었거나 필요 없는 가상환경 폴더 및 파일 완전 삭제하는 방법.

☐ 가상환경 폴더 확인

- 소스 코드 폴더로 이동

```
$ cd ~/smart_factory
```

- 가상환경 폴더 확인

```
$ ls
```

smart_factory_venv가 보아야 함

☐ 가상환경 폴더(smart_factory_venv) 삭제

```
$ rm -rf smart_factory_venv
```

- r : 폴더 안에 있는 폴더 및 파일 모두 삭제
- f : 강제 삭제, 즉 확인 절차 없이 바로 삭제

⑦ 패키지 도구 pip 업데이트 (설치 오류 방지)

```
(smart_factory_venv) $ pip install --upgrade pip
```

☐ 하드웨어 제어 및 센서 통신 라이브러리 설치

```
(smart_factory_venv) $ pip install gpiozero pyserial pynmea2 mpu6050-raspberrypi
```

☐ 컴퓨터 비전(OpenCV) 라이브러리 설치

```
(smart_factory_venv) $ pip install opencv-contrib-python
```

☐ 임시 작업 폴더 생성 (설치시 용량 부족 현상 방지, ★ 꼭 하셔야합니다.)

- SD 카드의 홈 폴더에 넉넉한 임시 폴더 만듦
- 라즈베리파이의 기본 임시 폴더인 /tmp는 2G 용량뿐임.

☐ AI 추론 및 엣지 라이브러리들은 설치시 매우 큰 임시공간이 필요함

```
(smart_factory_venv) $ mkdir ~/pip_tmp
```

☐ 환경변수(TMPDIR)에 위에서 만든 임시 폴더로 바꿔서 설치

; 명령어 맨 앞에 TMPDIR=~/.pip_tmp를 붙여주면, 해당 폴더를 임시 작업장으로 사용하여 2GB 제한 없이 쾌적하게 설치됩니다.

; 캐시를 남기지 않는 --no-cache-dir 옵션도 추가

```
(smart_factory_venv) $ TMPDIR=~/.pip_tmp pip install --no-cache-dir 설치패키지이름
```

☐ AI 추론 및 엣지 최적화 라이브러리 설치 (시간 정말 많이 소요됨 40분~1시간)

```
(smart_factory_venv) $ TMPDIR=~/.pip_tmp pip install --no-cache-dir torch torchvision ultralytics onnx onnxruntime
```

- torch, torchvision : 현존하는 가장 강력한 인공지능 프레임워크인 PyTorch

로봇의 딥러닝 연산을 담당.

- ultralytics : 표지판이나 신호등, 사람을 눈 깜짝할 새에 찾아낼 YOLO(You Only Look Once) 알고리즘의 최신 라이브러리입니다.
- opencv-python: 방금 2장에서 우리가 카메라를 제어할 때 썼던, 영상 처리의 세계 표준

☐ 설치 끝난 후 임시 폴더(pip_tmp) 완전 삭제하기

```
(smart_factory_venv) $ rm -rf ~/pip_tmp
```

☐ 라이브러리 설치 및 버전 확인

```
(smart_factory_venv) $ pip show torch
(smart_factory_venv) $ pip show ultralytics
(smart_factory_venv) $ pip show onnx
```

- show 명령어는 설치가 잘 되어 있는지 여부만 알수 있음. 실제 동작 여부는 모름.

☐ 라이브러리 동작 확인

```
(smart_factory_venv) $ python3
>>> import torch
>>> import ultralytics
>>> import cv2
>>> exit()
```

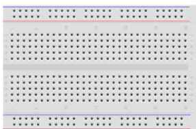
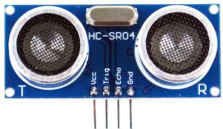
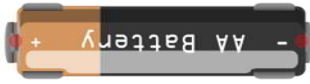
- 결과 확인: 명령어를 쳤을 때 붉은색 에러 메시지 없이 다음 줄(>>>)이 부드럽게 나오면 완벽하게 성공한 것입니다! 확인이 끝났으면 exit()를 입력하여 파이썬 모드를 빠져나옵니다.

Chapter 4. 물류 자동화(AGV) 기초 주행

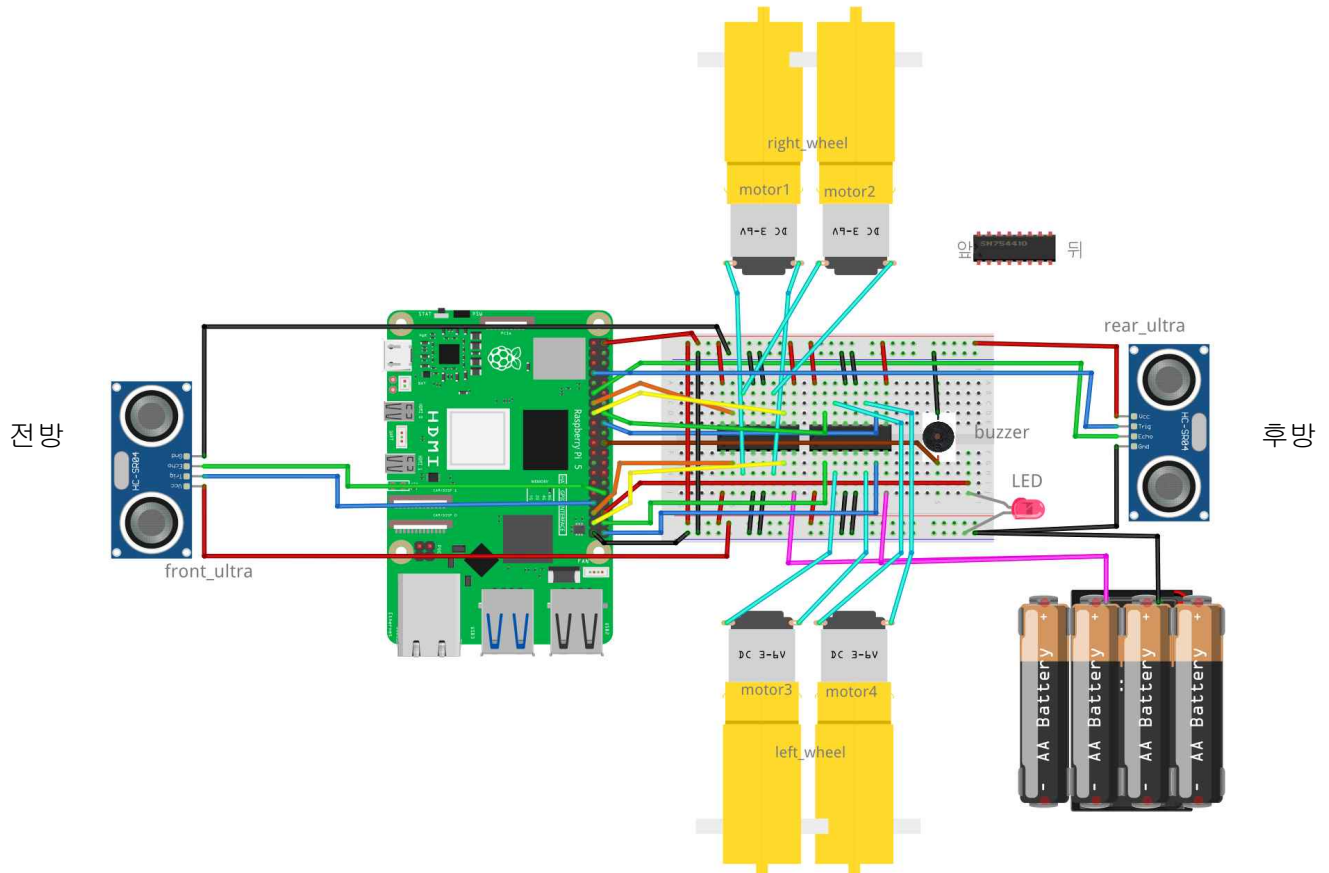
1. AGV 뼈대 및 모터 조립

이번 장에서는 무인 운반차를 뜻하는 AGV(Automated Guided Vehicle)를 제어해 보겠습니다.

1) AGV 뼈대 조립을 위한 준비물

재료명	이미지	수량	재료명	이미지	수량
라즈베리파이		1	브레드보드 (half, 400홀)		1
LED (5mm)		3	피에조 수동 부저		1
초음파센서 (HC-SR04P)		2	자동차 프레임 키트		1
적외선 동작 감지 HC-SR505		1			
파이 카메라		1	카메라 홀더 (아크릴)		
모터드라이버 SN754410		2	DC모터		4
건전지 (1.5V)		4	건전지 홀더 (4구 직렬)		1

2) 한 장으로 끝내는 회로도 : AGV 전체 하드웨어 회로도



3) 두뇌와 신경망 연결 : 핀맵(Pin Map)과 GPIO 할당표

① 라즈베리파이 연결 정보

	3V3	1		2	5V	브레드보드 VCC
	SDA	3		4	5V	
	SCL	5		6	GND	
후방 초음파 trig	GPIO 4	7		8	TX	
	GND	9		10	RX	
후방 초음파 echo	GPIO 17	11		12	GPIO 18	
SN754410①15	GPIO 27	13		14	GND	
SN754410①10	GPIO 22	15		16	GPIO 23	SN754410②15
	3V3	17		18	GPIO 24	SN754410②10
	MOSI	19		20	GND	
	MISO	21		22	GPIO 25	피에조 부저
	SCLK	23		24	CE0	
	GND	25		26	CE1	
	EED	27		28	EEC	
	GPIO 5	29		30	GND	
	GPIO 6	31		32	GPIO 12	전방 초음파 Echo
전방 초음파 trig	GPIO 13	33		34	GND	GND
SN754410 ① 2	GPIO 19	35		36	GPIO 16	led
SN754410 ① 7	GPIO 26	37		38	GPIO 20	SN754410② 2
라즈베리파이 GND	GND	39		40	GPIO 21	SN754410② 7

; vcc 와 GND 는 회로도 참고

; vcc 와 GND 는 회로도 참고

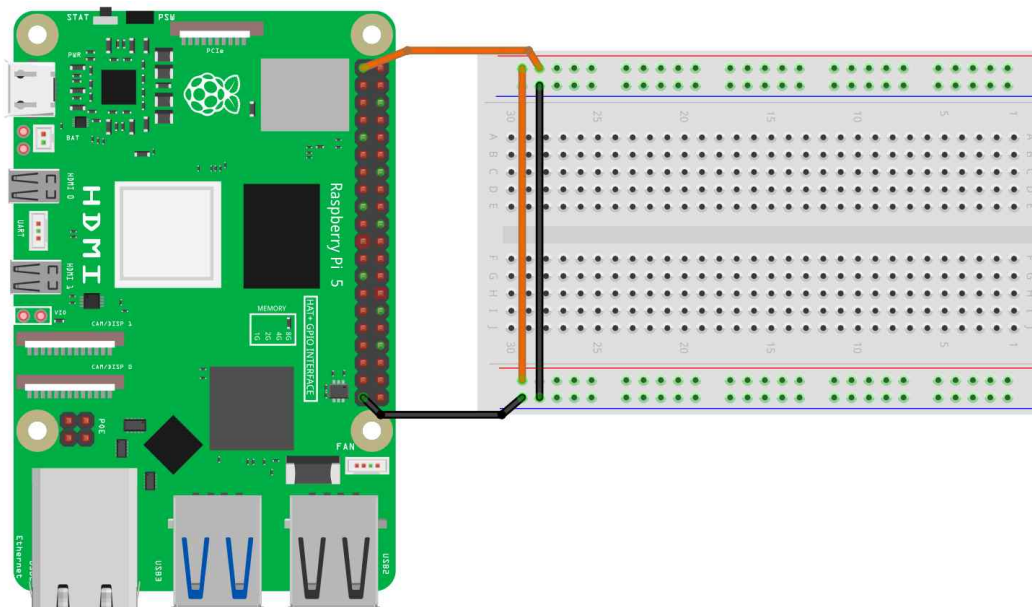
② 부품별 GPIO 연결 정보

☐ 표의 배경색은 회로도의 색과 동일합니다.

1. LED		GPIO 16
2. 부저		GPIO 25
3. 전방 초음파	echo	GPIO 12
	trig	GPIO 13
4. 후방 초음파	echo	GPIO 17
	trig	GPIO 4
5. 1번 SN754410 모터 드라이버	motor 1	GPIO 26
		GPIO 19
	motor 2	GPIO 27
		GPIO 22
6. 2번 SN754410 모터 드라이버	motor 3	GPIO 20
		GPIO 21
	motor 4	GPIO 24
		GPIO 23

③ 라즈베리파이와 브레드보드 기초 연결

- ☐ 라즈베리파이의 3.3V (물리적핀번호 1번) 브레드보드 빨강 라인에 연결
- ☐ 라즈베리파이의 GND (물리적 핀번호 39번), 브레드보드 파랑 라인에 연결



4) AGV 뼈대, 프레임 제작하기

- ① dc motor : 점퍼 연결선 납땜
 - ② 하체 프레임 : 보호필름 제거, dc motor 고정, 라인트레이서 고정, mpu6050 부착
 - ③ 상체 프레임 : 후방 초음파 부착, 4개의 지지대로 하체 프레임 연결
 - ④ 브레드보드 : 라즈베리파이 기초 배선 연결, TB6612FNG 모터 드라이버 연결
 - ⑤ 상체 프레임 : 라즈베리파이, 브레드보드, 건전지 홀더 부착
라인트레이서, 후방 초음파 센서, LED 라즈베리파이 GPIO에 연결
-
1. 서보모터 : 서보모터 브라켓 조립 및 서보모터 브라켓 결합
서보모터 브라켓과 전방 초음파 결합
서보모터의 중앙에 위치하도록 test_servo.py 실행 후 부착
서보모터와 전방 초음파를 라즈베리파이 GPIO에 연결

□ Step 1. DC motor 준비(납땜 및 절연)

; 가장 먼저 모터에 전선을 연결. 납땜이 튼튼하게 되어야 동작 에러가 없습니다.

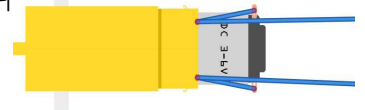
1. 납땜 : 모터 단자에 점퍼선 납땜.

[주위] 선이 모터 몸통 안쪽으로 향하게 함. 바깥으로 튀어나오면 바퀴에 걸립니다.



2. 1차 절연 : 납땜 부분 서로 닿지 않도록(합선 방지) 절연테이프로 꼼꼼하게 감습니다.

3. 2차 고정 : 작업할 때 선이 당겨져서 끊어지지 않도록, 선을 모터 위로 반대편으로 꺾어서 테이프로 한 번 더 고정해 줍니다.



□ Step 2. 하부 프레임 조립

1. 보호 필름 제거 : 아크릴 프레임의 양면 보호 필름을 벗겨냅니다.

2. 모터 장착 :

- 모터의 전선이 나오는 쪽이 서로 마주 보게(프레임 안쪽) 배치
- 전선은 프레임 가운데 있는 직사각형 구멍을 통해 위쪽(상판 쪽)으로 미리 빼놓음
- 긴 볼트와 너트를 이용해 모터를 단단히 고정

3. 작업 팁 : 종이컵을 뒤집어 놓고 그 위에 프레임을 올리면 모터가 바닥에 눌리지 않아 작업이 편합니다.

□ Step 3. 상부 프레임 및 제어부 구성

; 두뇌 역할을 하는 라즈베리파이와 브레드보드, 배터리 구성

1. 프레임 결합 :

- 지지대(서포터) 4개를 이용해 하판과 상판을 결합
- [체크] 모터 선(8개)과 센서 선들이 상판 구멍을 통해 위로 잘 올라왔는지 확인

2. 라즈베리파이 및 브레드보드 부착 :

- 상부 프레임의 앞쪽(라인 센서가 부착된 곳)에 2cm(서보모터 브라켓 들어갈 자리) 띄우고 라즈베리파이, 브레드보드 순으로 배치

3. 모터 드라이버(SN754410) 장착 :

- 칩 2개를 브레드보드 가운데에 1칸 정도 띄고 연달아 꼽음.

[매우 중요] 칩의 반달 모양 홈(Notch)이 앞쪽 방향(라즈베리파이쪽)을 향하도록 통일해서 꽂아야 합니다. 방향이 바뀌면 칩이 탈 수 있습니다.

	1번 SN754410 ●	
앞		뒤

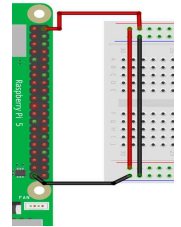
	2번 SN754410	
앞		뒤

□ Step 4. 라즈베리파이와 브레드보드 연결

; 두뇌 역할을 하는 라즈베리파이와 브레드보드, 배터리 구성

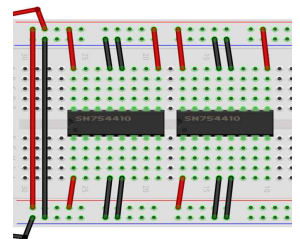
1. 전원 및 드라이버 기본 배선 :

- 라즈베리파이 VCC(물리 핀번호 2)를 브레드보드 빨간 라인
- 라즈베리파이의 GND(물리 핀번호 39)를 브레드보드의 파란 라인과 연결
- 브레드보드의 위, 아래 VCC, GND를 연결해준다.



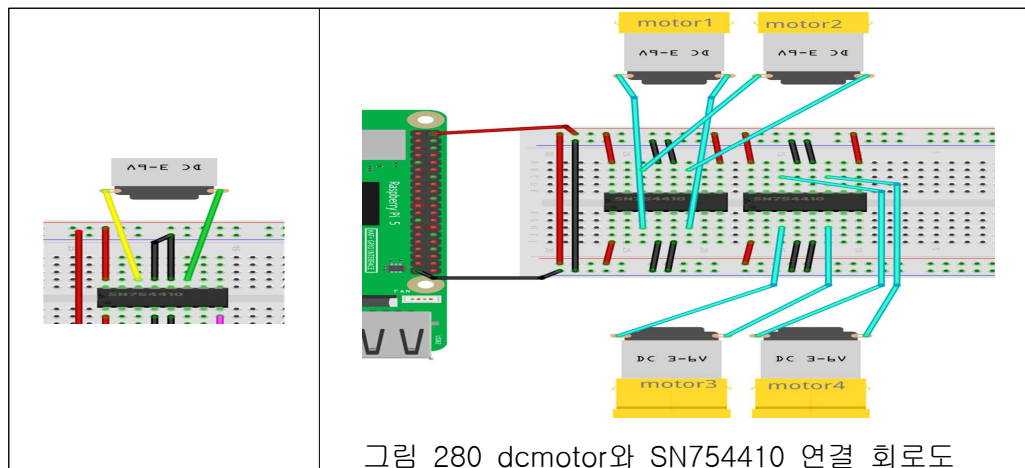
2. 칩 전원 연결 :

- 칩 전원 : SN754410 칩의 16번(VCC), 8번(VCC2), 1번(Enable) 핀 등을 브레드보드 빨강 라인(+)에 연결.
- 칩 접지 : SN754410 칩의 가운데 방열핀(4,5,12,13번)을 브레드보드 파랑 라인(-)에 연결.



3. 모터와 SN754410 연결 :

- 모터 드라이버 1개당 2개의 모터를 연결
 - 왼쪽 모터 선 2 개 : SN754410 칩의 3번(1Y), 6번핀(2Y) 핀 위치에 꽂습니다.
 - 오른쪽 모터 선 2개 : SN754410 칩의 11번(3Y), 14번핀(4Y) 핀 위치에 꽂습니다.
- 모터 선 2개끼리는 순서가 바뀌어도 상관없습니다. (나중에 코드로 방향을 수정할 수 있습니다.)



4. 라즈베리파이 GPIO와 브레드보드 연결

- 점퍼 케이블 : 암수(총 8개), 주황-노랑-녹색-파랑(4개)순으로 연결된 2개
- 라즈베리파이 GPIO 핀과 연결은 SN754410 칩의 VCC 연결핀의 옆에 꼽음.
- 연결선은 앞에서부터 주황, 노랑, 녹색, 파랑 순
- 연결 정보

sensor	라즈베리파이	sensor	라즈베리파이
DC motor 1	GPIO 26 (주황색선) GPIO 19 (노란색선)	DC motor 2	GPIO 22 (주황색선) GPIO 27 (노란색선)
sensor	라즈베리파이	sensor	라즈베리파이
DC motor 3	GPIO 20 (녹색선) GPIO 21 (파랑선)	DC motor 4	GPIO 24 (녹색선) GPIO 23 (파랑선)

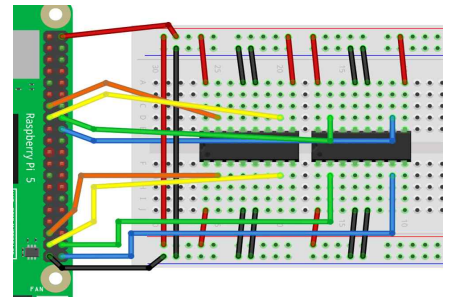


그림 281 GPIO SN754410 회로도

5. 건전지 홀더 전원선 작업 및 연결

- 건전지 홀더 + : 3개선으로 분기 (2개 SN754410 8핀, 서보모터 VCC)
- 건전지 홀더 - : 브레드보드 파란 라인 어느 곳이나 연결

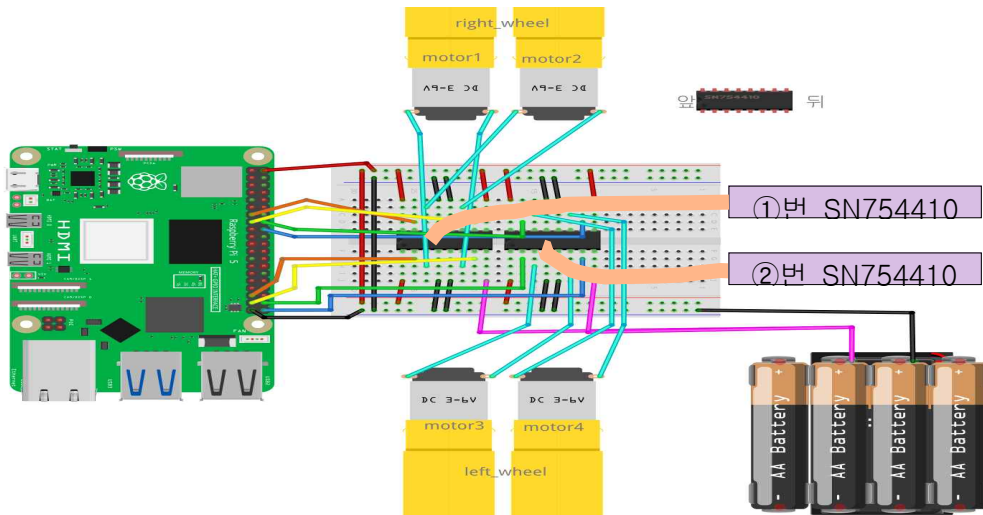


그림 282 모터 연결 최종 회로도

1. 선 정리.

□ 칩이 타는 냄새가 날 때
실습 중 "타는 냄새가 나요!"라고 한다면 십중팔구 SN754410 칩을 거꾸로 꽂은 것입니다. 브레드보드에 모터 드라이버를 꽂을 때는 칩 표면에 파인 '반달 모양 홈(Notch)'이나 동그란 점이 반드시 1번 핀 쪽을 향하도록, 모든 칩의 방향을 한쪽으로 통일해서 꽂아야 합니다.

5) 모터 드라이버와 모터 연결정보

① right_wheel (motor1, motor2)

- ☐ motor1 : front_right_wheel
motor2 : rear_right_wheel

sensor	라즈베리파이	sensor	라즈베리파이
DC motor 1	GPIO 26, GPIO 19	DC motor 2	GPIO 27, GPIO 22

GPIO	기능		모터2 입력	모터2 출력			모터2 출력	모터2 입력	
	GPIO		27	모터연결			모터연결	22	
SN754410 ①	기능	Vcc1	4A	4Y	GND	GND	3Y	3A	3,4EN
	핀번호	16	15	14	13	12	11	10	9
	칩 앞쪽								
	핀번호	1	2	3	4	5	6	7	8
	기능	1,2EN	1A	1Y	GND	GND	2Y	2A	Vcc2
	GPIO		26	모터연결			모터연결	19	
GPIO	기능		모터1 입력	모터1 출력			모터1 출력	모터1 입력	외부전원

- ☐ motor 3 : front_left_wheel
motor 4 : rear_left_wheel

sensor	라즈베리파이	sensor	라즈베리파이
DC motor 3	GPIO 20, GPIO 21	DC motor 4	GPIO 24, GPIO 23

GPIO	기능		모터4 입력	모터4 출력			모터4 출력	모터4 입력	
	GPIO		23	모터연결			모터연결	24	
SN754410	기능	Vcc1	4A	4Y	GND	GND	3Y	3A	3,4EN
	핀번호	16	15	14	13	12	11	10	9
	칩 앞쪽								
	핀번호	1	2	3	4	5	6	7	8
	기능	1,2EN	1A	1Y	GND	GND	2Y	2A	Vcc2
	GPIO		20	모터연결			모터연결	21	
GPIO	기능		모터3 입력	모터3 출력			모터3 출력	모터3 입력	외부전원

② [실습] 바퀴 회전 테스트 및 최소 구동 속도 파악

배선이 완료되면 코드를 통해 모터가 정상적으로 회전하는지 테스트함.

☐ 테스트 꿀팁 : 종이컵을 뒤집어 AGV의 아래 중앙에 받쳐두면 바퀴가 허공에 뜨게 되어 회전 상태를 파악하기 매우 쉽습니다.

☐ 전진 테스트 및 방향 수정,

4개 모터가 모두 앞으로 전진, **모터 방향이 반대로 회전하면 핀번호를 서로 바꿔줌.**

○ 예를 들어 motor3의 바퀴가 거꾸로 돌고 있다면

motor1 = Motor(19, 26) motor2 = Motor(22, 27) motor3 = Motor(20, 21) motor4 = Motor(24, 23)	motor3 = Motor(21, 20)
---	---------------------------------

☐ 모터 정지 :

○ try - finally 구문에서 프로그램 정지(ctrl +c)되면 마지막(finally)에 모터 정지

```
# file name : test_motor.py
# Test All Motor Forward

from gpiozero import Motor
from time import sleep

motor1 = Motor(26, 19)
motor2 = Motor(22, 27)
motor3 = Motor(20, 21)
motor4 = Motor(24, 23)

print('Press Ctrl+C to stopped')
print('-'*30)

speed = 0.5

try:
    while True:
        print('Motor Forwarding...', end='\r')
        motor1.forward(speed)
        motor2.forward(speed)
        motor3.forward(speed)
        motor4.forward(speed)

except KeyboardInterrupt:
    print('Stopped. ctrl+c pressed.')
except Exception as err:
    print(f'Error : {err}')
finally:
    print('Motor stopped')
    motor1.stop()
    motor2.stop()
    motor3.stop()
    motor4.stop()
```

□ 모터 구동 최소 속도 파악

- 속도가 0.1부터 서서히 증가합니다. 모터에 외부 전원을 연결하지 않았거나 속도가 너무 낮을 때(0.1~0.2)는 "부~웅" 소리만 나고 모터가 움직이지 않을 수 있습니다. 이를 통해 내 AVG가 굴러가기 시작하는 '최소 구동 속도'를 파악할 수 있습니다.

```
# file name : test_allmotor_speed.py
# Test All Motor Forward with Speed Control

from gpiozero import Motor
from time import sleep

motor1 = Motor(26, 19)
motor2 = Motor(22, 27)
motor3 = Motor(20, 21)
motor4 = Motor(24, 23)

print('Press Ctrl+C to stopped')
print('-'*30)

try:
    speed = 0.0
    step = 0.1
    is_increasing = True

    while True:
        print(f'Motor Forwarding. speed : {speed:.1f}', end='\r')
        motor1.forward(speed=speed)
        motor2.forward(speed=speed)
        motor3.forward(speed=speed)
        motor4.forward(speed=speed)

        if is_increasing:
            speed = round(speed + step, 1)

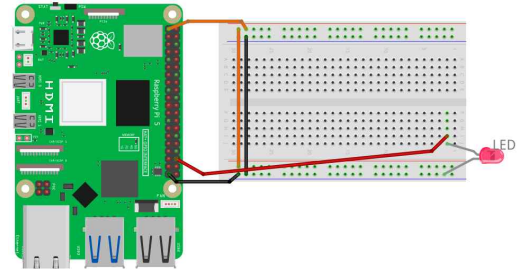
            if speed >= 1.0:
                is_increasing = False
        else:
            speed = round(speed - step, 1)
            if speed <= 0.0:
                is_increasing = True

        sleep(2)

except KeyboardInterrupt:
    print('Stopped. ctrl+c pressed.')
except Exception as err:
    print(f'Error : {err}')
finally:
    print('Motor stopped')
    motor1.stop()
    motor2.stop()
    motor3.stop()
    motor4.stop()
```


6) [실습] AI-Rover의 상태를 알리는 빛 : LED 연결

- ① 시스템 점검용으로 사용 혹은 주행시 돌발 상황이 생길 때 알람용
- ② LED 긴 선 : 라즈베리파이 GPIO 16
- ③ LED 짧은 선 : 공통 접지

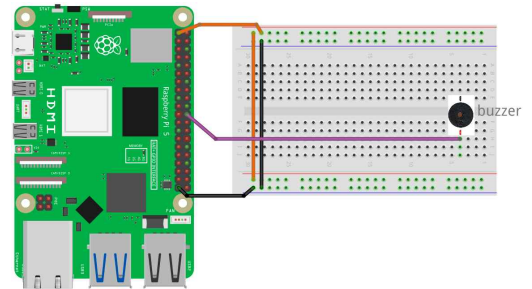


☐ TiP, 제일먼저 LED on/off 코드 실행하세요.

라즈베리파이를 접속한 후 제일 먼저 LED on/off 코드를 제일 먼저 실행함. 정상적으로 접속이 됐고 또한 라즈베리파이와 센서들이 정상적인지 파악하기 좋습니다.

7) [실습] AGV의 목소리 : 부저(Buzzer) 연결

- ① + : 라즈베리파이 GPIO 25
- ② - : 브레드보드 공통 접지



8) [테스트] 감각이 살아있나요? : 부품별 정상 작동 상태 점검

① 통합 테스트

☐ 통합 테스트를 해보신 후 동작이 되지 않는 센서들은 개별 테스트 하세요.

```
# file name: test_sensor.py
# Test order: Buzzer -> LED -> Motor

from time import sleep
from gpiozero import TonalBuzzer, LED, Motor

def test_buzzer():
    print('[1/3] Buzzer test')
    bz = TonalBuzzer(25)
    for _ in range(3):
        bz.play('A4')
        sleep(0.5)
        bz.stop()
        sleep(0.5)
    print('Buzzer test completed\n')

def test_led():
    print('[2/3] LED test')
    led1 = LED(16)
    for _ in range(3):
        led1.on()
        sleep(0.5)
        led1.off()
        sleep(0.5)
```

```

    print('LED test completed\n')

def test_motor():
    print('[3/3] Motor forward test')
    motor1 = Motor(26, 19)
    motor2 = Motor(22, 27)
    motor3 = Motor(20, 21)
    motor4 = Motor(24, 23)

    try:
        print('  Motor forward started...')
        motor1.forward()
        motor2.forward()
        motor3.forward()
        motor4.forward()
        sleep(1.5)
    finally:
        motor1.stop()
        motor2.stop()
        motor3.stop()
        motor4.stop()
    print('Motor test completed\n')

def main():
    print('All sensors sequential test started')
    print('-' * 30 + '\n')

    try:
        test_buzzer()
        sleep(1)

        test_led()
        sleep(1)

        test_motor()

        print('All tests completed!\n')
        print('-' * 30)

    except KeyboardInterrupt:
        print('\n\nTest interrupted\n')
    except Exception as err:
        print(f'\n\nError occurred: {err}')

if __name__ == '__main__':
    main()

```

2. 충돌. 안돼요~ : 전후방 초음파 센서를 활용한 작업자 감지 및 긴급 제동 시스템(AEB) 실전 코딩

1) AEB (Autonomous Emergency Braking, 긴급 제동 시스템)

스마트 팩토리에서 물류를 나르는 무인 운반차(AGV)에게 가장 중요한 원칙은 '작업자의 안전'입니다. AEB는 로봇이 주행 중 전방에 작업자나 장애물이 갑자기 튀어나왔을 때, 카메라나 초음파 센서로 이를 즉각 감지하고 스스로 모터를 정지시켜 충돌을 막아주는 첨단 안전 기술입니다.

이번 실습에서는 AGV의 전방 초음파 센서를 활용한 '긴급 제동(AEB)'과 후방 초음파 센서를 활용한 '후방 접근 경고(RCW)' 시스템을 동시에 구현해 봅니다.

2) 하드웨어 핀맵(Pin Map) 복습

코딩 전, 우리가 앞서 조립한 부품들의 주요 GPIO 핀 번호를 다시 한번 확인합니다.

- ① DC 모터: motor1(26, 19), motor2(27, 22), motor3(20, 21), motor4(24, 23)
- ② 전방 초음파 센서: Echo(12), Trig(13)
- ③ 후방 초음파 센서: Echo(17), Trig(4)
- ④ 경고 장치: 피에조 부저(25), LED(16)

3) 스마트 팩토리 적용 시나리오

- ① 정상 주행 : AGV가 물류 이동을 위해 초당 50%의 속도로 직진합니다.
- ② 긴급 제동 (전방 20cm 이내 장애물 감지) : 작업자가 앞을 가로막으면 즉시 모터를 멈추고 빨간색 LED를 켜며 경고음을 강하게 울립니다.
- ③ 주행 재개 : 작업자가 비켜서 20cm 밖으로 안전거리가 확보되면, 경고를 끄고 다시 직진합니다.
- ④ 후방 경고 (후방 15cm 이내 접근 감지) : 뒤에서 지게차나 다른 작업자가 너무 가까이 다가오면, 주행은 멈추지 않되 부저를 짧게 울려 '뒤쪽 위험'을 알립니다.

4) source code

```
# file name: agv_aeb_system.py
# 전후방 초음파 센서 기반 긴급 제동(AEB) 및 후방 경고 시스템

from gpiozero import Motor, DistanceSensor, TonalBuzzer, LED, Robot
from time import sleep
from signal import pause

# 1. 하드웨어 객체 생성
# 모터 설정 (이전 절에서 조립한 핀 번호)
motor1 = Motor(26, 19)
motor2 = Motor(27, 22)
motor3 = Motor(20, 21)
motor4 = Motor(24, 23)

# 오른쪽 축 및 왼쪽 축 설정
right_wheel = Robot(motor1, motor2)
left_wheel = Robot(motor3, motor4)
```

```

# 초음파 센서 설정 (threshold_distance: 이벤트가 발생하는 기준 거리 설정)
# 전방은 20cm(0.2m), 후방은 15cm(0.15m)로 설정
front_ultra = DistanceSensor(echo=12, trigger=13, max_distance=1.0, threshold_distance=0.2)
rear_ultra = DistanceSensor(echo=17, trigger=4, max_distance=1.0, threshold_distance=0.15)

# 경고용 출력 장치
buzzer = TonalBuzzer(25)
led = LED(16)

speed = 0.5 # AGV 기본 주행 속도 (50%)

# 2. 로봇 제어 통합 함수 정의
def stop_all_motors():
    right_wheel.stop()
    left_wheel.stop()

def drive_forward():
    right_wheel.forward(speed)
    left_wheel.forward(speed)

# 3. 센서 이벤트 발생 시 실행될 안전 로직(콜백 함수) 정의
def aeb_activation():
    """전방에 장애물 감지 시 긴급 제동"""
    stop_all_motors()
    led.on()
    buzzer.play('A4')
    print(f'\n[긴급 제동] 전방 작업자 감지! 충돌 방지 시스템 작동!')

def resume_driving():
    """전방 장애물이 사라지면 주행 재개"""
    led.off()
    buzzer.stop()
    print(f'\n[전방 안전 확보] AGV 물류 이송을 재개합니다.')
    drive_forward()

def rear_warning_on():
    """후방에 무언가 접근하면 짧은 경고음 발생 (주행은 유지)"""
    buzzer.play('E4')
    print(f'\n[주의] 후방 접근자 감지!')

def rear_warning_off():
    """후방 안전거리 확보 시 경고음 해제"""
    buzzer.stop()
    print(f'\n[후방 안전 확보] 후방 안전 확보.')

# 4. 센서와 이벤트 함수 연결 (이벤트 구동 방식)
front_ultra.when_in_range = aeb_activation
front_ultra.when_out_of_range = resume_driving

```

```

rear_ultra.when_in_range = rear_warning_on
rear_ultra.when_out_of_range = rear_warning_off

# 5. 메인 실행부
print('=====')
print(' 스마트 팩토리 AGV 안전 주행 시스템 가동 시작')
print(' 전방 20cm 감지 시 정지 / 후방 15cm 감지 시 경고')
print(' 종료하려면 Ctrl+C 를 누르세요.')
print('=====')

try:
    # 최초 주행 시작
    drive_forward()

    # 프로그램이 종료되지 않고 백그라운드에서 센서 이벤트를 계속 감지하도록 무한 대기
    pause()

except KeyboardInterrupt:
    print('\n관리자에 의해 AGV 시스템이 종료되었습니다.')

finally:
    # 프로그램 종료 시 AGV가 폭주하지 않도록 모든 하드웨어 자원 정지 및 반환
    stop_all_motors()
    buzzer.stop()
    led.off()
    print('모터 및 안전 시스템의 전원을 안전하게 차단했습니다.')

```

□ Tip 이벤트 기반 제어의 강력함

코드를 자세히 보면 "메인 실행부(try 구문 안)에 복잡한 while 문과 if 문이 없죠?"

로봇 코딩을 처음 할 때 흔히 while 루프 안에서 "초음파 센서 거리 재기 → 20cm 이하인지 비교 → 모터 끄기"를 무한 반복하는 코드를 씁니다. 하지만 이렇게 짜면 로봇이 다른 작업(예: 카메라로 사물 인식, 서버와 통신)을 할 때 코드가 멈칫거리며 제동 타이밍을 놓칠 수 있습니다.

우리가 사용한 when_in_range 속성은 백그라운드에서 센서를 감시하다가, 위험 거리에 도달하는 그 순간에만 낚실탄 채듯 즉각 함수(aeb_activation)를 실행시킵니다. 이를 통해 라즈베리파이의 메인 두뇌는 쉴 수 있고, 로봇의 반응 속도는 획기적으로 빨라집니다.

'이벤트 구동 프로그래밍'의 철학이 멋있지 않나요?

3. 덜컹거림 없는 주행의 비밀 : 노이즈를 극복하기 위한 센서 데이터 필터링 기초

1) 센서 노이즈(Noise)란 무엇인가요?

초음파 센서(HC-SR04)는 음파를 쏘고 돌아오는 시간을 재서 거리를 측정합니다. 하지만 현장의 기계 소음이나 바닥의 요철, 측정 대상의 재질 문제로 인해 실제 거리는 20cm인데 가끔 5cm나 100cm처럼 엉뚱한 값(노이즈)이 튀어나올 때가 있습니다.

만약 이 튀는 값을 그대로 모터의 속도 조절이나 정지 조건(AEB)에 사용하면, 로봇이 혼자서 멈칫거리거나 브레이크를 밟는 등 매우 불안정하게 주행합니다.

① 해결책: 필터링 (Filtering)

이런 튀는 값을 걸러내고 데이터를 부드럽게 만드는 기술을 '필터링'이라고 합니다. 이번 실습에서는 가장 기본적인 '이동 평균 필터(Moving Average Filter)'를 파이썬 코드로 구현해 봅니다.

2) 이동 평균 필터(Moving Average Filter)의 원리

원리는 간단합니다. 방금 측정한 데이터 하나만 믿지 않고, 최근에 측정한 데이터 N개의 평균을 내서 진짜 거리라고 믿는 것입니다. (여기서 N을 '윈도우 크기(Window Size)'라고 부릅니다.)

① 원리 : 최근 측정값 5개를 주머니(리스트)에 담아두고 평균을 구합니다. 새로운 값이 들어오면 가장 오래된 값을 버리고 새 값을 넣어 다시 평균을 냅니다.

② 장점 : 갑자기 엉뚱한 값이 하나 튀어도, 나머지 정상적인 값들과 평균이 섞이면서 로봇이 부드럽게 반응하게 됩니다.

3) Source Code (필터가 적용된 초음파 센서)

파이썬 리스트(List) 구조를 활용하여 최신 5개의 데이터를 모아 평균을 내는 필터 함수를 만들어, 앞서 2장에서 배운 초음파 테스트 코드에 접목해 봅니다.

```
# file name: filtering_ultra.py
# 이동 평균 필터를 적용한 초음파 센서 노이즈 제거 테스트

from gpiozero import DistanceSensor
from time import sleep

# 전방 초음파 센서 객체 생성 (Echo: 12, Trig: 13)
front_ultra = DistanceSensor(echo=12, trigger=13, max_distance=1.0)

# 이동 평균 필터를 위한 설정
WINDOW_SIZE = 5      # 최근 측정값 5개를 모아서 평균을 냅니다.
history_data = []     # 측정된 데이터들을 담아둘 빈 리스트(주머니) 생성

def get_filtered_distance(raw_distance):
    """
    들어온 raw_distance를 리스트에 넣고, 최근 5개의 평균값을 반환하는 함수
    """
    # 1. 새 데이터를 리스트 맨 끝에 추가 (단위 변환: m -> cm)
    history_data.append(raw_distance * 100)

    # 2. 리스트의 개수가 WINDOW_SIZE(5개)를 넘어가면, 가장 앞쪽(오래된) 데이터 삭제
```

```

if len(history_data) > WINDOW_SIZE:
    history_data.pop(0)

# 3. 리스트에 담긴 데이터들의 합을 개수로 나누어 평균(Average) 계산
average_distance = sum(history_data) / len(history_data)

return average_distance

print('=====')
print(' 초음파 센서 필터링 테스트 시작 (Ctrl+C 종료)')
print('=====')

try:
    while True:
        # 센서에서 거리를 한 번 읽어옴 (Raw 데이터)
        raw_val = front_ultra.distance

        # 필터 함수를 거쳐 부드러워진 평균 거리를 얻어옴
        filtered_val = get_filtered_distance(raw_val)

        # 원본 데이터와 필터링된 데이터를 나란히 출력하여 비교해 봅니다.
        # \r을 사용하여 화면이 아래로 내려가지 않고 한 줄에서 계속 갱신되게 합니다.
        print(f'[원본] {raw_val*100:5.1f}cm | [필터 적용] {filtered_val:5.1f}cm', end='\r')

        sleep(0.1) # 0.1초마다 측정

except KeyboardInterrupt:
    print('\n테스트가 종료되었습니다.')

```


Chapter 5. Vision AI 기반 품질 검수(머신 비전)

1. 설비의 시신경 깨우기 : Picamera2 라이브러리를 활용한 실시간 영상 스트리밍 및 기초 설정

1) 스마트 팩토리 및 머신 비전(Machine Vision)

스마트 팩토리에서 카메라는 단순한 촬영 장비가 아닙니다. 컨베이어 벨트를 지나가는 제품의 스크래치를 찾아내고, AGV(무인 운반차)가 신호등의 색상을 판별하게 해주는 '설비의 시신경(Optic Nerve)'입니다. 이번 실습에서는 라즈베리파이 공식 카메라 모듈과 최신 Picamera2 라이브러리를 사용하여, 카메라가 받아들인 빛을 파이썬 코드로 읽어 들이고 모니터(VNC 화면)에 실시간으로 출력하는 기초 시각화 작업을 진행합니다.

2) 하드웨어 연결 및 환경 설정

① 라즈베리파이 4에서 카메라 연결

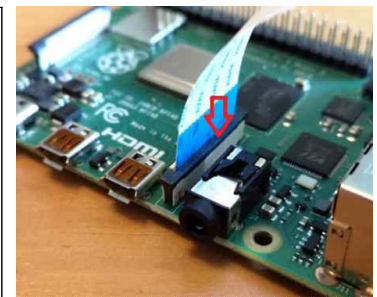
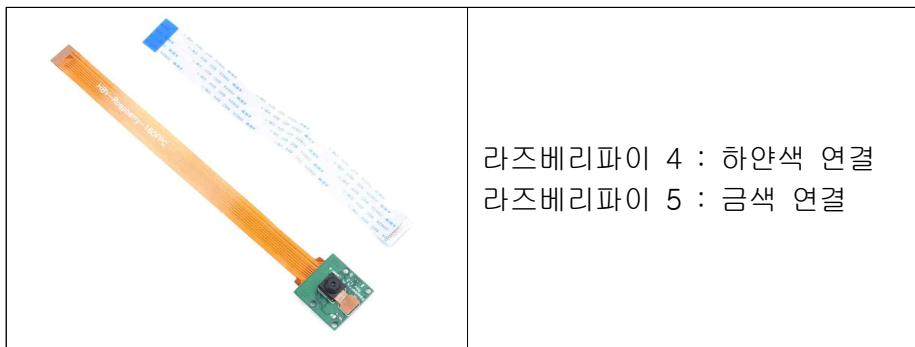
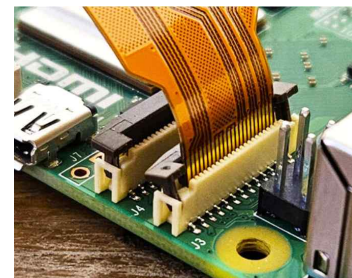


그림 286 라즈베리파이 4

② 라즈베리파이 5 전용 카메라 케이블 확인



- ☐ 라즈베리파이 5는 기존 모델(3, 4)보다 카메라 연결 단자(CSI 포트)의 크기가 작아졌습니다. 따라서 기존 카메라 모듈을 연결하려면 반드시 '라즈베리파이 5 전용 미니 필름 케이블(22-pin to 15-pin)'로 교체하여 연결해야 합니다.
- ☐ 카메라 연결 : 라즈베리파이 5 보드의 'CAM/DISP' 포트 커버를 조심스럽게 들어 올립니다.
- ☐ 방향 주의 : 필름 케이블의 금속 접점(은색 부분)이 보드 안쪽(가운데)을 향하도록 꽂고 커버를 닫아 고정합니다.
- ☐ 카메라 고정 : AGV의 서보모터 헤드 브라켓(전방 초음파 센서 위쪽)에 양면테이프나 나사로 카메라 모듈을 단단히 부착합니다.

③ 연결 확인(터미널)

- ☐ 아래 명령어 실행. preview 상태가 보이면 정상

```
(ai_rover_venv) $ libcamera-hello
```

- ☐ 라즈베리파이 4B모델 이후부터는 카메라 자동 연결됨, interface 설정 않아도 됨.

3) 라이브러리 : Picamera2

① 개요

- ☐ Raspberry Pi의 카메라 시스템에 접근할 수 있는 Python 라이브러리
- ☐ Raspberry Pi의 카메라와 이미징 하드웨어의 기능을 효과적으로 활용

② 연결 확인(터미널)

- 아래 두 개의 명령어 중 한 개만 실행해보세요
 - preview 상태가 보이면 정상

```
(basic) $ libcamera-hello
```

```
(basic) $ rpikam-hello
```

③ 카메라 촬영

```
# file name: test_snapshot.py
# Picamera2를 활용한 단일 사진 촬영 및 파일 저장

from picamera2 import Picamera2
from time import sleep

# 1. 카메라 객체 생성 및 사진용 해상도 자동 설정
picam2 = Picamera2()
picam2.configure(picam2.create_still_configuration())

# 2. 카메라 렌즈 열기 (가동)
picam2.start()
print('카메라 센서 예열 중...')

# [중요] 카메라가 켜진 직후에는 빛을 조절(자동 노출 및 화이트 밸런스)할 시간이 필요합니다.
# 이 대기 시간이 없으면 사진이 시커멓게 찍힐 수 있습니다.
sleep(2)

# 3. 사진 촬영 및 파일로 바로 저장
file_name = "test_photo.jpg"
picam2.capture_file(file_name)
print(f'사진 촬영 완료! 현재 폴더에 {file_name} 파일이 저장되었습니다.')

# 4. 하드웨어 자원 반환 (카메라 렌즈 닫기)
picam2.stop()
```

④ 동영상 촬영

```
# file name: test_snapshot.py
# Picamera2를 활용한 단일 사진 촬영 및 파일 저장

from picamera2 import Picamera2
from time import sleep
```

```
# 1. 카메라 객체 생성 및 사진용 해상도 자동 설정
picam2 = Picamera2()
picam2.configure(picam2.create_still_configuration())

# 2. 카메라 렌즈 열기 (가동)
picam2.start()
print('카메라 센서 예열 중...')

# [중요] 카메라가 켜진 직후에는 빛을 조절(자동 노출 및 화이트 밸런스)할 시간이 필요합니다.
# 이 대기 시간이 없으면 사진이 시커멓게 찍힐 수 있습니다.
sleep(2)

# 3. 사진 촬영 및 파일로 바로 저장
file_name = "test_photo.jpg"
picam2.capture_file(file_name)
print(f'사진 촬영 완료! 현재 폴더에 {file_name} 파일이 저장되었습니다.')

# 4. 하드웨어 자원 반환 (카메라 렌즈 닫기)
picam2.stop()
```

4) 색깔이 뒤집혔어요! : OpenCV만의 독특한 색 표현 방식(BGR)

자율주행 영상 처리에 전 세계적으로 가장 많이 쓰이는 파이썬 라이브러리가 바로 OpenCV(cv2)임.

① 이미지 파일 읽는 순서

일반 이미지 파일	OpenCV 파일
Red-Green-Blue (RGB)	Blue-Green-Red (BGR)

- ☐ OpenCV가 처음 개발되던 과거 하드웨어 제조사들의 표준이 BGR이었기 때문이며, 이 전통이 지금까지 이어지고 있음.
- ☐ OpenCV로 읽은 이미지를 다른 AI 모델(YOLO 등)에 넘겨줄 때는, 색상이 파랗게 변하지 않도록 반드시 BGR을 RGB로 변환(cv2.cvtColor)해 주어야 하는 경우가 많음.

```
# file name: bgr_to_rgb.py
import cv2

# [기초] 저장된 이미지 파일 읽기
img_bgr = cv2.imread("image.jpg")
img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)

# [AGV 실전 팁] 카메라가 실시간으로 뜯 눈! 2.2절에서 확인할게요.
# cap = cv2.VideoCapture(0) # 0번 카메라 연결
# ret, frame_bgr = cap.read() # 카메라에서 1프레임 찰칵!
# frame_rgb = cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB)
```

5) Source Code

카메라의 영상을 읽어와 OpenCV 창에 실시간으로 띄우는(스트리밍) 기본 뼈대 코드입니다. 이 코드는 앞으로 진행될 '색상 추적', 'YOLO 사물인식' 등 모든 비전 AI 실습의 도화지가 됩니다.

```
# file name: test_camera_stream.py
# Picamera2와 OpenCV를 활용한 실시간 영상 스트리밍
```

```

from picamera2 import Picamera2
import cv2
from time import sleep

# 1. Picamera2 객체 생성 및 초기화
picam2 = Picamera2()

# 2. 카메라 해상도 및 포맷 설정
# 스마트 팩토리 실시간 비전 처리는 연산 속도가 생명이므로,
# 해상도를 640x480으로 낮춰서 설정합니다. (FHD 1080p는 연산이 느려짐)
config = picam2.create_preview_configuration(main={"size": (640, 480)})
picam2.configure(config)

# 3. 카메라 가동 시작
picam2.start()
print('=====')
print(' AGV 시신경 가동 완료! (스트리밍 시작)')
print(' 비디오 창을 선택한 후 "q" 키를 누르면 종료됩니다.')
print('=====')

# 영상 출력을 위한 창 생성
cv2.namedWindow("AGV Vision", cv2.WINDOW_AUTOSIZE)

try:
    while True:
        # 4. 카메라로부터 현재 프레임(한 장의 사진)을 넘파이 배열(Array) 형태로 캡처
        # 이 frame 변수 안에 수십만 개의 픽셀(색상 데이터)이 숫자로 담겨 있습니다.
        frame = picam2.capture_array()

        # 5. OpenCV를 이용해 화면에 프레임 출력
        cv2.imshow("AGV Vision", frame)

        # 6. 종료 조건: 1ms 동안 키보드 입력을 대기하며, 'q' 키가 눌리면 루프 탈출
        # (주의: VNC 환경에서 비디오 창을 마우스로 클릭하여 활성화한 상태에서 눌러야 함)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            print("사용자에 의해 영상 스트리밍을 종료합니다.")
            break

except Exception as e:
    print(f"카메라 스트리밍 중 오류 발생: {e}")

finally:
    # 7. 하드웨어 자원 안전 반환
    cv2.destroyAllWindows() # 열려있는 모든 OpenCV 창 닫기
    picam2.stop()          # 카메라 렌즈 닫기
    print("카메라 자원이 안전하게 반환되었습니다.")

```

2. [실습] 인공지능의 진짜 맛을 보다 : 사전 학습(Pre-trained) YOLO 모델로 작업자 인식하기

1) 개요 : 준비된 인공지능, 사전 학습(Pre-trained) 모델

현장의 직장인 분들께 수천 장의 사진을 찍고 라벨링하는 과정은 너무 지루하고 벅찬 일일 수 있습니다. 그래서 이번 실습에서는 '이미 대학 과정을 마친 똑똑한 신입사원'을 데려와 바로 현장에 투입하는 방법을 배웁니다.

전 세계의 천재적인 AI 연구자들이 수백만 장의 사진(COCO 데이터셋)을 슈퍼컴퓨터로 미리 학습시켜 놓은 모델 파일(Pre-trained Model)을 무료로 제공하고 있습니다. 우리는 단 한 줄의 코드로 이 뇌를 다운로드하여, AGV가 곧바로 사람(작업자), 스마트폰, 의자, 물병 등 80가지의 사물을 즉각적으로 알아 보게 만들 것입니다.

2) YOLO

3) 모델 준비 : 초경량 Nano 모델 선택

라즈베리파이 5와 같은 엣지 디바이스에서는 무거운 모델을 돌리면 화면이 심하게 끊깁니다. 우리는 가장 가볍고 속도가 빠른 YOLOv11 Nano 모델(yolo11n.pt)을 사용합니다. 코드를 처음 실행할 때 파이썬이 알아서 인터넷에서 이 뇌 파일(약 6MB)을 다운로드해 줍니다.

4) Source Code : AGV 인공지능 시력 테스트

카메라 스트리밍 코드에 YOLO 라이브러리를 결합하여 실시간으로 사물을 탐지하는 코드입니다.

```
# file name : agv_yolo_pretrained.py
# 사전 학습된 YOLO 모델을 활용한 실시간 객체 탐지

from picamera2 import Picamera2
from ultralytics import YOLO
import cv2

# 1. Picamera2 초기화 및 스트리밍 시작
picam2 = Picamera2()
config = picam2.create_preview_configuration(main={"size": (640, 480)})
picam2.configure(config)
picam2.start()

print('=====')
print(' AGV 인공지능 시력 가동 준비 중...')
print(' (최초 실행 시 yolo11n.pt 다운로드로 수 초 소요)')
print('=====')

# 2. 인공지능 뇌 이식 : 사전 학습된 모델 자동 다운로드 및 불러오기
# 경로에 파일이 없으면 ultralytics 서버에서 자동으로 받아옵니다.
model = YOLO('yolo11n.pt')

cv2.namedWindow("AGV AI Vision", cv2.WINDOW_AUTOSIZE)

try:
    while True:
```

```

# 3. 카메라에서 프레임 읽기
frame = picam2.capture_array()

# 4. YOLO 모델에 프레임을 전달하여 사물 탐지 수행
# conf=0.5 : 인공지능이 50% 이상 확신하는 사물만 표시합니다.
# verbose=False : 터미널에 출력되는 복잡한 연산 과정을 숨깁니다.
results = model.predict(source=frame, conf=0.5, verbose=False)

# 5. 분석 결과(바운딩 박스, 사물 이름, 확률)가 덧그려진 이미지 추출
annotated_frame = results[0].plot()

# 6. 화면 출력
cv2.imshow("AGV AI Vision", annotated_frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    print("\n인공지능 비전 테스트를 종료합니다.")
    break

except Exception as e:
    print(f"오류 발생 : {e}")

finally:
    # 7. 자원 반환
    cv2.destroyAllWindows()
    picam2.stop()

```

3. 인공지능의 눈과 로봇의 몸 연결하기 : 탐지된 객체 추출 및 판별 로직

1) 화면에 그리는 것을 넘어, 데이터로 읽어오기

앞선 실습에서 YOLO는 사물을 찾아내 화면에 멋진 네모를 그려주었습니다. 하지만 라즈베리파이(로봇의 두뇌)는 화면의 그림을 보고 모터를 멈추지 않습니다. 우리는 YOLO가 분석한 결과 속에서 "어떤 사물들이 발견되었는지" 이름(텍스트)만 쏙 뽑아내어 파이썬 리스트(List)로 만드는 기술이 필요합니다. 그래야 "만약 리스트에 'person(사람)'이 있으면 모터를 멈춰라!" 같은 제어 코드를 작성할 수 있기 때문입니다.

2) [실습] 스마트폰을 불량품으로 인식하는 비전 검수기 만들기

이번 실습에서는 카메라 앞에 다양한 사물이 지나갈 때, 오직 '스마트폰(cell phone)'이 등장했을 때만 이를 불량품으로 인식하고 경고 메시지를 띄우는 실전 판별 로직을 작성해 봅니다.

```
# file name : vision_defect_check.py
# 탐지된 객체 이름을 추출하여 특정 사물(불량품) 판별하기

from picamera2 import Picamera2
from ultralytics import YOLO
import cv2

picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(main={"size" : (640, 480)}))
picam2.start()

model = YOLO('yolo11n.pt')

print('=====')
print(' 비전 검수 로직 테스트 가동')
print(' 카메라 앞에 스마트폰을 비춰보세요.')
print('=====')

cv2.namedWindow('Vision Inspector', cv2.WINDOW_AUTOSIZE)

try:
    while True:
        frame = picam2.capture_array()
        results = model.predict(source=frame, conf=0.6, verbose=False)
        annotated_frame = results[0].plot()

        # [핵심] 화면에 인식된 사물들의 '이름'만 뽑아서 리스트로 만들기
        # 예: ['person', 'cell phone']
        detected_classes = [model.names[int(box.cls)] for box in results[0].boxes]

        # 판별 로직: 리스트 안에 'cell phone'이 포함되어 있는지 확인
        if 'cell phone' in detected_classes:
            # 화면 좌측 상단에 붉은색 글씨로 경고 출력
            cv2.putText(annotated_frame, 'DEFECT DETECTED!', (30, 50),
                        cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 3)
            print('불량품(스마트폰) 감지!')

        cv2.imshow('Vision Inspector', annotated_frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

except Exception as err:
```



```
print(f'오류 발생 : {err}')
finally:
    cv2.destroyAllWindows()
    picam2.stop()
```

3) 사전 학습(COCO) 클래스 활용

① 음료 및 패키징 공정 (F&B 라인)

- ☐ bottle : 생수병이나 음료수병. "내용물이 채워진 정상 제품" 또는 반대로 "잘못 섞여 들어온 타사 제품"으로 시나리오를 짜기 좋습니다.
- ☐ cup: 종이컵이나 텀블러. 포장 용기 검수 시나리오에 적합합니다.

② 전자제품 조립 공정

- ☐ mouse : 강의실에 있는 컴퓨터 마우스를 컨베이어 벨트 위에 올려두고 전자 부품 조립 라인의 "정상 조립품"으로 간주할 수 있습니다.
- ☐ laptop: 노트북. 라인에 진입하는 대형 전자 기기 부품으로 활용 가능합니다.

③ 스마트 농업 및 식품 분류

- ☐ apple, banana, orange: 실제 과일이나 다이소 등에서 파는 저렴한 과일 모형을 활용하면 아주 직관적인 식품 분류 및 선별기(Sorting Machine) 시나리오가 똑딱 완성됩니다.

④ 이물질 및 위험물 혼입 검사

- ☐ scissors: 가위. 식품이나 안전이 중요한 공정 라인에 절대 들어가서는 안 되는 '금속 이물질'이나 '위험 도구'가 혼입된 비상 상황을 연출할 때 매우 효과적입니다.

names:	20: elephant	41: cup	62: tv
0: person	21: bear	42: fork	63: laptop
1: bicycle	22: zebra	43: knife	64: mouse
2: car	23: giraffe	44: spoon	65: remote
3: motorcycle	24: backpack	45: bowl	66: keyboard
4: airplane	25: umbrella	46: banana	67: cell phone
5: bus	26: handbag	47: apple	68: microwave
6: train	27: tie	48: sandwich	69: oven
7: truck	28: suitcase	49: orange	70: toaster
8: boat	29: frisbee	50: broccoli	71: sink
9: traffic light	30: skis	51: carrot	72: refrigerator
10: fire hydrant	31: snowboard	52: hot dog	73: book
11: stop sign	32: sports ball	53: pizza	74: clock
12: parking meter	33: kite	54: donut	75: vase
13: bench	34: baseball bat	55: cake	76: scissors
14: bird	35: baseball glove	56: chair	77: teddy bear
15: cat	36: skateboard	57: couch	78: hair drier
16: dog	37: surfboard	58: potted plant	79: toothbrush
17: horse	38: tennis racket	59: bed	
18: sheep	39: bottle	60: dining table	
19: cow	40: wine glass	61: toilet	

Chapter 6. 공장 중앙 센터 웹 모니터링

스마트 팩토리 현장(OT)에서 아무리 많은 데이터를 수집하고 로봇이 부지런히 움직여도, 관리자(IT)가 이를 한눈에 파악하고 통제할 수 없다면 반쪽짜리 자동화에 불과합니다.

과거 산업 현장에서는 설비 상태를 확인하기 위해 기계 앞에 설치된 전용 모니터(HMI) 패널을 직접 들여다보거나, 무거운 SCADA(원격 감시 제어) 시스템을 구축해야 했습니다. 하지만 최신 스마트 팩토리는 웹(Web) 기반의 관제 시스템을 도입하여, 전 세계 어디서든 스마트폰이나 태블릿 브라우저만 열면 공장 상황을 실시간으로 확인하고 제어할 수 있습니다.

이번 6장에서는 파이썬을 활용해 라즈베리파이를 '스마트 팩토리 중앙 관제 서버'로 탈바꿈시키고, 스마트폰이 곧 공장의 운전대가 되는 놀라운 경험을 실습해 봅니다.

1. 스마트 팩토리 웹 서버의 만남 : Flask 기초

서버와 클라이언트의 개념을 이해하고, 라즈베리파이를 가장 빠르고 가볍게 웹 서버로 변신시켜 주는 파이썬 'Flask' 라이브러리의 기초를 배웁니다.

핵심 내용 : 라우팅(@app.route)의 개념 이해, 스마트폰 웹 브라우저에서 라즈베리파이 IP로 접속하여 "Hello, Factory!" 화면 띄워보기

1) 왜 Flask(플라스크)인가?

우리가 매일 사용하는 네이버나 유튜브 같은 웹 사이트들은 수많은 기능이 얹혀 있는 거대한 시스템입니다. 이런 거대한 웹을 파이썬으로 만들 때는 보통 'Django(장고)'라는 뼈대(프레임워크)를 사용합니다. 하지만 우리의 라즈베리파이는 자원이 한정된 엣지(Edge) 디바이스입니다. 온도 하나 띄우고 모터 하나 돌리자고 무거운 뼈대를 가져올 필요는 없습니다. 파이썬에는 단 5줄의 코드만으로 완벽한 웹 서버를 구동해 주는 초경량 프레임워크인 'Flask(플라스크)'가 존재합니다. 가볍고 빠르기 때문에 현업의 마이크로 서비스 아키텍처(MSA)나 IoT 디바이스 관제에 가장 널리 쓰이는 핵심 기술입니다.

2) 환경 준비 : Flask 설치

가상환경(smart_factory_venv)이 활성화된 상태인지 터미널 프롬프트를 확인한 후, Flask 라이브러리를 설치합니다.

```
(smart_factory_venv) $ pip install flask
```

3) [실습] 세상에서 가장 작은 관제 서버 열기 (Hello, Factory!)

가장 기본이 되는 첫 번째 웹 서버를 만들어 보겠습니다. 코드를 작성하고 실행하면, 여러분의 라즈베리파이는 훌륭한 웹 서버로 변신합니다.

```
# file name : flask_hello.py
# Flask 웹 서버 구동 기초 테스트

from flask import Flask

# 1. Flask 서버 객체 생성 (우리의 공장 웹 서버)
app = Flask(__name__)

# 2. 라우팅(Routing) 설정 : 접속할 주소와 화면 연결
# 사용자가 웹 브라우저 주소창에 아무것도 안 붙이고 기본 주소('/')로 접속했을 때 실행될 함수
@app.route('/')

```

```
def home():
    # 웹 브라우저 화면에 띄워줄 글자(HTML)를 반환합니다.
    return "<h1>Hello, Smart Factory!</h1><p>이 화면이 보인다면 서버 접속 성공입니다.</p>"

# 3. 서버 구동
if __name__ == '__main__':
    print("스마트 팩토리 웹 서버를 시작합니다...")
    # host='0.0.0.0' : 외부(스마트폰 등)의 접속을 허용하겠다는 마법의 옵션
    # port=5000 : Flask의 기본 접속 포트(출입문 번호)
    app.run(host='0.0.0.0', port=5000)
```

4) 접속 및 확인 : 내 스마트폰으로 서버 들어가기

코드를 실행(Run)하면 터미널 하단에 Running on http://0.0.0.0:5000이라는 메시지가 뜨며 서버가 켜진 채로 대기 상태에 들어갑니다. 프로그램이 종료된 것이 아니라 '손님을 기다리고 있는 상태(Listen)'입니다.

자, 이제 스마트폰을 꺼내어 라즈베리파이가 보내는 화면을 확인해 봅시다.

- ① 동일 네트워크 확인 : 스마트폰이 라즈베리파이와 동일한 와이파이(또는 핫스팟)에 연결되어 있는지 확인합니다. (LTE/5G 데이터는 잠시 꺼두는 것이 좋습니다.)
 - ② IP 주소 확인 : 라즈베리파이 터미널에서 ifconfig 명령을 입력해 무선랜(wlan0)의 IP 주소를 확인합니다. (예: 192.168.137.50)
 - ③ 브라우저 접속 : 스마트폰의 웹 브라우저(크롬, 사파리 등)를 열고 주소창에 아래와 같이 입력합니다. 포트 번호(:5000)를 잊지 마세요.
- ☐ 접속 주소 : http://자신의IP주소:5000
(예: http://192.168.137.50:5000)

스마트폰 화면에 커다란 글씨로 "Hello, Smart Factory!"가 출력되었나요? 축하합니다! 여러분은 엷지 디바이스 기반의 웹 관제 서버를 스스로 구축해 내는 데 성공했습니다.

💡 Tip (라우팅과 host='0.0.0.0'의 중요성)

@app.route('/test')라고 코드를 수정하면, 스마트폰 접속 주소는 http://IP주소:5000/test가 됩니다. 이렇게 주소창의 경로에 따라 각기 다른 동작을 지정해 주는 것을 '라우팅(Routing)'이라고 합니다.

host='0.0.0.0' 옵션은 초보자들이 가장 많이 실수하는 부분입니다. 이 옵션을 빼고 실행하면 Flask는 기본적으로 자신의 내부(127.0.0.1)에서만 접속을 허용합니다. 즉, 라즈베리파이 본체에서는 화면이 열리지만 외부의 스마트폰으로는 절대 접속할 수 없습니다. 현장에서 통신 에러가 난다면 가장 먼저 이 옵션이 빠졌거나 오타 등을 체크해야 합니다.

2. 실시간 데이터 스트리밍 : 온습도 대시보드

단순히 "Hello"라는 고정된 텍스트를 띄우는 서버를 넘어, 이번에는 2장에서 실습했던 온습도 센서(DHT11)를 웹 서버와 결합해 보겠습니다.

라즈베리파이에 연결된 센서가 측정한 물리적인 현장(OT) 데이터를 파이썬 서버(IT)가 읽어 들여, 내 스마트폰의 예쁜 대시보드 화면에 실시간으로 띄워주는 '데이터 모니터링 시스템'의 뼈대를 완성하는 실습입니다.

1) 화면이 깜빡이지 않는 마법 : 비동기 통신(AJAX/Fetch)의 이해

과거의 웹 페이지들은 새로운 데이터를 확인하려면 브라우저의 '새로고침(F5)' 버튼을 눌러야만 했습니다. 화면 전체가 하얗게 깜빡이며 다시 로딩되는 방식이죠.

하지만 스마트 팩토리의 관제 모니터가 1초마다 깜빡거린다면 관리자의 눈이 피로해서 작업을 할 수 없을 것입니다.

그래서 현대의 웹 대시보드는 '화면(UI) 껍데기'는 한 번만 그려두고, 알맹이(데이터)만 백그라운드에서 몰래몰래 가져와서 숫자만 갈아 끼우는 기술을 사용합니다. 이를 비동기 통신(AJAX, Fetch)이라고 합니다. 이번 실습 코드에 적용된 이 놀라운 기술을 직접 체감해 보십시오.

2) [실습] 스마트폰에 현장 온습도 대시보드 띄우기

2장에서 구성했던 DHT11 센서(GPIO 21번 핀 연결)가 라즈베리파이에 잘 꽂혀 있는지 확인한 후, 아래의 코드를 작성하고 실행합니다.

```
<!DOCTYPE html>
<html>
<head>
  <title>환경 모니터링 대시보드</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    body { font-family: 'Malgun Gothic', sans-serif; background-color: #2c3e50; color:
white; text-align: center; padding-top: 50px; }
    .card { background-color: #34495e; margin: 20px auto; padding: 30px; border-radius:
15px; width: 80%; max-width: 400px; box-shadow: 0 4px 8px rgba(0,0,0,0.2); }
    .value { font-size: 40px; font-weight: bold; color: #f1c40f; margin: 10px 0; }
    .label { font-size: 18px; color: #bdc3c7; }
  </style>
  <script>
    // 1초(1000ms)마다 데이터를 새로고침 없이 가져오는 마법의 코드
    setInterval(function() {
      fetch('/api/sensor')
        .then(response => response.json())
        .then(data => {
          document.getElementById('temp_value').innerText = data.temperature + " °C";
          document.getElementById('humid_value').innerText = data.humidity + " %";
        })
        .catch(error => console.log("데이터 수신 에러"));
    }, 1000);
  </script>
</head>
<body>
  <h2>□ 스마트 팩토리 환경 관제</h2>
```

```

<div class="card">
  <div class="label">현재 온도 (Temperature)</div>
  <div class="value" id="temp_value">로딩 중...</div>
</div>

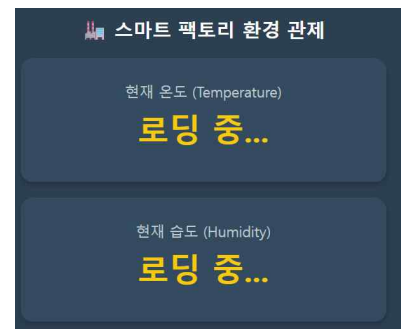
<div class="card">
  <div class="label">현재 습도 (Humidity)</div>
  <div class="value" id="humid_value">로딩 중...</div>
</div>
</body>
</html>

```

3) 실행 및 결과 확인

코드를 실행한 뒤, 6.1절과 마찬가지로 스마트폰 브라우저에서 <http://IP주소:5000>으로 접속해 보세요.

처음에는 '로딩 중...'이라고 뜨던 화면이 곧바로 센서의 데이터로 채워집니다. 센서를 손으로 꺾 쥐고 입김을 불어보세요. 스마트폰의 화면이 깜빡임 없이 부드럽게 온도와 습도 수치만 계속 업데이트되는 것을 확인할 수 있습니다.



3. 스마트폰이 운전대가 되다 : AGV 원격 제어

로봇이 바닥에 그려진 선만 따라가는 물리적 방식(라인 트레이서)은 센서의 감도, 바닥의 재질, 조명의 반사도 등 외부 환경에 매우 취약합니다.

실제 복잡한 스마트 팩토리나 위험한 재난 현장에서는 중앙 관제실의 관리자가 로봇을 직접 조종하는 '원격 제어(Teleoperation)' 기술이 필수적으로 사용됩니다. 이번 실습에서는 6.2절에서 배운 웹 서버 기술을 응용하여, 내 스마트폰을 AGV의 무선 조이스틱(운전대)으로 탈바꿈시켜 보니다.

1) 웹 버튼으로 모터를 돌리는 원리

스마트폰 화면의 버튼을 눌러 물리적인 바퀴를 굴리려면 3단계를 거쳐야 합니다.

스마트폰 (클라이언트) : 화면의 '전진' 버튼을 누르면, 백그라운드에서 /move/forward라는 주소를 몰래 호출(Fetch)합니다.

라즈베리파이 (웹 서버) : Flask가 /move/forward라는 접속 요청을 감지하고, 이 주소와 연결된 파이썬 함수를 실행합니다.

하드웨어 (엣지 제어) : 파이썬 함수 안에서 4장에서 배웠던 motor.forward() 명령이 실행되며 바퀴가 굴러갑니다

2) [실습] 무선 조이스틱 웹 서버 구축하기

4장에서 조립했던 AGV의 모터 전원(배터리)이 켜져 있는지 확인한 후, 아래의 코드를 작성하여 실행합니다.

```
<!DOCTYPE html>
<html>
<head>
  <title>AGV 무선 원격 제어</title>
  <meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no">
  <style>
    body { text-align: center; margin-top: 30px; font-family: sans-serif; background-color: #2c3e50; color: white; }
    .btn { width: 80px; height: 80px; font-size: 30px; margin: 10px; border-radius: 15px; background-color: #3498db; color: white; border: none; font-weight: bold; }
    .btn:active { background-color: #e74c3c; } /* 버튼을 누르면 빨간색으로 변경 */
    .btn-stop { background-color: #95a5a6; }
    table { margin: 0 auto; }
  </style>
  <script>
    // 서버로 이동 명령을 몰래 보내는 함수
    function sendCmd(cmd) {
      fetch('/move/' + cmd);
    }
  </script>
</head>
<body>
  <h2>□ AGV Remote Control</h2>
  <p>버튼을 누르고 있는 동안만 이동합니다.</p>

  <!-- 십자 모양 방향키 배열 -->
  <table>
    <tr>
```

```

        <td></td>
        <td><button class="btn"
            onmousedown="sendCmd('forward')" onmouseup="sendCmd('stop')"
            ontouchstart="sendCmd('forward')" ontouchend="sendCmd('stop')">▲</button></td>
    <td></td>
</tr>
<tr>
    <td><button class="btn"
        onmousedown="sendCmd('left')" onmouseup="sendCmd('stop')"
        ontouchstart="sendCmd('left')" ontouchend="sendCmd('stop')">◀</button></td>
    <td><button class="btn btn-stop" onclick="sendCmd('stop')">■</button></td>
    <td><button class="btn"
        onmousedown="sendCmd('right')" onmouseup="sendCmd('stop')"
        ontouchstart="sendCmd('right')" ontouchend="sendCmd('stop')">▶</button></td>
</tr>
<tr>
    <td></td>
    <td><button class="btn"
        onmousedown="sendCmd('backward')" onmouseup="sendCmd('stop')"
        ontouchstart="sendCmd('backward')" ontouchend="sendCmd('stop')">▼</button></td>
    <td></td>
</tr>
</table>
</body>
</html>

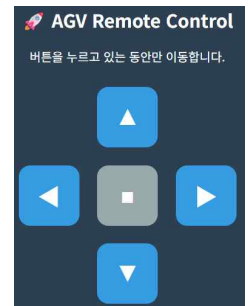
```

3) 실행 및 무선 주행 테스트

코드를 실행한 뒤 스마트폰으로 접속하면 화면에 십자 모양의 조이스틱 버튼이 나타납니다.

스마트폰 화면의 '▲(전진)' 버튼을 꺾 누르고 있어 보세요. 누르고 있는 동안만 AGV가 앞으로 달려 나가고, 화면에서 손가락을 떼는 순간 즉시 정지합니다.

단순한 까만색 터미널 창을 벗어나, 내가 짠 파이썬 코드와 스마트폰이 무선으로 연결되어 무거운 하드웨어를 움직이는 이 순간이 바로 진정한 피지컬 AI 엣지 컴퓨팅의 묘미입니다.



4. 1인칭 주행 영상(FPV) 스트리밍 : 이동형 CCTV

방금 6.3절에서 우리는 스마트폰 버튼으로 AGV를 조종하는 데 성공했습니다. 하지만 로봇이 내 시야에서 벗어나 벽 뒤로 넘어가 버리면 어떻게 될까요? 보이지 않는 로봇을 조종하는 것은 불가능합니다. 원격 제어(Teleoperation)의 완성은 결국 '눈(Vision)'입니다. AGV에 장착된 카메라 영상을 스마트폰 웹 화면에 실시간으로 띄워, 로봇을 직접 보지 않고 스마트폰 화면만 보면서 운전할 수 있는 FPV(First-Person View, 1인칭 시점) 시스템을 구축해 보겠습니다. 이것이 바로 공장 곳곳을 누비는 '이동형 CCTV'의 원리입니다.

1) 영상 스트리밍의 원리 : 초고속 플립북(MJPEG)

유튜브나 넷플릭스처럼 복잡한 영상 압축 기술을 라즈베리파이에서 구현하려면 시스템이 너무 무거워집니다. 대신 우리는 가장 가볍고 직관적인 MJPEG(Motion JPEG) 방식을 사용합니다. 원리는 간단합니다. 카메라가 찍은 사진(JPEG)을 1초에 10~20장씩 스마트폰으로 쉴 새 없이 쏘아 보내는 것입니다. 어릴 적 책장 모서리에 그림을 그려놓고 후루룩 넘기면 움직이는 것처럼 보이던 '플립북(Flip Book)' 애니메이션과 정확히 같은 원리입니다.

2) [실습] 스마트폰에 실시간 카메라 영상 띄우기

아래 코드는 AGV 카메라의 영상을 스마트폰 웹 브라우저로 끊임없이 전송해 주는 스트리밍 서버 코드입니다.

```
# file name : flask_fpv_camera.py
# Flask 기반 실시간 FPV 카메라 스트리밍 서버

import cv2
from flask import Flask, Response, render_template_string
from picamera2 import Picamera2

# 1. 카메라 초기화 (빠른 전송을 위해 해상도를 640x480으로 설정)
print("[System] 카메라 초기화 중...")
picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(main={"size": (640, 480)}))
picam2.start()

# 2. Flask 서버 초기화
app = Flask(__name__)

# 3. 사용자 화면(HTML) 구성
# <img> 태그의 src(출처)를 일반 이미지 파일이 아닌 '/video_feed' 주소로 연결합니다.
HTML_VIDEO = """
<!DOCTYPE html>
<html>
<head>
  <title>AGV FPV Camera</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <style>
    body { text-align: center; background-color: #2c3e50; color: white; margin-top: 20px;
font-family: sans-serif; }
    img { width: 90%; max-width: 640px; border: 5px solid #34495e; border-radius: 10px; }
    .recording { color: #e74c3c; font-weight: bold; animation: blink 1s step-start 0s
infinite; }
  </style>
</head>
</html>

```

```

        @keyframes blink { 50% { opacity: 0.0; } }
    </style>
</head>
<body>
    <h2>□ AGV 1인칭 관제 화면</h2>
    <p class="recording">● LIVE STREAMING</p>

    <!-- 이 이미지 태그가 파이썬이 보내주는 영상을 끊임없이 받아옵니다 -->
    
</body>
</html>
"""

# 4. 카메라 프레임 생성 엔진 (플립북 제작소)
def generate_frames():
    """카메라가 찍은 사진을 JPEG로 압축하여 웹 브라우저로 끊임없이 밀어내는(yield) 함수"""
    while True:
        # 카메라에서 한 장면(Frame) 찰칵!
        frame = picam2.capture_array()

        # 이미지를 전송하기 쉬운 JPEG 포맷으로 압축
        ret, buffer = cv2.imencode('.jpg', frame)
        frame_bytes = buffer.tobytes()

        # MJPEG 포맷 규격에 맞게 포장하여 브라우저로 전송 (yield: 끊기지 않고 계속 보냄)
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')

# 5. 접속 라우팅
@app.route('/')
def index():
    """스마트폰 접속 시 HTML 화면 제공"""
    return render_template_string(HTML_VIDEO)

@app.route('/video_feed')
def video_feed():
    """HTML의 <img> 태그가 영상 데이터를 달라고 요청하는 주소"""
    # 일반 텍스트(Response)가 아닌, 계속해서 이어지는 다중 파트(multipart) 데이터임을 명시
    return Response(generate_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')

# 6. 서버 가동
if __name__ == '__main__':
    print("=====")
    print(" □ FPV 영상 스트리밍 서버가 가동되었습니다.")
    print(" 스마트폰 주소창에 아래 주소를 입력하세요.")
    print(" □ http://자신의IP주소:5000")
    print("=====")

    try:

```

```
app.run(host='0.0.0.0', port=5000)
finally:
    picam2.stop()
    print("카메라를 안전하게 종료했습니다.")
```

3) 실행 및 결과 확인

코드를 실행하고 스마트폰으로 접속하면, 화면 상단에 붉은색 '● LIVE STREAMING' 글씨가 깜빡이며 로봇의 시점이 실시간으로 중계됩니다. 스마트폰을 들고 라즈베리파이 렌즈 앞으로 손을 흔들어 보세요. 약간의 딜레이(지연)는 발생할 수 있지만, 현장의 생생한 모습이 내 손안의 웹 브라우저로 전달되는 것을 확인할 수 있습니다.



□ Tip. 스트리밍 딜레이 현상과 해상도의 관계

실습실 환경(공유기의 와이파이 대역폭)에 따라 카메라 영상이 1~2초가량 늦게 뜨는 '랙(Lag)' 현상이 발생할 수 있습니다. 무거운 이미지 수십 장이 비좁은 무선망을 통과하며 생기는 병목 현상입니다.

영상이 너무 끊긴다면, 코드 1번 항목의 해상도를 (640, 480)에서 (320, 240)으로 확 낮춰 보세요. 화질은 투박해지지만 데이터 전송량이 4분의 1로 줄어들어 영상이 눈에 띄게 부드러워집니다. 이것이 바로 산업 현장에서 통신 속도와 데이터 크기를 조율하는 '네트워크 최적화'의 기본입니다.

Chapter 7. 통합 스마트 팩토리 시스템 구현

최종적으로 공장 환경 + AGV 이송(모빌리티) + 비전 품질 검수(OpenCV+YOLO) → 웹 관제 대시보드에서 확인할 수 있는 통합 스마트 팩토리 시스템을 구현하겠습니다.

1. 융합의 시작 : 통합 시스템 아키텍처 설계

스마트 팩토리는 단일 기계로 완성되지 않습니다. 수십 대의 로봇, 수백 개의 센서, 그리고 이를 통제하는 메인 서버가 유기적으로 데이터를 주고받는 거대한 신경망입니다. 3일간의 특강을 마무리하는 이번 장에서는 두 명이 한 조가 되어 각자의 라즈베리파이를 활용해 실제 공장과 동일한 구조의 '마이크로 스마트 팩토리(Micro Smart Factory)'를 통합 구축합니다.

1) 왜 역할을 나누는가? : 분산 제어(Distributed Edge Computing)의 이해

지금까지 우리는 하나의 라즈베리파이 안에서 모터도 돌리고, 초음파 센서도 읽고, 카메라도 켜습니다. 하지만 여기에 무거운 인공지능(YOLO) 시각 처리와 웹 서버까지 동시에 구동하면 어떻게 될까요? CPU의 연산 한계(과부하)로 인해 영상이 끊기거나, 앞차를 발견해도 브레이크(AEB)를 제때 밟지 못해 충돌하는 대형 사고가 발생합니다. 실제 산업 현장에서도 하나의 중앙 컴퓨터가 모든 설비를 제어하지 않습니다. 각 설비 옆에 위치한 엣지 디바이스(Edge Device)들이 자신의 임무만 독립적으로 처리하고, 그 결과 데이터만 중앙 관제 서버로 보내는 '분산 제어(Distributed Edge Computing)' 방식을 사용합니다. 우리의 실습도 이 원칙을 철저히 따라, 주행과 관제를 담당하는 엣지와 비전 검수를 담당하는 엣지로 역할을 완벽하게 분리합니다.

2) 2인 1조 역할과 책임 정의 : IT 파트 vs OT 파트

각각 산업 현장의 IT(정보 기술) 파트와 OT(현장 운영 기술) 파트로 역할을 나누어 협업합니다.

구분	IT / 모빌리티 및 관제 파트	OT / 현장 비전 및 환경 파트
핵심 역할	공장의 중앙 관제(Control Tower) 및 물류 이송	공장 라인의 품질 검수 및 환경 모니터링
하드웨어	라즈베리파이 + AGV + 카메라	라즈베리파이 + 카메라 + DHT11(온습도) + LCD
소프트웨어 임무	1. AGV 충돌 방지 자율주행 (AEB) 2. 1인칭 주행 영상(FPV) 웹 스트리밍 3. Flask 기반 통합 관제 웹 대시보드 호스팅	1. 컨베이어 라인 비전 품질 검수 (YOLO / 색상) 2. 작업장 온습도 측정 및 현장 디스플레이 출력 3. 수강생 A의 웹 서버로 검수 및 환경 데이터 전송

3) 엣지와 엣지가 대화하는 법 : 동일 네트워크와 HTTP 통신

두 대의 라즈베리파이가 데이터를 주고받으려면 '통신망'과 '대화 규칙'이 필요합니다.

① 동일 네트워크 구성 (물리적 연결) :

3장에서 다루었던 '모바일 핫스팟'이 여기서 빛을 발합니다. 각 조의 라즈베리파이는 반드시 같은 와이파이(핫스팟) 이름으로 접속되어 있어야 합니다. 그래야 서로의 IP 주소를 찾아갈 수 있습니다.

② 통신 기초 (HTTP POST/GET) :

우리는 인터넷 웹 서핑을 할 때 쓰는 가장 대중적이고 안정적인 HTTP 방식을 사용합니다.

- ☐ GET : 조원 A(서버)가 스마트폰 웹 브라우저에 공장의 현재 상태를 보여줄 때 사용합니다. ("화면 띄워줘")
- ☐ POST : 조원 B(클라이언트)가 불량품을 발견하거나 온도를 측정했을 때, 조원 A의 라즈베리파이 IP 주소로 데이터를 쏘아 보낼 때 사용합니다. ("여기 불량품 1개 추가, 현재 온도 25도 저장해 줘")

□ Tip : 웹 대시보드 접속 시 주의사항

라즈베리파이 A(서버)와 B(클라이언트)가 동일한 모바일 핫스팟(예: 조원 A의 스마트폰 혹은 노트북의 핫스팟)에 연결되어 있어야 데이터가 오갈 수 있습니다.

또한, 완성된 관제 화면을 스마트폰 웹 브라우저로 보려면, 해당 스마트폰 역시 라즈베리파이들과 같은 와이파이(또는 핫스팟을 제공 중인 기기 자체) 네트워크 안에 있어야 합니다. 만약 스마트폰의 와이파이를 끄고 5G/LTE망으로 접속을 시도하면 내부망(192.168.x.x)에 있는 라즈베리파이 서버를 찾을 수 없어 접속 에러가 발생합니다.

2. [조원 B] 현장(OT) 파트 : AI 비전 검수 및 공장 환경 모니터링

1) 엣지 노드(Edge Node)의 임무

- ① 조원 B는 스마트 팩토리 현장에 고정되어 컨베이어 벨트를 감시하는 '엣지 노드(Edge Node)' 역할을 맡습니다.
- ② 여러분의 라즈베리파이는 쉴 새 없이 공장의 온도와 습도를 체크하고(DHT11), 카메라를 통해 지나가는 제품의 불량 여부(YOLO)를 검사합니다. 그리고 이 귀중한 데이터를 1초마다 조원 A(관제 서버)에게 보고해야 합니다.

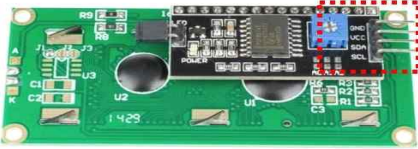
2) 통신 준비 : 파트너(조원 A)의 IP 주소 확인하기

- ① 데이터를 보내려면 받는 사람의 주소가 필요하겠죠? 조원 A의 라즈베리파이 터미널에서 ifconfig 명령어를 입력하여 무선랜(wlan0)의 IP 주소(예: 192.168.137.50)를 확인한 뒤, 메모해 둡니다.
- ② 그리고 원활한 통신을 위해 파이썬의 대표적인 통신 라이브러리인 requests를 가상환경에 설치합니다.

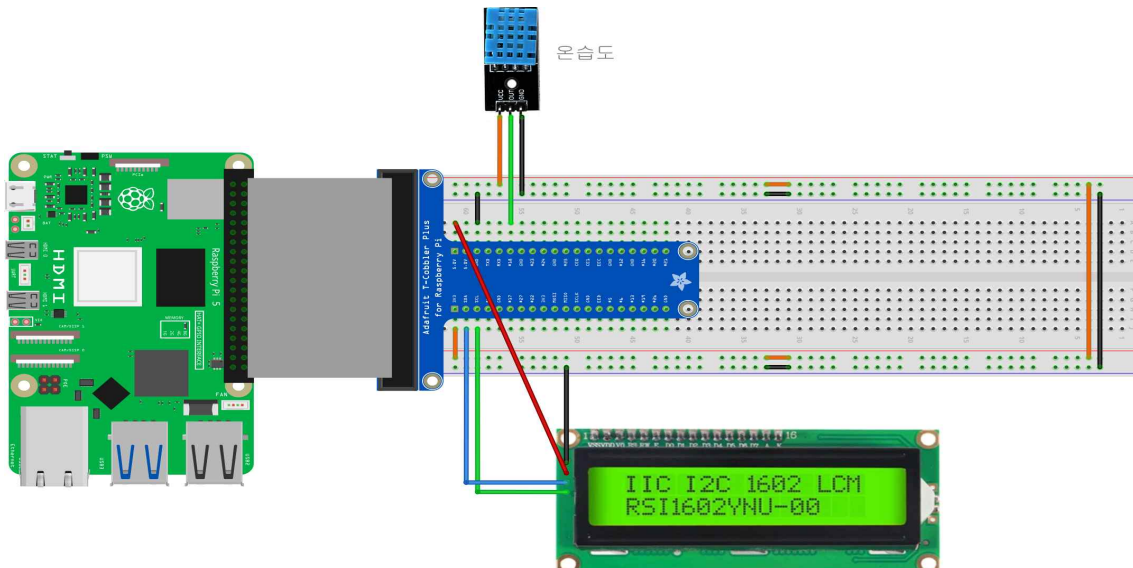
```
(smart_Factory_venv) $ pip install requests
```

3) 고정형 검수 스테이션 구축 및 센서(온습도 DHT11, I2C LCD) 통합

① 연결정보

sensor		라즈베리파이
	-GND	GND
	-VCC	5V
	-SDA	GPIO 2
	-SCL	GPIO 3

② 회로도



4) 컨베이어 벨트 위의 눈 : YOLO(또는 색상 추적)를 활용한 실시간 불량품 판별

통합 코드를 작성하기 전에, 비전 검수(OT) 파트의 핵심인 인공지능 시력 테스트를 먼저 진행합니다. 앞선 4장에서는 YOLO 모델이 사물을 찾아 화면에 네모(바운딩 박스)를 그려주는 것까지만 확인했습니다. 이번에는 한 발 더 나아가, 화면에 특정 사물(예: 스마트폰)이 등장하면 이를 '불량품(Defect)'으로 규정하고 개수를 카운트하는 실전 판별 로직을 추가합니다.

① 비전 기반 불량품 카운터

```
# file name : test_vision_inspector.py
# YOLOv11 모델을 활용한 특정 객체(불량품) 감지 및 카운팅

import cv2
from picamera2 import Picamera2
from ultralytics import YOLO

# 1. 카메라 초기화 (연산 속도를 위해 640x480 해상도 사용)
picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(main={"size": (640, 480)}))
picam2.start()

# 2. 사전 학습된 인공지능 두뇌 불러오기
model = YOLO('yolo11n.pt')

defect_count = 0 # 불량품을 발견할 때마다 1씩 증가할 변수

print("=====")
print("비전 검수 시스템 단독 테스트 시작")
print("카메라 렌즈 앞에 '스마트폰'을 보여주세요.")
print("=====")

cv2.namedWindow("Vision Inspector", cv2.WINDOW_AUTOSIZE)

try:
    while True:
        # 3. 실시간 프레임 캡처 및 AI 추론
        frame = picam2.capture_array()
```

```

# conf=0.6 : 60% 이상 확신할 때만 인식하도록 기준을 높여 오작동 방지
results = model.predict(source=frame, conf=0.6, verbose=False)

# 4. 분석 결과(바운딩 박스)가 덧그려진 도화지(이미지) 생성
annotated_frame = results[0].plot()

# 5. [핵심] 화면에 인식된 사물들의 '이름'만 뽑아서 리스트로 만들기
# 예: ['person', 'cell phone', 'bottle']
detected_classes = [model.names[int(box.cls)] for box in results[0].boxes]

# 6. 불량품 판별 로직 (스마트폰을 불량품으로 가정)
if 'cell phone' in detected_classes:
    defect_count += 1

# 빨간색 굵은 글씨로 화면에 경고 문구 출력
cv2.putText(annotated_frame, "DEFECT DETECTED!", (30, 50),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 0, 255), 3)

print(f"□ 불량품 감지! (현재까지 누적: {defect_count}개)")

# 7. 모니터에 최종 화면 출력
cv2.imshow("Vision Inspector", annotated_frame)

# 'q' 키를 누르면 종료
if cv2.waitKey(1) & 0xFF == ord('q'):
    print("비전 검수 테스트를 종료합니다.")
    break

except Exception as err:
    print(f"오류 발생 : {err}")

finally:
    cv2.destroyAllWindows()
    picam2.stop()

```

영문 클래스명 주의:

파이썬 코드에서 조건문을 걸 때 반드시 COCO 데이터셋에 등록된 공식 영문 명칭인 'cell phone'(가운데 띄어쓰기 포함)으로 적어주셔야만 정확히 필터링이 됩니다. 스마트폰뿐만 아니라 본인의 얼굴(person)이나 마우스(mouse), 물병(bottle) 등을 카메라에 비춰보며 80가지 사물을 찾는 재미에 푹 빠지게 될 것입니다.

5) Source Code : 현장 엣지 통합 코드

```

# file name : ot_edge_B.py
# 현장(OT) 파트 : 비전 검수, 환경 모니터링 및 서버 데이터 전송

import cv2
import requests
import time
from picamera2 import Picamera2
from ultralytics import YOLO
import board
import adafruit_dht # (수정) 구형 Adafruit_DHT 대신 최신 CircuitPython 라이브러리 사용

```



```

from RPLCD.i2c import CharLCD

# 1. 서버 주소 설정 (조원 A의 라즈베리파이 IP 주소로 반드시 변경하세요!)
SERVER_URL = 'http://192.168.137.50:5000/api/update'

# 2. 하드웨어 및 AI 초기화
print('[System] 하드웨어 및 AI 모델 초기화 중...')

# DHT11 센서 초기화 (최신 라즈베리파이 5 Bookworm 환경 적용)
dht_device = adafruit_dht.DHT11(board.D21, use_pulseio=False)

# I2C LCD 초기화
lcd = CharLCD(i2c_expander='PCF8574',
              address=0x27,
              port=1,
              cols=16,
              rows=2,
              charmap='A00')
lcd.clear()
lcd.backlight_enabled = True

# 카메라 초기화
picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(main={"size": (640, 480)}))
picam2.start()

# 사전 학습된 가벼운 YOLO nano 모델 불러오기
model = YOLO('yolo11n.pt')
defect_count = 0          # 불량품 누적 카운트 변수

cv2.namedWindow('OT Vision Station', cv2.WINDOW_AUTOSIZE)
print('[System] 현장 엣지 노드 가동을 시작합니다.')

# 안정적인 통신을 위한 초기 온습도값 세팅
temperature = 25.0
humidity = 50.0

try:
    while True:
        # 3. 환경 데이터 읽기 (에러가 나도 프로그램이 멈추지 않도록 예외 처리)
        try:
            temp = dht_device.temperature
            hum = dht_device.humidity
            if temp is not None and hum is not None:
                temperature = temp
                humidity = hum
        except RuntimeError as error:
            # DHT11은 타이밍 이슈로 읽기 에러가 자주 발생하므로, 에러 발생 시 무시하고 넘어감
            pass

        # 4. LCD 출력 (화면 깜빡임과 밀림을 방지하기 위해 커서를 맨 앞으로 초기화)
        lcd.cursor_pos = (0, 0)
        # 이전 글자 잔상을 지우기 위해 끝에 여백을 살짝 둡니다.
        lcd.write_string(f'Temp: {temperature:.1f}C   \nHumid: {humidity:.1f}%   ')

        # 5. 카메라 프레임 캡처 및 YOLO 추론
        frame = picam2.capture_array()

```

```

results = model.predict(source=frame, conf=0.6, verbose=False)
annotated_frame = results[0].plot()

# 6. 불량품 판별 로직
detected_classes = [model.names[int(box.cls)] for box in results[0].boxes]

if 'cell phone' in detected_classes:
    defect_count += 1
    cv2.putText(annotated_frame, "DEFECT!", (50, 50), cv2.FONT_HERSHEY_SIMPLEX, 1,
(0, 0, 255), 3)
    print(f'불량품 감지! (누적: {defect_count}개)')

# 7. 관제 서버(조원 A)로 데이터 전송 (POST 요청)
payload = {
    "temperature": temperature,
    "humidity": humidity,
    "defect_count": defect_count
}

try:
    response = requests.post(SERVER_URL, json=payload, timeout=0.5)
except requests.exceptions.RequestException:
    # 화면이 스크롤되지 않도록 end='\r' 처리
    print('서버와 통신 실패 (관제 서버 상태를 확인하세요)', end='\r')

# 8. 현장 모니터(OpenCV 창) 출력
cv2.imshow('OT Vision Station', annotated_frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    print('\n현장 엡지 노드를 종료합니다.')
    break

except Exception as err:
    print(f'시스템 에러 : {err}')

finally:
    cv2.destroyAllWindows()
    picam2.stop()
    lcd.clear()
    dht_device.exit() # DHT 센서 자원 안전 반환

```

3. [조원 A] 관제(IT) 파트 : 통합 대시보드 및 AGV 원격 제어

1) 수동 조작과 자동 안전의 결합 : 원격 제어(Teleoperation)와 긴급 제동(AEB)

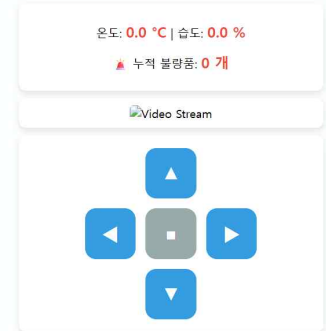
조원 A는 공장의 심장인 '중앙 관제 서버'를 운영함과 동시에, 물류를 나르는 AGV의 운전대를 잡습니다. 이번 실습의 핵심은 6장에서 배운 '스마트폰 조이스틱'으로 AGV를 직접 운전하면서, 4장에서 배운 '초음파 센서 긴급 제동(AEB)'이 백그라운드에서 동시에 작동하도록 융합하는 것입니다. 관리자가 실수로 전진 버튼을 계속 누르고 있더라도, 로봇 앞 20cm 이내에 장애물이 감지되면 초음파 센서가 조종 신호를 무시하고 강제로 브레이크를 밟습니다. 이것이 최신 산업용 무인 지게차(AMR)와 자율주행차(ADAS)의 핵심 안전 기술입니다.

2) 대통합의 장 : Flask 기반 마스터 대시보드 구축

6장에서 개별적으로 실습했던 ①실시간 FPV 카메라 영상, ②파트너의 데이터 수신 대시보드, ③AGV 조종 조이스틱 코드를 파이썬 파일 단 한 개로 모두 합쳐냅니다.

조원 A의 스마트폰은 이제 공장의 모든 상황을 보고, 받고, 조종할 수 있는 무적의 '마스터 컨트롤러'가 됩니다.

스마트 팩토리 통합 시스템 마스터 대시보드



3) [실습] 조원 A 마스터 코드 실행 : it_control_A.py

아래 코드는 조원 A가 실행할 최종 통합 코드입니다. 조원 B가 보내는 데이터(POST)를 받고, 카메라 영상을 쏴주며, 안전하게 로봇을 움직이는 모든 로직이 포함되어 있습니다.

```
# file name : it_control_A.py
# 관제(IT) 파트 : Flask 웹 서버, AGV 자율주행(AEB) 및 실시간 스트리밍 통합

from flask import Flask, Response, request, jsonify, render_template_string
import cv2
from picamera2 import Picamera2
from gpiozero import Motor, DistanceSensor
from time import sleep

# 1. 전역 변수 : 조원 B(현장)로부터 받을 데이터를 저장할 공간
factory_status = {
    "temperature" : 0.0,
    "humidity" : 0.0,
    "defect_count" : 0
}

# 2. 하드웨어 초기화 (카메라 및 AGV 모터/센서)
picam2 = Picamera2()
picam2.configure(picam2.create_preview_configuration(main={"size" : (640, 480)}))
picam2.start()

motor1 = Motor(26, 19)
motor2 = Motor(27, 22)
motor3 = Motor(20, 21)
motor4 = Motor(24, 23)
```

```

front_ultra = DistanceSensor(echo=12, trigger=13, threshold_distance=0.2)

def stop_agv():
    motor1.stop(); motor2.stop(); motor3.stop(); motor4.stop()
    print('[AEB 작동] 전방 장애물 감지! AGV 정지')

def move_agv():
    motor1.forward(0.5); motor2.forward(0.5); motor3.forward(0.5); motor4.forward(0.5)
    print('[주행 재개] 안전 확보. 전진합니다.')

# 이벤트 구동 방식 연결 (웹 서버와 충돌 없이 독립적으로 작동)
front_ultra.when_in_range = stop_agv
front_ultra.when_out_of_range = move_agv

# 3. Flask 웹 서버 설정
app = Flask(__name__)

# --- [웹 대시보드 UI (HTML/JS 템플릿)] ---
# 복잡한 파일 구성 없이 1개의 파일로 실행되도록 HTML을 파이썬 안에 품었습니다.
HTML_TEMPLATE = """
<!DOCTYPE html>
<html>
<head>
    <title>마이크로 스마트 팩토리 대시보드</title>
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <style>
        body { font-family: 'Malgun Gothic', sans-serif; background-color: #f4f7f6;
text-align: center; }
        h1 { color: #2c3e50; }
        .dashboard { display: flex; flex-wrap: wrap; justify-content: center; gap: 20px;
padding: 20px; }
        .card { background: white; padding: 20px; border-radius: 10px; box-shadow: 0 4px 8px
rgba(0,0,0,0.1); min-width: 300px; }
        .data-text { font-size: 24px; font-weight: bold; color: #e74c3c; }
        img { max-width: 100%; border-radius: 5px; }
    </style>
</script>
    // 1초마다 조원 B의 최신 데이터를 가져와서 화면을 몰래(비동기로) 새로고침 합니다.
    setInterval(() => {
        fetch('/api/status')
            .then(response => response.json())
            .then(data => {
                document.getElementById('temp').innerText = data.temperature.toFixed(1) +
' °C';
                document.getElementById('humid').innerText = data.humidity.toFixed(1) + '
%';
                document.getElementById('defect').innerText = data.defect_count + ' 개';
            });
    }, 1000);

```

```

    </script>
</head>
<body>
    <h1>□ 통합 관제 대시보드</h1>
    <div class="dashboard">
        <div class="card">
            <h3>□ AGV 1인칭 주행 영상</h3>
            
        </div>
        <div class="card">
            <h3>□ 공장 현장 (OT) 데이터 보고</h3>
            <p>현재 온도 : <span id="temp" class="data-text">0.0 °C</span></p>
            <p>현재 습도 : <span id="humid" class="data-text">0.0 %</span></p>
            <hr>
            <p>□ 누적 불량품 : <span id="defect" class="data-text">0 개</span></p>
        </div>
    </div>
</body>
</html>
"""

# --- [Flask 라우팅 (경로 설정)] ---

@app.route('/')
def index():
    """스마트폰으로 접속 시 가장 먼저 보여지는 메인 화면(HTML)을 렌더링합니다."""
    return render_template_string(HTML_TEMPLATE)

def generate_frames():
    """카메라 영상을 스마트폰으로 끊임없이 보내주는(스트리밍) 엔진 역할"""
    while True:
        frame = picam2.capture_array()
        ret, buffer = cv2.imencode('.jpg', frame)
        frame_bytes = buffer.tobytes()
        # MJPEG 포맷으로 이미지들을 하나로 묶어 송신
        yield (b'--frame\r\n'
              b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')

@app.route('/video_feed')
def video_feed():
    """HTML의 <img> 태그가 이 주소를 호출하여 영상을 받아갑니다."""
    return Response(generate_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')

@app.route('/api/update', methods=['POST'])
def update_data():
    """조원 B(현장)가 requests.post()로 쏘아 보낸 데이터를 받아 저장합니다."""
    data = request.get_json()
    factory_status.update(data)
    print(f"□ [데이터 수신] 온도:{data['temperature']} / 불량:{data['defect_count']}개")

```

```

    return jsonify({"status" : "success"})

@app.route('/api/status', methods=['GET'])
def get_status():
    """스마트폰(웹 브라우저)이 1초마다 최신 데이터를 가져갈 수 있도록 제공합니다."""
    return jsonify(factory_status)

# 4. 시스템 가동 시작
if __name__ == '__main__':
    print("=====")
    print(" □ 스마트 팩토리 통합 시스템 서버 가동을 시작합니다.")
    print(" 스마트폰 웹 브라우저를 열고 아래 주소로 접속하세요.")
    print(" □ 주소 : http://자신의IP주소:5000")
    print("=====")

    # AGV 최초 출발
    move_agv()

    # Flask 웹 서버 실행 (0.0.0.0 : 외부 접속 허용)
    try:
        app.run(host='0.0.0.0', port=5000, threaded=True)
    except KeyboardInterrupt:
        pass
    finally:
        stop_agv()
        picam2.stop()
        print('관제 서버를 안전하게 종료했습니다.')

```

4. [통합 실습] 마이크로 스마트 팩토리, 시스템 가동!

드디어 모든 톱니바퀴가 준비되었습니다. 조원 A가 만든 '관제/모빌리티 서버'와 조원 B가 만든 '현장/비전 엣지 노드'를 하나로 연결하여 완벽한 마이크로 스마트 팩토리를 가동해 봅니다.

1) 접속 준비 : 두 대의 라즈베리파이 네트워크 연동 및 IP 주소 교환

서로 데이터를 주고받기 위한 최종 점검 단계입니다.

- ① 네트워크 통일 : 조원 A와 조원 B의 라즈베리파이, 그리고 결과를 확인할 스마트폰이 모두 똑같은 와이파이(또는 모바일 핫스팟)에 연결되어 있는지 다시 한번 확인합니다. 스마트폰의 5G/LTE 데이터는 잠시 꺼두는 것이 좋습니다.
- ② IP 주소 교환 : 조원 A는 터미널에 ifconfig를 입력하여 자신의 IP 주소를 확인하고 조원 B에게 알려줍니다. (예 : 192.168.137.50)
- ③ 코드 주소 수정 : 조원 B는 자신의 파이썬 코드(ot_edge_B.py) 맨 위쪽에 있는 SERVER_URL 주소에 조원 A의 IP를 정확히 기입하고 저장합니다.

2) 시스템 런칭 : 스마트폰 브라우저로 접속하는 우리 조만의 공장 관제 화면

자, 이제 서버의 스위치를 켜 시간입니다! 실행 순서가 매우 중요합니다.

- ① [조원 A] 서버 가동 : 먼저 조원 A가 it_control_A.py를 실행합니다. AGV가 움직일 준비를 마치고, "관제 시스템 서버 가동을 시작합니다"라는 문구가 터미널에 뜹니다.
- ② [조원 B] 엣지 노드 가동 : 조원 A의 서버가 열린 것을 확인한 후, 조원 B가 ot_edge_B.py를 실행합니다. 카메라 렌즈가 열리며 온도 데이터 전송을 시작합니다.
- ③ [조원 모두] 대시보드 접속 : 각자의 스마트폰을 꺼내어 웹 브라우저(크롬, 사파리 등)를 엽니다. 주소창에 http://조원A의IP주소:5000을 입력하고 접속합니다. 화면에 화려한 대시보드가 나타나면 시스템 런칭 성공입니다!

3) 융합 시나리오 구동 : AGV 물류 이송과 비전 품질 검수의 완벽한 양상들

스마트폰 화면을 띄워둔 채로, 두 조원이 협력하여 실제 스마트 팩토리 시나리오를 구동해 봅니다.

① [Mission 1] 조원 A의 FPV 물류 이송 배달 :

조원 A는 AGV를 직접 쳐다보지 않고, 오직 스마트폰의 카메라 영상(FPV)만 보면서 조이스틱 버튼을 눌러 조원 B의 '검수 스테이션' 앞까지 AGV를 운전하여 이동시킵니다.

② [Mission 2] 안전 최우선, AEB 테스트 :

이동 중, AGV의 앞을 손이나 장애물로 가로막아 봅니다. 조원 A가 스마트폰의 '전진(▲)' 버튼을 계속 누르고 있더라도 로봇이 스스로 위험을 인지하고 브레이크를 밟아 제자리에 멈춰야(AEB 작동) 합니다. 장애물을 치우면 다시 전진이 가능해집니다.

③ [Mission 3] 조원 B의 AI 비전 검수 및 데이터 동기화 :

AGV가 무사히 도착하면, 조원 B는 카메라 렌즈 앞에 정상 제품과 불량품(스마트폰)을 차례로 비춥니다. AI(YOLO)가 불량품을 판별해 내는 그 즉시, 멀리 떨어져 있는 조원 A의 스마트폰 대시보드에 '누적 불량품' 개수가 실시간으로 1개 올라가는 것을 확인합니다!

4) 현장 트러블슈팅(Troubleshooting) : 데이터 통신 지연 및 스트리밍 끊김 해결법

실제 공장처럼 30명이 넘는 수강생이 한정된 공간에서 무선 데이터를 주고받다 보면 병목 현상이 발생할 수 있습니다. 당황하지 말고 아래의 방법으로 해결해 보세요.

① Q1. "스마트폰 대시보드에 접속할 수 없음(사이트에 연결할 수 없음)이 뜹니다."

☐ 원인 : 스마트폰이 실습용 핫스팟이 아닌 외부망(LTE/5G)에 연결되어 있거나, IP 주소 입력 시 포트 번호(:5000)를 빼먹은 경우입니다.

☐ 해결 : 스마트폰의 셀룰러 데이터를 끄고 Wi-Fi 연결을 재확인하세요.

② Q2. "조원 B 터미널에 '서버와 통신 실패'가 계속 뜹니다."

☐ 원인 : 조원 A의 서버 코드가 아직 실행되지 않았거나 에러로 꺼진 경우입니다.

☐ 해결 : 조원 A의 터미널을 확인하여 코드를 재시작하고, 조원 B의 코드 내 SERVER_URL 오타(IP 주소 틀림)가 없는지 확인합니다.

③ Q3. "AGV 카메라 스트리밍 영상이 너무 심하게 끊깁니다 (Lag 발생)."

☐ 원인 : 강의실 전체의 와이파이 대역폭이 꽉 차서 발생하는 네트워크 지연 현상입니다.

☐ 해결 : 실제 산업 현장에서는 이 문제를 해결하기 위해 '5G 특화망(이음 5G)'을 구축합니다. 실습 시에는 조원 A 코드의 카메라 해상도를 (640, 480)에서 (320, 240)으로 낮추면 훨씬 부드럽게 전송됩니다.

💡 Tip : 에필로그(5.5절)로 이어지는 자연스러운 브릿지 멘트

"여러분, 방금 네트워크가 끊겨서 화면이 버벅거리는 것을 경험하셨죠? 바로 이것이 현업 대기업들이 수백억 원을 들여 공장 안에 5G 전용망(이음5G)을 깔고, 중앙 서버가 아닌 로봇 단말기(엣지)에 직접 인공지능을 넣는 이유입니다. 통신이 지연되어 브레이크를 제때 못 밟으면 사고가 나기 때문이죠. 오늘 여러분이 겪은 이 짧은 끊김이, 현대 스마트 팩토리가 엣지 컴퓨팅을 도입하는 가장 강력한 당위성입니다."

5. 에필로그 : 피지컬 AI가 바꿀 우리의 산업 현장

1) 장난감을 넘어 현장으로 : 스마트 팩토리 실무 적용 아이디어 발굴

여러분은 이런 의문을 가질 수 있습니다.

"라즈베리파이와 레고 같은 장난감 RC카로 만든 이 시스템이, 과연 진짜 공장에 적용될 수 있을까?"
결론부터 말씀드리면, '하드웨어의 스케일'만 다를 뿐 '소프트웨어의 뼈대(Architecture)'는 글로벌 최고 수준의 스마트 팩토리와 정확히 일치합니다. 여러분이 작성한 파이썬 코드의 논리 구조를 그대로 둔 채 하드웨어만 산업용으로 교체하면, 그것이 바로 수억 원짜리 자동화 설비가 됩니다.
우리가 만든 '마이크로 스마트 팩토리'를 현업의 스케일로 확장해 보는 상상(Ideation)을 해봅시다.

① 아이디어 1. RC카 모터 ➡ 산업용 PLC와 자율주행 지게차(AMR)

우리가 사용한 손가락만 한 DC 모터 제어 코드를 산업용 서보모터(Servo Motor) 통신 프로토콜로 바꾸기만 하면? 여러분의 코드는 수 톤의 화물을 나르는 무인 지게차(AMR)의 자율주행 두뇌가 됩니다. 앞을 가로막으면 멈추던 초음파 센서는 산업용 라이다(LiDAR) 센서로 교체되어 더욱 정밀한 회피 기동을 수행할 것입니다.

② 아이디어 2. 라즈베리파이 카메라 ➡ 초고속 머신 비전(Machine Vision)

컨베이어 벨트의 속도가 빨라지면 라즈베리파이 카메라로는 잔상(Motion Blur)이 생깁니다. 이때 카메라를 초당 수백 장을 찍는 산업용 GigE 비전 카메라로 바꾸고, 연산 장비를 AI 전용 NPU(예: Nvidia Jetson)로 교체해 보세요. 여러분이 학습시킨 YOLO 모델은 1초에 수십 개의 불량품을 완벽하게 속아내는 현장의 매의 눈이 될 것입니다.

③ 아이디어 3. Flask 대시보드 ➡ 전사적 생산 관리 시스템(MES / ERP)

조원 B가 조원 A에게 온습도와 불량품 개수를 requests.post()로 쏘아 보냈던 통신 코드를 기억하시나요? 이 데이터의 도착지를 여러분 회사의 MES(생산 관리 시스템) 서버 데이터베이스로 연결해 보십시오. 현장의 모든 생산 및 불량 데이터가 실시간으로 임원진의 모니터에 집계되는 진정한 '데이터 기반 경영'이 실현됩니다.

□ 마무리 : 모니터를 뚫고 나온 인공지능, 피지컬 AI

지금까지 세상이 열광했던 챗GPT나 생성형 AI는 오직 '모니터 안'에서 텍스트와 이미지를 만들어내는 데 머물렀습니다. 하지만 산업 현장에서 진짜 필요한 혁신은 모니터 밖의 물리적 세상, 즉 기계를 움직이고, 센서로 현장을 읽어내며, 제품을 판별하는 '피지컬 AI(Physical AI)'입니다.

여러분은 파이썬 기초부터 시작해 하드웨어 센서를 제어하고, 인공지능 시각 모델(YOLO)을 다루었으며, 최종적으로 두 대의 엣지 디바이스를 하나로 통신시키는 거대한 여정을 완주해 냈습니다. 여러 메시지와 사투를 벌이고, 끊어지는 네트워크를 디버깅하며 흘린 땀방울은 여러분을 단순한 '기계 조작자'에서 현장의 디지털 전환(DX)을 이끄는 '설계자'로 성장시켰습니다.

오늘 여러분이 조립한 것은 작은 라즈베리파이 보드였지만, 내일 여러분이 설계할 것은 대한민국 산업 현장의 새로운 미래일 것입니다.

💡 Tip : 강의 피날레 멘트 제안

"여러분, 앞에 띄워진 이 화면들이 보이십니까? 까만 터미널 창만 쳐다보며 막막해하셨던 여러분이 스스로 만들어낸 우리 공장의 살아 숨 쉬는 데이터입니다. 현장에 돌아가셔도 지금 이 화면을 띄웠을 때의 짜릿함을 잊지 마시고, 현장의 작은 문제부터 하나씩 피지컬 AI로 혁신해 나가시길 응원합니다!"

